STARLET Utility Routines

Reference Manual

STARLET-RM-0171

STARLET Version 1.7.1 *** 31-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

StarLet is free software, distributed under the Apache License, Version 2.0. It comes with ABSOLUTELY NO WARRANTY.

Contents

Preface **6**
Intended Audience . . . 6
Document Structure . . . 7
Related Documents . . . 8
Changes in Version 1.7.1 . . . 9
Changes in Version 1.7.0 . . . 10
Trademarks . . . 11
Conventions . . . 12

Chapter 1. Introduction to the Routines **13**
1.1 Why These Routines . . . 13
1.2 The Condition-Value Convention . . . 15
1.3 Argument Types . . . 16
1.4 Naming . . . 17
1.5 Thread Safety . . . 18
1.6 How an Entry Reads . . . 19

Chapter 2. Utility Routines **20**
2.1 Condition Values and Messages . . . 21
STS$K_* – Severity Constants . . . 22
$DEFFAC, $DEFMSG, $DEFSEV – Build a Condition Value . . . 23
$FAC, $MSG, $SEV – Take a Condition Value Apart . . . 24
$STS_ERROR – Test for an Error . . . 25
UTIL$S_INVARG, UTIL$S_INQUE, UTIL$S_NOLOCK, UTIL$S_DUPFAC – Condition Values of the Library . . . 26
EMSG_RECORD, EMSG_RECORD_DESC – Message Catalogue Types . . . 27
__util$inimsg – Register a Message Catalogue . . . 28
__util$getmsg – Find a Message Record . . . 29
__util$putmsg – Signal a Message (printf Format) . . . 30
__util$putmsgd – Signal a Message with the Place of the Call . . . 31
__util$putmsg_fao – Signal a Message (FAO Control String) . . . 32
__util$putmsgd_fao – Signal a Message (FAO) with the Place of the Call . . . 33
$PUTMSG – Signal a Message (Macro) . . . 34
$PUTMSG_FAO – Signal a Message, FAO (Macro) . . . 35
2.2 Logging, Tracing and Output . . . 36
__util$log – Write a Log Record (printf Format) . . . 37
__util$logd – Write a Log Record with the Place of the Call . . . 38
__util$log_fao – Write a Log Record (FAO Control String) . . . 39
__util$logd_fao – Write a Log Record (FAO) with the Place of the Call . . . 40
$LOG – Write a Log Record (Macro) . . . 41
$LOG_FAO – Write a Log Record, FAO (Macro) . . . 42
$IFLOG – Write a Log Record if a Flag is Set (Macro) . . . 43
__util$log2buf – Format a Log Record into a Buffer . . . 44
__util$out – Write a Line to the Log Descriptor . . . 45
__util$deflog – Direct the Log to a File and a Syslog Host . . . 46
__util$setlogfd – Direct the Log to an Open Descriptor . . . 47

__util$rewindlogfile – Truncate the Log File Past a Limit . . . 48
__util$rotatelogfile – Rotate the Log File Past a Limit . . . 49
__util$syslog – Send a Message to a Syslog Host . . . 50
__util$trace – Write a Trace Record . . . 51
$TRACE, $IFTRACE – Trace (Macros) . . . 52
__util$dumphex – Write a Hexadecimal Dump . . . 53
$DUMPHEX – Write a Hexadecimal Dump (Macro) . . . 54
2.3 Counted Strings and String Handling . . . 55
ASC, ASC$K_SZ – The Counted String Type . . . 55
$ASCDECL, $ASCINI, $ASCLEN, $ASCPTR, $ASC, $ASCNIL – Counted String Macros . . . 56
__util$str2asc – Copy a C String to a Counted String . . . 57
__util$asc2str – Copy a Counted String to a C String . . . 58
__util$cpyasc – Copy a Counted String . . . 59
__util$cmpasc – Compare Counted Strings . . . 60
__util$cmpasc_blind – Compare Counted Strings, Ignoring Case . . . 61
__util$strelem – Take an Element of a Delimited String . . . 62
__util$uint32_2_dec – Convert an Unsigned Integer to Decimal Text . . . 63
__util$trim – Remove Leading and Trailing White Space . . . 64
__util$collapse – Remove All White Space . . . 65
__util$uncomment – Cut a Comment Off a Line . . . 66
__util$strcat_rx – Concatenate a List of Strings . . . 67
__util$strstr – Find a Counted String in a Counted String . . . 68
CRLF, CRLFCRLF, CRLFCRLF_LW – Line Terminator Constants . . . 69
2.4 Memory . . . 70
__util$movc5 – Move Characters with Fill . . . 70
__util$fill – Fill a Buffer with a Pattern . . . 71
__util$bzero – Clear Memory . . . 72
__util$iszero – Test Memory for Zeros . . . 73
__util$strxor – Exclusive-OR Data with a Key . . . 74
$ARRSZ – Number of Elements of an Array . . . 75
2.5 Pattern Matching . . . 76
__util$pattern_match – Match a String Against a Wildcard Pattern . . . 76
2.6 Hexadecimal, UUID, Base64 and CRC . . . 77
__util$hex2bin – Convert Hexadecimal Text to Binary . . . 77
__util$bin2hex – Convert Binary to Hexadecimal Text . . . 78
$BIN2HEX – Convert Binary to Hexadecimal (Macro) . . . 79
__util$faohex – Format Octets as One Hexadecimal Dump Line . . . 80
__util$uuid2str – Convert a UUID to Text . . . 81
__util$base64 – Encode Octets as Base64 . . . 82
__util$unbase64 – Decode Base64 . . . 83
__util$crc32c – Compute a CRC-32 . . . 84
2.7 Keywords and Symbols . . . 85
KWDENT, UTIL$K_LOOKUP_* – Keyword Table Types . . . 85
__util$lookup_key – Look a Keyword Up in a Table . . . 86
SYM_REC, $SYM_REC_INI, $SYM_REC_EOL – Symbol Table Types . . . 88
__util$val2sym – Translate a Value to Its Name . . . 89
__util$mask2sym – Translate a Mask to Names . . . 90
2.8 Parameters and Configuration . . . 91
OPTS, OPTS$K_*, OPTS_NULL – Option Table Types . . . 91
__util$getparams – Read the Options from the Command Line . . . 93
__util$readconfig – Read the Options from a File . . . 95
__util$showparams – Log the Options . . . 96
2.9 Queues, Locks and Atomic Operations . . . 97
ENTRY, __QUEUE, QUEUE_INITIALIZER – Queue Types . . . 98
__util$lockspin – Take a Spin Lock . . . 99
__util$unlockspin – Release a Spin Lock . . . 100
$LOCK_LONG, $UNLOCK_LONG – Lock Macros . . . 101

__util$insqtail – Insert an Entry at the Tail . . . 102
__util$insqhead – Insert an Entry at the Head . . . 103
__util$remqhead – Remove the Entry at the Head . . . 104
__util$remqtail – Remove the Entry at the Tail . . . 105
__util$remqent – Remove a Given Entry . . . 106
__util$movqhead – Move an Entry to the Head . . . 107
__util$movqtail – Move an Entry to the Tail . . . 108
__util$q_unlink – Take an Entry Out of the Chain of a Queue . . . 109
__util$clrqueue – Empty a Queue . . . 110
$INSQHEAD, $INSQTAIL, $REMQHEAD, $REMQTAIL, $REMQENT, $MOVQHEAD, $MOVQTAIL, $CLRQUE – Queue Macros . . . 111
$CMP_STORE_LONG – Compare and Swap . . . 112
$ATOMIC_INC, $ATOMIC_DEC – Atomic Increment and Decrement . . . 113
2.10 Ranges, Minimum and Maximum . . . 114
__util$min, __util$max, $MIN, $MAX – Smaller and Larger of Two Integers . . . 114
__util$isinrange, $ISINRANGE – Test a Value Against a Closed Range . . . 115
__util$isinrange2, $ISINRANGE2 – Test a Value Against a Closed Range (Second Form) . . . 116
__util$range, $RANGE – Bound a Value to a Range . . . 117
2.11 Item Lists . . . 118
ILE2, ILE3 – Item List Element Types . . . 118
$ILE2INI, $ILE3INI, $ILENULL – Item List Macros . . . 119
2.12 Time Helpers . . . 120
__util$timbuf – Break Down the Local Time . . . 121
__util$add_time – Add Two Times . . . 122
__util$sub_time – Subtract Two Times . . . 123
__util$cmp_time – Compare Two Times . . . 124
2.13 The No-Operation Routine . . . 125
__util$nope – Do Nothing . . . 125
Miscellaneous Definitions . . . 126

Chapter 3. FAO and Time Services 127
3.1 Conventions of the FAO Routines . . . 128
3.2 The FAO Directives . . . 130
__util$fao – Formatted ASCII Output . . . 135
__util$faol – Formatted ASCII Output with List . . . 137
$FAO_DESC – Initialize an Output Descriptor . . . 139
3.3 The Binary Time Format . . . 140
__util$gettim – Get Binary Time . . . 141
__util$bintim – Convert ASCII Time to Binary Time . . . 142
__util$asctim – Convert Binary Time to ASCII . . . 144
__util$add_times – Add Two Binary Times . . . 146
__util$sub_times – Subtract Two Binary Times . . . 147
__util$add_time – Add Two Timespecs . . . 148
__util$sub_time – Subtract Two Timespecs . . . 149
__util$cmp_time – Compare Two Timespecs . . . 150
__util$timbuf – Break Time into a Calendar Structure . . . 151
3.4 FAO in Logging and Message Calls . . . 152
3.5 Differences from OpenVMS . . . 153
3.6 Changes in Version 1.7.0 . . . 154

Chapter 4. Command Language Routines 155
4.1 The Command Tables . . . 156
4.2 Value Types . . . 158
4.3 Flags and Options . . . 160
4.4 Parsing Rules . . . 161
4.5 Keywords . . . 162
4.6 Nested Verbs . . . 163
__cli$parse – Parse a Command Line . . . 164

__cli$dispatch – Call the Action Routine . . . 166
__cli$get_value – Get a Value . . . 167
__cli$get_keyword – Resolve a Keyword Value . . . 168
__cli$present – Test Whether an Item Is Present . . . 170
__cli$cleanup – Release a Context . . . 171
__cli$show_verbs – List a Table of Verbs . . . 172
__cli$help – Print a Help Screen . . . 173
__cli$show_ctx – List a Context . . . 174
4.7 Changes in Version 1.7.0 . . . 175

Chapter 5. TLV Protocol Routines 176
5.1 The PDU Layout . . . 177
__tlv$hset – Set the PDU Header . . . 179
__tlv$hget – Get the PDU Header . . . 180
__tlv$encode_tag – Pack a Tag . . . 182
__tlv$decode_tag – Unpack a Tag . . . 183
__tlv$put – Append a Record . . . 184
__tlv$get – Get a Value . . . 186
__tlv$lookup – Look Up a Record . . . 188
__tlv$dump – List a PDU . . . 190
5.2 Changes in Version 1.7.0 . . . 191

Appendix A. Condition Values and Severity 192
A.1 Layout of a Condition Value . . . 193
A.2 Severity Values . . . 194
A.3 Macros . . . 195
A.4 Values of the Library Modules . . . 196

Appendix B. Index of Routines and Macros 198

Preface

Intended Audience

This manual is for the programmer who writes C code with the StarLet library and needs to know, for one routine or one macro, exactly what it takes, what it does and what it returns. It assumes C11 and the use of a POSIX or Windows toolchain. It does not assume OpenVMS, although the reader who knows the OpenVMS Run-Time Library will find the layout, the names and the condition values familiar.

The manual is a reference. It is read by entry, not from cover to cover. For the reasons behind the design and for worked programs, see the *STARLET Programmer's Guide*.

Document Structure

Part	Contents
Chapter 1	Introduction: the condition-value convention, the argument types, the naming, thread safety, how an entry reads
Chapter 2	Utility Routines (`utility_routines`): messages, logging, strings, memory, pattern matching, encodings, keywords, parameters, queues, ranges, item lists
Chapter 3	FAO and Time Services (`starlet`)
Chapter 4	Command Language Routines (`cli_routines`)
Chapter 5	TLV Protocol Routines (`avproto`)
Appendix A	Condition Values and Severity
Appendix B	Index of Routines and Macros

Related Documents

- *STARLET Programmer's Guide*, order number STARLET-PG-0171: the design of the library, the coding rules it follows and worked examples.

The sources of the routines are `utility_routines.h` and `utility_routines.c`, `starlet.h` and `starlet.c`, `cli_routines.h` and `cli_routines.c`, `avproto.h` and `avproto.c`. Every routine of the `.c` files carries a `FUNCTIONAL DESCRIPTION` block at its head. The regression tests in the directory `tests/` show real calls. Where this manual and the behaviour of the code disagree, the code is right and this manual is to be corrected.

Changes in Version 1.7.1

Version 1.7.1 closes the eight items left from 1.7.0: `__tlv$lookup` checks the length of a fixed-size record; the log routines return `STS$K_ERROR` for an undefined severity; `$LOG_FAO` and `$PUTMSG_FAO` mark a bad FAO status in the record (`[FAO BADDIR]`); `__util$hex2bin` validates; `OPTS$K_INT` of 1, 2 and 8 octets is unsigned and of 4 octets and `int` signed; matching quotes are dropped from an option value; `__cli$get_keyword` returns `STS$K_WARN` for an empty value; the comment of `__util$deflog` is corrected. Every entry describes the 1.7.1 behaviour, and Appendix D of the *STARLET Programmer's Guide* (STARLET-PG-0171) lists what a consumer can notice.

Changes in Version 1.7.0

Version 1.7.0 corrects a good number of defects of 1.6.11 and changes a few values and prototypes with them. Every entry of this manual describes the 1.7.1 behaviour. The list of what a consumer of the library can notice, with the reason for each change, is Appendix D of the *STARLET Programmer's Guide* (STARLET-PG-0171).

Trademarks

OpenVMS, VMScluster, Alpha and Digital are trademarks of Hewlett Packard Enterprise Development LP or of VMS Software, Inc. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. Windows is a trademark of the Microsoft group of companies. Android is a trademark of Google LLC. Arm is a trademark of Arm Limited. Intel is a trademark of Intel Corporation. All other names of products and companies are trademarks of their respective owners.

StarLet is an independent implementation. It contains no code of OpenVMS and is not affiliated with, endorsed or supported by any of the owners named here; their names are used only to say what the library is modelled on or runs on.

Conventions

An entry of the manual describes one routine or one macro. It is laid out like an entry of a run-time library reference:

Heading	Meaning
title	The name of the routine and, after two dashes, what it does
Format	The C prototype, or the macro, exactly as the header gives it
Returns	The type of the value, and what the value is
Arguments	One item per argument: its type, its access, its mechanism, its description
Description	What the routine does, its edge cases and its limits, as the code behaves
Condition Values Returned	A table of the values the routine returns and when
Example	A short fragment of C

Access is one of: *read only*, the routine does not change what the argument designates; *write only*, the routine stores a value there and does not read the old one; *modify*, the routine reads the value and stores a new one. *Mechanism* is *by value* or *by reference*, that is, the address is passed.

Names of routines and macros are set in `monospace`. In running text a routine is written with empty parentheses, `__util$strstr()`. A dollar sign is part of a name, as in OpenVMS; it is not special in any way in C with GCC, Clang or MinGW-w64.

Dates have the form `31-OCT-2026`. A number in the form `0x1F` is hexadecimal.

Chapter 1. Introduction to the Routines

1.1 Why These Routines

The routines of `utility_routines` are the small, steady parts that every program of a project ends up writing for itself: a log, a message, a queue, a parameter table, a checksum. Here they are written once. What they get right, and the code shows it:

Every call tells how it ended, and in one way. A routine that can fail returns a condition value. The low bit of it is set for success and clear for failure, so the whole library is checked with one test, `1 & status`, the test of OpenVMS. Nothing is signalled by `errno`, by a null pointer in one place and by a negative number in the next. A status carries its severity, so a caller can tell a warning from an error and an error from a fatal one without a table.

A length is carried, not searched for. A string that may hold a zero octet is an `ASC`, a counted string; a buffer is an address and a length; the encoders and decoders, `__util$strstr()`, `__util$trim()` and the rest take the length and do not call `strlen()` again and again. A routine that writes into a buffer takes its size and does not write beyond it; the logging routines truncate a record rather than overrun the buffer, and tests (`tests/test_log_overflow.c`, `tests/test_strings.c`, `tests/test_queue.c`) hold them to it.

One message catalogue, one form of message. A program registers its messages once with `__util$inimsg()`, and `$PUTMSG_FAO()` finds a message by its condition value and prints it in the form of OpenVMS, `%FACILITY-S-IDENT, text`. The text of a message is written in one place, not at every place that signals it, and a status that crosses a module boundary is still the same message.

One formatter for logs, messages and output. The `_FAO` routines, `$LOG_FAO()` and `$PUTMSG_FAO()`, format with the FAO service of the `starlet` module. The same control strings serve a log record, a catalogue message and a string built into a buffer. The macros cast each parameter and count them, so a caller writes the values and nothing else.

A log record says where it came from. Each record carries the time, with the milliseconds, and the number of the thread. A record of a debug build carries the module, the routine and the line of the call, `[module\routine:line]`, taken from `__MODULE__`, `__FUNCTION__` and `__LINE__` by the macro. A log of several threads is read by thread; a record is found in the source by its line.

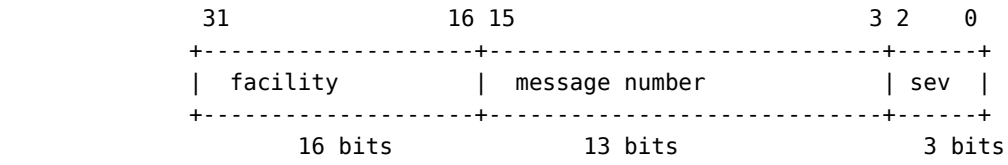
Small, static and free of dependencies. Most of the routines are `static inline` in one header and need no library at all. The rest are in one source file that needs the C library and, for `__util$fao()` and friends, the `starlet` module of the same package. There is no allocation in the common paths: the buffers are the caller's or the stack's. The code builds for Linux on x86-64 and ARMv8, for Windows with MinGW-w64 and for Android, and the checksum uses the instructions of the processor where it has them.

Tested. The directory `tests/` holds a regression program for the condition-value layout, the message catalogue, the log record and its overflow, rotation of the log file, the descriptor-redirected log, pattern matching, the keyword lookup, base64, the checksum, the queues,

the string and memory helpers, and the option routines. A defect that is found gets a test before it gets a fix.

1.2 The Condition-Value Convention

A **condition value** is an unsigned 32-bit integer. The routines of the library return it, and the messages of the catalogue are named by it. It is laid out as in OpenVMS:



The **severity** is the low three bits, and it is the only part the plain constants carry.

Constant	Value	Letter	Meaning
`STS$K_WARN`	0	W	Warning: done, but look at it
`STS$K_SUCCESS`	1	S	Success
`STS$K_ERROR`	2	E	Error: not done
`STS$K_INFO`	3	I	Information; counts as success
`STS$K_FATAL`	4	F	Fatal error
`STS$K_UNDEF`, `STS$K_UNDEF5`	5	?	Not a severity; a severity out of range is shown as `?`, and the log routines return `STS$K_ERROR` for it
`STS$K_UNDEF6`, `STS$K_UNDEF7`	6, 7	?	Not used

The low bit is the test. A condition value with the low bit set is a success; with the low bit clear, a failure. So `STS$K_SUCCESS` (1) and `STS$K_INFO` (3) pass the test, and `STS$K_WARN` (0), `STS$K_ERROR` (2) and `STS$K_FATAL` (4) do not. The test is written `1 & status`:

```
if ( !(1 & (status = __util$base64(src, srclen, dst, dstsz, &dstlen))) )
    return  $LOG(STS$K_ERROR, "base64 failed, status = %d", status);
```

Do not compare a status with `STS$K_SUCCESS` unless the routine's entry says that the value is exactly that: a routine may return `STS$K_INFO`, and a status built with `$DEFMSG()` has its message number in the upper bits.

`$DEFFAC()`, `$DEFMSG()` and `$DEFSEV()` build a condition value from its parts, and `$FAC()`, `$MSG()` and `$SEV()` take it apart. `$MSG()` is the inverse of `$DEFMSG()`; it takes the 13 bits of the message number and no more. `$STS_ERROR()` is true for an error or a fatal error. The macros are described in Section 2.1.

A few routines are not part of the convention, and their entries say so: `__util$pattern_match()` returns a truth value, `__util$crc32c()` returns a checksum, and a routine that converts or measures returns a length. A routine that returns a length returns it in an `int`, and a negative value or zero is not an error unless the entry says so.

1.3 Argument Types

Counted strings. The type `ASC` is the counted string of OpenVMS, ASCIC: one octet of length and up to 255 octets of text.

```
typedef struct _asc { unsigned char len; char sts[ASC$K_SZ]; } ASC;    /* ASC$K_SZ is 255
 ↪  */
```

The text is not required to end with a zero octet, and may contain one. Many routines of the library keep an extra zero after the text for the convenience of C, and where they do, the entry says it. The macros `$ASCDECL()`, `$ASCINI()`, `$ASCLEN()`, `$ASCPTR()`, `$ASC()` and `$ASCNIL` declare, initialize and take apart an `ASC`. The macro `$ASC()` expands to a length and an address, so that an `ASC` is printed with `"%.*s"`.

Buffers with lengths. A buffer is passed as an address and a length, in this order, or, when the routine writes into it, as an address, a size and the address of a variable that receives the length of the result. An output length is set even when the buffer was too small, so that the caller learns what the buffer should have been (`__util$base64()`, `__util$unbase64()`).

Item lists. The types `ILE2` and `ILE3` are the item-list elements of OpenVMS: a code, a length and an address, and for `ILE3` the address of a word that receives the length returned. A list ends with an element of zeros (`$ILENULL`). The types are defined here, so that a program has one definition of them; the routines that take a list are in the other chapters. Mind the layout of `ILE3`: the code comes first and the length second, the reverse of the OpenVMS `ILE3`; see Section 2.11.

Tables. A table is an array that ends with a terminator element or whose size is passed. The tables of the library are `OPTS` (options, ends with `OPTS_NULL`), `KWDENT` (keywords, size passed), `SYM_REC` (symbols, ends with `$SYM_REC_EOL`) and `EMSG_RECORD` (messages, size in the descriptor).

Pointers. A pointer argument that is an *output* is by reference and is not checked for null unless the entry says so. The macros that wrap queue routines cast their arguments, so that any pointer to a structure that begins with an `ENTRY` is accepted.

1.4 Naming

__util$name. A routine is named __util$ and a lower-case word, the facility prefix of the library as in LIB$ of OpenVMS. The name is a legal C identifier with the GCC, Clang and MinGW-w64 compilers; the dollar sign is not special.

$NAME. A macro that wraps a routine, or a constant of the facility, begins with a dollar sign and is in upper case: $LOG(), $INSQTAIL(), $ASCDECL(). A constant of a group is named GROUP$K_NAME, a mask GROUP$M_NAME: STS$K_ERROR, OPTS$K_STR, UTILK_LOOKUP_NCASE, STSM_SYSLOG.

__MODULE__ and __FAC__. A program names itself to the logging routines with two macros, defined *before* utility_routines.h is included:

```
#define __MODULE__  "CONF$"     /* the module: shows in a debug record, default NULL    */
#define __FAC__     "MYAPP"     /* the facility: shows in every record, default "UTILS" */
#include "utility_routines.h"
```

__MODULE__ is a string, normally the name of the source file or of the module; if it is not defined it is NULL and the debug record shows the routine and the line only. __FAC__ is the facility name that stands between % and the severity letter in a record, %MYAPP-E:. If it is not defined it is "UTILS".

Debug and release. The macros $LOG(), $LOG_FAO(), $PUTMSG(), $PUTMSG_FAO(), $TRACE() and $IFTRACE() differ with NDEBUG. Without NDEBUG (a debug build) they call the d variants, __util$logd() and the like, with __MODULE__, __FUNCTION__ and __LINE__, and the record shows where the call was. With NDEBUG they call the plain routines, and $TRACE() and $IFTRACE() compile to nothing.

Compilers. The header uses the GNU extension , ## __VA_ARGS__ and __sync_* and __atomic_* built-ins. It is written for GCC, Clang and MinGW-w64. -std=gnu11 is recommended.

Linking. The routines that are not static inline are in utility_routines.c; the program is linked with the library, libstarlet. The FAO routines are in the module starlet, and the _FAO routines of this chapter call them: include starlet.h where $LOG_FAO() or $PUTMSG_FAO() is used, because the macros cast their parameters to fao_prm_t, defined there. starlet.h includes utility_routines.h itself from 1.7.0, so the two headers may be included in either order.

1.5 Thread Safety

The library is meant for programs of several threads. What holds:

- **Logging is safe.** A record is built on the stack of the caller and is written by one `write()` call, so the records of different threads are not mixed within a record. The date and the thread number of the prefix are cached in thread-local storage, per thread, with no lock.
- **The log descriptor is not protected.** `__util$deflog()`, `__util$setlogfd()` and `__util$rotatelogfile()` change the descriptor that every thread writes to. Call `__util$deflog()` or `__util$setlogfd()` at start-up, before the threads are started. `__util$rotatelogfile()` is safe to call from a thread while others log; only one thread rotates at a time and the others return at once.
- **The message catalogue.** `__util$inimsg()` is thread-safe from 1.7.0: registrations are serialised, and the list is published only when a descriptor is complete, so a concurrent `__util$getmsg()` sees the old list or the new one. Registering at start-up is still the tidy way. `__util$getmsg()` and the routines that signal messages only read the list.
- **Queues are interlocked.** The queue routines take a spin lock in the queue header. An entry is inserted, removed or counted atomically with respect to the other queue calls on the same queue. The restrictions of individual routines are in their entries (Section 2.9).
- **The checksum** selects its engine on the first call. Two threads that call it for the first time decide alike, so the race is harmless.
- **`__util$syslog()`** keeps a socket in a `static` variable. The socket is created once, under a guard, so two threads that make the first call at once are safe (1.7.0).
- **The option routines** (`__util$getparams()`, `__util$readconfig()`) write the variables of the table and are not meant to be called from several threads; they are used once, in `main()`.

Routines that are `static inline` and work only on their arguments are as safe as their arguments are: two threads may call `__util$trim()` on two buffers, not on one.

1.6 How an Entry Reads

An entry starts with the name of the routine and a short title. **Format** is the prototype as it stands in the header. **Returns** gives the type and the meaning of the value. Under **Arguments**, an argument is described as

> *name* – `type`; access; mechanism. What it is.

so that `src -- void *; read only; by reference` is an address of data the routine reads. **Description** states what the routine does, what it does not do, and any limit: a size, a range, a condition under which it does nothing. **Condition Values Returned** lists the values and the cases. An entry of a routine that does not return a condition value says so there, and lists what it does return. **Example** is a fragment that compiles when it is put into a function; it assumes the includes and the macros of Section 1.4 and is short on purpose.

A macro is documented the same way. Its **Format** shows the macro as it is defined; where the macro expands to a call of a routine, the entry says which, and the arguments of the macro are described as the arguments of the macro, not of the routine.

Chapter 2. Utility Routines

This chapter describes every routine and macro declared in `utility_routines.h` and defined there (`static inline`) or in `utility_routines.c`. They are grouped by purpose, in this order:

Section	Group
2.1	Condition values and messages
2.2	Logging, tracing and output
2.3	Counted strings and string handling
2.4	Memory
2.5	Pattern matching
2.6	Hexadecimal, UUID, base64 and CRC
2.7	Keywords and symbols
2.8	Parameters and configuration
2.9	Queues, locks and atomic operations
2.10	Ranges, minimum and maximum
2.11	Item lists
2.12	Time helpers
2.13	The no-operation routine

The split with the `starlet` module. The routines `__util$fao()`, `__util$faol()`, `__util$bintim()`, `__util$asctim()`, `__util$gettim()`, `__util$add_times()` and `__util$sub_times()` have the prefix `__util$` but are *defined in `starlet.c`* and declared in `starlet.h`. They are described in Chapter 3. `__util$faohex()`, `__util$timbuf()`, `__util$add_time()`, `__util$sub_time()` and `__util$cmp_time()` are defined in `utility_routines.*` and are described here (Sections 2.6 and 2.12). Note the difference between `__util$add_time()` (this chapter: two `struct timespec`) and `__util$add_times()` (Chapter 3: `fao_time_t`).

The routines `__util$trace()`, `__util$dumphex()`, the logging routines and the message routines write to the **log descriptor**: the standard output until `__util$deflog()` or `__util$setlogfd()` changes it.

2.1 Condition Values and Messages

The constants and macros that build and take apart a condition value, and the message catalogue. The layout of a condition value is in Section 1.2.

STS$K_* – Severity Constants

Format

```
enum {
    STS$K_WARN  = 0,
    STS$K_SUCCESS   = 1,
    STS$K_ERROR = 2,
    STS$K_INFO  = 3,
    STS$K_FATAL = 4,
    STS$K_UNDEF = 5,
    STS$K_UNDEF5    = 5,
    STS$K_UNDEF6    = 6,
    STS$K_UNDEF7    = 7,
    STS$K_MAX   = 8
};

#define STS$M_SYSLOG    16
#define STS$C__WARN "W"
#define STS$C__SUCCESS  "S"
#define STS$C__ERROR    "E"
#define STS$C__INFO "I"
#define STS$C__FATAL    "F"
#define STS$C__UNDEF    "?"
```

Description

The severities of a condition value. `STS$K_MAX` is the end of the list, the number of severities. `STS$M_SYSLOG` is a flag, not a severity: it is ORed into the severity argument of `__util$log()`, `__util$log_fao()` and `$LOG()` to have the record also sent to the system log (see `__util$log()`). `STS$C__*` are the letters shown in a record.

$DEFFAC, $DEFMSG, $DEFSEV – Build a Condition Value

Format

```
#define $DEFFAC(code)   ((code) << 16)
#define $DEFMSG(code)   ( ((code) << 3) & 0x0000FFF8UL )
#define $DEFSEV(code)   ((code) & 0x7)
```

Returns

An integer expression: the field, shifted to its place in a condition value.

Arguments

- *code* – integer; read only; by value. For `$DEFFAC()` a facility number, up to 16 bits; for `$DEFMSG()` a message number, 13 bits; for `$DEFSEV()` a severity, `STS$K_*`.

Description

The three macros build the three fields of a condition value; the fields are ORed together. `$DEFMSG()` drops the bits of the message number above the 13th. `$DEFFAC()` does not mask: a facility number must fit 16 bits.

Example

```
#define MYAPP$K_FACNO   0x0123
#define MYAPP$_OPENFAIL ($DEFFAC(MYAPP$K_FACNO) | $DEFMSG(1) | $DEFSEV(STS$K_ERROR))
#define MYAPP$_STARTED  ($DEFFAC(MYAPP$K_FACNO) | $DEFMSG(2) | $DEFSEV(STS$K_INFO))
```

$FAC, $MSG, $SEV – Take a Condition Value Apart

Format

```
#define $FAC(code)  ((code) >> 16)
#define $MSG(code)  ( ((code) >> 3) & 0x00001FFFUL )
#define $SEV(code)  ((code) & 0x7)
```

Returns

An integer expression: the facility number, the message number, the severity.

Arguments

- *code* – unsigned integer; read only; by value. A condition value.

Description

`$FAC()`, `$MSG()` and `$SEV()` are the inverses of `$DEFFAC()`, `$DEFMSG()` and `$DEFSEV()`. Give them an `unsigned` value: `$FAC()` shifts without masking, and a negative `int` would extend its sign.

Example

```
unsigned l_sts = MYAPP$_OPENFAIL;

if ( $SEV(l_sts) == STS$K_ERROR && $FAC(l_sts) == MYAPP$K_FACNO )
    $LOG(STS$K_INFO, "message number %u", $MSG(l_sts));
```

$STS_ERROR – Test for an Error

Format

```
#define $STS_ERROR(code)    ( ($SEV(code) == STS$K_ERROR) || ($SEV(code) == STS$K_FATAL) )
```

Returns

Non-zero if the severity of *code* is `STS$K_ERROR` or `STS$K_FATAL`; zero otherwise.

Arguments

- *code* – unsigned integer; read only; by value. A condition value. The macro evaluates it twice.

Description

True for the two severities that mean the operation failed. A warning is not an error here, though its low bit is clear. For the success test use `1 & code`.

UTIL$S_INVARG, UTIL$S_INQUE, UTIL$S_NOLOCK, UTIL$S_DUPFAC – Condition Values of the Library

Format

```
#define UTIL$S_INVARG   0x102
#define UTIL$S_INQUE    0x202
#define UTIL$S_NOLOCK   0x302
#define UTIL$S_DUPFAC   0x400
```

Description

Condition values of the queue insertion routines and of `__util$inimsg()`. They follow the layout of Section 1.2: the message number is above the low three bits, which are the severity.

Value	Severity	Meaning
`UTIL$S_INVARG` (0x102)	`STS$K_ERROR`	A null or otherwise unusable argument
`UTIL$S_INQUE` (0x202)	`STS$K_ERROR`	The entry is already in another queue
`UTIL$S_NOLOCK` (0x302)	`STS$K_ERROR`	The spin lock of the queue was not acquired
`UTIL$S_DUPFAC` (0x400)	`STS$K_WARN`	A catalogue of this facility is registered already

The queue values are returned by `__util$insqtail()` and, since 1.7.0, by `__util$insqhead()` as well. In 1.6.11 `UTIL$S_NOLOCK` was 0x202 and could not be told from `UTIL$S_INQUE`; `UTIL$S_DUPFAC` is new. All four have the low bit clear. `UTIL$S_DUPFAC` is a warning, so `$STS_ERROR()` is false for it, but `1 & status` is false as well.

EMSG_RECORD, EMSG_RECORD_DESC – Message Catalogue Types

Format

```
typedef struct __emsg_record__  {
        int     sts;
        struct {
        unsigned char   textl;
            char    text[254];
        };
        const char  *facname;
        const char  *mnemonic;
    } EMSG_RECORD;

typedef struct __emsg_record_desc__ {
        struct __emsg_record_desc__ *link;
        unsigned        msgnr;
        unsigned        facno;
        struct __emsg_record__  *msgrec;
        const char      *facname;
    } EMSG_RECORD_DESC;
```

Description

An `EMSG_RECORD` is one message: its condition value (*sts*), its text as an ASCIC (*textl* and *text*; the text is also used as a C string, so a zero octet must follow it), the name of the facility (*facname*) and the mnemonic (*mnemonic*). The last two are filled by `__util$inimsg()`; a table need not set them, except that *mnemonic* may be set to the stringised name of the condition, `"MYAPP$_OPENFAIL"`, from which `__util$inimsg()` keeps what follows `$_`, `"OPENFAIL"`.

An `EMSG_RECORD_DESC` describes a catalogue of one facility: *msgrec* is the array, *msgnr* the count of its elements, *facno* the facility number (the number given to `$DEFFAC()`), *facname* the name shown in `%FACNAME-S-`. *link* is for the library. The descriptor and the array must stay valid for the life of the program.

The text of a message is a printf format for `__util$putmsg()` and an FAO control string for `__util$putmsg_fao()`. A catalogue is therefore written for one of the two.

Example

```
#define MYAPP$_MSGENT(a_sts, a_txt) { (int) (a_sts), { sizeof(a_txt) - 1, a_txt }, NULL,
 ↪  #a_sts }

static EMSG_RECORD s_msgs [] = {
    MYAPP$_MSGENT(MYAPP$_OPENFAIL,  "Cannot open !AZ, errno = !SL"),
    MYAPP$_MSGENT(MYAPP$_STARTED,   "Started, version !AZ"),
};

static EMSG_RECORD_DESC s_msgdsc = { NULL, $ARRSZ(s_msgs), MYAPP$K_FACNO, s_msgs, "MYAPP"
 ↪  };
```

__util$inimsg – Register a Message Catalogue

Format

```
unsigned    __util$inimsg   (EMSG_RECORD_DESC *msgdsc);
```

Returns

`unsigned`: a condition value. In 1.6.11 both failures were a plain `STS$K_WARN`.

Arguments

- *msgdsc* – `EMSG_RECORD_DESC *`; modify; by reference. The descriptor of the catalogue.

Description

Registers a catalogue so that the message routines find its records. The routine sorts the array *msgrec* in place by facility and message number, stamps *facname* into every record, trims every *mnemonic* to the part after `$_`, and links the descriptor at the head of the list of catalogues. The array must therefore be writable and must not be a `const` object.

A facility is registered once. A second descriptor with the same *facno* is refused and nothing is changed. The routine is thread-safe (1.7.0): registrations are serialised, and the list is published only when the descriptor is complete (Section 1.5).

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The catalogue is registered
`UTIL$S_INVARG` (0x102)	*msgdsc* is null
`UTIL$S_DUPFAC` (0x400)	A catalogue of the same facility number is registered already; nothing is changed

Example

```
unsigned l_sts;

if ( !(1 & (l_sts = __util$inimsg(&s_msgdsc))) && (UTIL$S_DUPFAC != l_sts) )
    $LOG(STS$K_ERROR, "catalogue is not registered, status = %#x", l_sts);
```

__util$getmsg – Find a Message Record

Format

```
unsigned    __util$getmsg   (unsigned sts, EMSG_RECORD **outmsg);
```

Returns

`unsigned`: a condition value.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value of the message wanted.
- *outmsg* – `EMSG_RECORD **`; write only; by reference. Receives the address of the record.

Description

Finds the catalogue of the facility of *sts*, then the record by a binary search. The facility and the message number are compared; the severity of *sts* is ignored, so one record serves a message signalled as information in one place and as a warning in another. *outmsg* is not changed if the record is not found.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Found; *outmsg* holds the record
`STS$K_ERROR`	No catalogue of the facility is registered, or it holds no such message number

Example

```
EMSG_RECORD *l_rec;

if ( 1 & __util$getmsg(MYAPP$_STARTED, &l_rec) )
    $LOG(STS$K_INFO, "the format is '%.*s'", l_rec->textl, l_rec->text);
```

__util$putmsg – Signal a Message (printf Format)

Format

```
unsigned    __util$putmsg   (unsigned sts, ...);
```

Returns

`unsigned`: the argument *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value of the message.
- *...* – the arguments of the format of the message text, by value.

Description

Writes one record to the log descriptor: the prefix (date, time, thread number), then the message. The record of the catalogue is found with `__util$getmsg()`; its text is used as a **printf format** with the arguments that follow. No `%FACILITY-S-` envelope is added.

If the record is not found, the default text is written instead: `%NONAME-<S>-NOMSG, Message number: <hex> [fac: ..., sev: ..., msgno: ...]`, with the severity letter of *sts*. The line begins with the percent sign, like every other line of the form.

A record of more than 2047 octets is cut. The routine returns *sts*, so a function may end with `return $PUTMSG(...)`. It is the routine that `$PUTMSG()` calls in a release build.

Condition Values Returned

This routine returns its argument *sts* and no condition value of its own.

__util$putmsgd – Signal a Message with the Place of the Call

Format

```
unsigned    __util$putmsgd  (unsigned sts, const char *__mod, const char *__fi, unsigned
↪  __li, ...);
```

Returns

`unsigned`: the argument *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value.
- *__mod* – `const char *`; read only; by reference. The module name; may be null.
- *__fi* – `const char *`; read only; by reference. The routine name.
- *__li* – `unsigned`; read only; by value. The line number.
- *...* – the arguments of the format of the message text.

Description

As `__util$putmsg()`, with the place of the call inserted after the prefix: `[module\routine:line]` if *__mod* is not null, `[routine:line]` if it is. `$PUTMSG()` calls this routine in a debug build.

Condition Values Returned

This routine returns its argument *sts*.

__util$putmsg_fao – Signal a Message (FAO Control String)

Format

```
unsigned    __util$putmsg_fao   (unsigned sts, int prmcnt, ...);
```

Returns

`unsigned`: the argument *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value.
- *prmcnt* – `int`; read only; by value. The number of FAO parameters that follow, 0 to 16. A larger number is taken as 16; a negative one as 0.
- *...* – `fao_prm_t`, by value. The FAO parameters. Each must be passed as `fao_prm_t`.

Description

Writes one record: the prefix, the envelope, the text. The record is found with `__util$getmsg()`; its text is an **FAO control string** formatted by `__util$faol()` (Chapter 3). The envelope is the form of OpenVMS:

```
%FACILITY-S-MNEMONIC, text        if the record has a mnemonic
%FACILITY-S-text                  if it has not
```

`S` is the letter of the severity of *sts* (not of the severity stored in the record). The facility name is *facname* of the catalogue, or `NONAME`. If the record is not found, the default text of `__util$putmsg()` is written. The text is cut to the buffer (2047 octets). Fewer parameters than directives in the control string is safe: formatting stops where the parameters stop.

If the formatting of the text fails (a bad directive, too few parameters, a cut field or output), the record is still written, with the marker `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]`, `[FAO BUFOVF]` or `[FAO 0x...]` after the text, as `__util$log_fao()` does; the return value is not changed. The same holds for `__util$putmsgd_fao()`.

Call it through `$PUTMSG_FAO()`, which casts the parameters and counts them.

Condition Values Returned

This routine returns its argument *sts*.

__util$putmsgd_fao – Signal a Message (FAO) with the Place of the Call

Format

```
unsigned    __util$putmsgd_fao  (unsigned sts, const char *__mod, const char *__fi,
↪  unsigned __li, int prmcnt, ...);
```

Returns

`unsigned`: the argument *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value.
- *__mod* – `const char *`; read only; by reference. The module name; may be null.
- *__fi* – `const char *`; read only; by reference. The routine name.
- *__li* – `unsigned`; read only; by value. The line number.
- *prmcnt* – `int`; read only; by value. The number of FAO parameters, 0 to 16.
- *...* – `fao_prm_t`, by value. The FAO parameters.

Description

As `__util$putmsg_fao()`, with `[module\routine:line]` (or `[routine:line]`) between the prefix and the envelope. `$PUTMSG_FAO()` calls this routine in a debug build.

Condition Values Returned

This routine returns its argument *sts*.

$PUTMSG – Signal a Message (Macro)

Format

```
#define $PUTMSG(sts, ...)   __util$putmsgd(sts, __MODULE__, __FUNCTION__ , __LINE__ , ##
↪  __VA_ARGS__)   /* debug   */
#define $PUTMSG(sts, ...)   __util$putmsg(sts, ## __VA_ARGS__)                  /* NDEBUG
↪  */
```

Returns

`unsigned`: *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value.
- *...* – the arguments of the printf format of the message text.

Description

Signals a catalogue message with a printf format text. Without `NDEBUG` the record shows the place of the call.

Example

```
return  $PUTMSG(MYAPP$_OPENFAIL, l_path, errno);    /* for a catalogue whose texts are
↪  printf formats */
```

$PUTMSG_FAO – Signal a Message, FAO (Macro)

Format

```
#define $PUTMSG_FAO(sts, ...)   __util$putmsgd_fao(sts, __MODULE__, __FUNCTION__ ,
 ↪  __LINE__ ,   /* debug  */
                FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))
#define $PUTMSG_FAO(sts, ...)   __util$putmsg_fao(sts, FAO$_NARG(__VA_ARGS__)
 ↪  FAO$_EACH(__VA_ARGS__))   /* NDEBUG */
```

Returns

`unsigned`: *sts*.

Arguments

- *sts* – `unsigned`; read only; by value. The condition value.
- *...* – 0 to 16 values for the FAO directives. Written plain.

Description

The macro counts the values with `FAO$_NARG()` and casts each to `fao_prm_t` with `FAO$_EACH()`, so a caller writes the values and neither the count nor the casts. The helper macros `FAO$_C`, `FAO$_E0` to `FAO$_E16`, `FAO$_GET`, `FAO$_NARG` and `FAO$_EACH` are part of this mechanism and are not called directly. More than 16 values do not compile. A pointer given for `!AZ` is converted to `fao_prm_t` by the cast. Include `starlet.h`.

Example

```
#include "starlet.h"

$PUTMSG_FAO(MYAPP$_OPENFAIL, "/etc/myapp.conf", (long) errno);
```

2.2 Logging, Tracing and Output

All the routines of this section write to the **log descriptor**. It is the standard output when a program starts. `__util$deflog()` points it at a file, `__util$setlogfd()` at a descriptor that is already open.

The record. Every record but that of `__util$out()` begins with the same prefix:

```
DD-MM-YYYY HH:MM:SS.fff TTTTTT
```

The date and time are local. *fff* is the fraction of the second in true milliseconds, 000 to 999, on every platform (in 1.6.11 it was the nanoseconds divided by 2^20 and ran from 000 to 953). The time is taken from the coarse clock of the system (`CLOCK_REALTIME_COARSE` on Linux), whose resolution is a few milliseconds. *TTTTTT* is the number of the thread (`gettid()` on Linux), right-aligned to six columns; it is fetched once for each thread. After a `fork()` the child still shows the number of the parent.

The routines then add, in this order, the place of the call (debug forms only), the envelope `%FACILITY-S:` and the text:

Routine	Record after the prefix
`__util$log()`, `__util$log_fao()`	`%FAC-S: text`
`__util$logd()`, `__util$logd_fao()`	`[module\routine:line] %FAC-S:  text` (two spaces)
`__util$trace()`	`[module\routine:line]` padded with spaces to column 72, then text
`__util$dumphex()`	`[routine:line] Dump of N octets follows:` and the lines of the dump

A record ends with a line feed and is written by one `write()`. The record is limited to 2047 octets (1023 for `__util$trace()`); a longer one is cut. An error of the `write()` is ignored.

__util$log – Write a Log Record (printf Format)

Format

```
unsigned    __util$log  (const char *fac, unsigned severity, const char *fmt, ...);
```

Returns

`unsigned`: the severity.

Arguments

- *fac* – `const char *`; read only; by reference. The facility name for the envelope.
- *severity* – `unsigned`; read only; by value. `STS$K_*`, optionally ORed with `STS$M_SYSLOG`.
- *fmt* – `const char *`; read only; by reference. A printf format.
- *...* – the arguments of *fmt*.

Description

Writes `prefix %fac-S: text`. The letter *S* is the one of `$SEV(severity)`.

`STS$M_SYSLOG` is taken off first. Then a severity above `STS$K_FATAL` (4) is not valid: the letter shown is `?` and the value returned is `STS$K_ERROR` (2, the low bit clear), because an undefined severity is a caller's error and must not read as success. Up to 1.7.0 the value returned was `STS$K_UNDEF` (5), which has the low bit set. Up to 1.6.11 the flag itself made the severity "out of range", so a record with `STS$M_SYSLOG` returned 5 whatever its severity was.

If `STS$M_SYSLOG` is ORed into *severity* the text is also sent with `syslog()` of the C library. The priority is `LOG_INFO` for a severity with the low bit set (success, information) and `LOG_ERR` for the others. Not available on Windows.

`$LOG()` calls this routine in a release build.

Condition Values Returned

The routine returns the severity logged, with `STS$M_SYSLOG` taken off, so that `return $LOG(STS$K_ERROR, ...)` returns an error and `return $LOG(STS$K_ERROR | STS$M_SYSLOG, ...)` does too.

Value	Meaning
`STS$K_WARN`, `STS$K_SUCCESS`, `STS$K_ERROR`, `STS$K_INFO`, `STS$K_FATAL`	The severity given
`STS$K_ERROR` (2)	The severity was undefined (5, 6 or 7 in the low bits); the record shows `?`

Example

```
__util$log("MYAPP", STS$K_INFO, "started, pid = %d", (int) getpid());
```

__util$logd – Write a Log Record with the Place of the Call

Format

```
unsigned    __util$logd (const char *fac, unsigned severity, const char *fmt, const char
 ↪  *mod, const char *__func, unsigned __line, ...);
```

Returns

`unsigned`: the severity logged (`STS$M_SYSLOG` cleared; `STS$K_ERROR` for an undefined one, 5 to 7).

Arguments

- *fac* – `const char *`; read only; by reference. The facility name.
- *severity* – `unsigned`; read only; by value. `STS$K_*`.
- *fmt* – `const char *`; read only; by reference. A printf format.
- *mod* – `const char *`; read only; by reference. The module name; may be null, and then the record shows `[routine:line]`.
- *__func* – `const char *`; read only; by reference. The routine name.
- *__line* – `unsigned`; read only; by value. The line number.
- *...* – the arguments of *fmt*.

Description

Writes `prefix [mod\func:line] %fac-S:  text` (`prefix [func:line] ...` for a null *mod*). `$LOG()` calls this routine in a debug build. The severity is treated as `__util$log()` treats it, including `STS$M_SYSLOG`: the record also goes to the system log, in the debug build as in the release one (1.6.11 did not do this).

Condition Values Returned

The severity logged, as `__util$log()` returns it: `STS$M_SYSLOG` is not part of it, and a value beyond `STS$K_FATAL` is `STS$K_ERROR` (up to 1.7.0, `STS$K_UNDEF`).

Example

```
#define __MODULE__ "CONF$"
...
$LOG(STS$K_ERROR, "no file '%s'", l_path);  /* 31-10-2026 12:00:00.123 4711
 ↪  [CONF$\main:42] %UTILS-E:  no file 'x' */
```

__util$log_fao – Write a Log Record (FAO Control String)

Format

```
unsigned    __util$log_fao  (const char *a_fac, unsigned a_sev, const char *a_ctl, int
↪  a_prmcnt, ...);
```

Returns

`unsigned`: the severity.

Arguments

- *a_fac* – `const char *`; read only; by reference. The facility name.
- *a_sev* – `unsigned`; read only; by value. `STS$K_*`, optionally ORed with `STS$M_SYSLOG`.
- *a_ctl* – `const char *`; read only; by reference. An FAO control string.
- *a_prmcnt* – `int`; read only; by value. The number of FAO parameters, 0 to 16 (more are taken as 16, fewer than 0 as 0).
- *...* – `fao_prm_t`, by value. The parameters.

Description

The FAO twin of `__util$log()`: writes `prefix %fac-S: text`, with the text formatted by `__util$faol()`. The range check of the severity and the `STS$M_SYSLOG` rule are those of `__util$log()`, including the returned value; the body already formatted is what `syslog()` receives. Call it through `$LOG_FAO()`. The expansion of the directives is described in Chapter 3.

If `__util$faol()` returns a status with the low bit clear (a bad directive, too few parameters, a field or the output cut), the record is still written, with a marker after the text: `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` or `[FAO BUFOVF]`, or `[FAO 0x...]` with the status in hexadecimal for another one. A text that fills the record is cut to leave room for the marker. The return value is not changed, so `return $LOG_FAO(STS$K_ERROR, ...)` still returns the severity. (Up to 1.7.0 the status was dropped.)

Condition Values Returned

As `__util$log()`.

__util$logd_fao – Write a Log Record (FAO) with the Place of the Call

Format

```
unsigned    __util$logd_fao (const char *a_fac, unsigned a_sev, const char *a_ctl, const
↪  char *a_mod, const char *a_func, unsigned a_line, int a_prmcnt, ...);
```

Returns

`unsigned`: the severity logged (`STS$M_SYSLOG` cleared; `STS$K_ERROR` for an undefined one, 5 to 7).

Arguments

- *a_fac* – `const char *`; read only; by reference. The facility name.
- *a_sev* – `unsigned`; read only; by value. `STS$K_*`.
- *a_ctl* – `const char *`; read only; by reference. An FAO control string.
- *a_mod* – `const char *`; read only; by reference. The module name; may be null (see `__util$logd()`).
- *a_func* – `const char *`; read only; by reference. The routine name.
- *a_line* – `unsigned`; read only; by value. The line number.
- *a_prmcnt* – `int`; read only; by value. The number of FAO parameters, 0 to 16.
- *...* – `fao_prm_t`, by value. The parameters.

Description

The FAO twin of `__util$logd(): prefix [mod\func:line] %fac-S:  text. $LOG_FAO()` calls it in a debug build. `STS$M_SYSLOG` is honoured, as in `__util$log_fao()`.

Condition Values Returned

As `__util$logd()`.

$LOG – Write a Log Record (Macro)

Format

```
#define $LOG(severity, fmt, ...)    __util$logd(__FAC__, severity, fmt, __MODULE__,
 ↪  __FUNCTION__ , __LINE__ , ## __VA_ARGS__)   /* debug  */
#define $LOG(severity, fmt, ...)    __util$log(__FAC__, severity, fmt, ## __VA_ARGS__)
 ↪  /* NDEBUG */
```

Returns

`unsigned`: the severity (see the routine called).

Arguments

- *severity* – `STS$K_*`; read only; by value.
- *fmt* – string literal or `const char *`; read only; by reference. A printf format.
- *...* – the arguments of *fmt*.

Description

The usual way to log. The facility is `__FAC__`; the module `__MODULE__`. Define `__MODULE__` as a string before including the header: a debug build with the default `NULL` shows the routine and the line only, `[routine:line]`.

Example

```
if ( 0 > (l_fd = open(l_path, O_RDONLY)) )
    return  $LOG(STS$K_ERROR, "open('%s') -> errno %d", l_path, errno);
```

$LOG_FAO – Write a Log Record, FAO (Macro)

Format

```
#define $LOG_FAO(severity, ctl, ...)    __util$logd_fao(__FAC__, severity, ctl,
 ↪  __MODULE__, __FUNCTION__ , __LINE__ ,   /* debug  */
                    FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))
#define $LOG_FAO(severity, ctl, ...)    __util$log_fao(__FAC__, severity, ctl,
 ↪  FAO$_NARG(__VA_ARGS__) FAO$_EACH(__VA_ARGS__))   /* NDEBUG */
```

Returns

`unsigned`: the severity.

Arguments

- *severity* – `STS$K_*`; read only; by value.
- *ctl* – string; read only; by reference. An FAO control string.
- *...* – 0 to 16 values for the directives, written plain.

Description

As `$PUTMSG_FAO()`, for the log: the macro counts the values and casts each to `fao_prm_t`. Include `starlet.h`.

Example

```
#include "starlet.h"

$LOG_FAO(STS$K_INFO, "!AZ: !UL octets in !UL blocks", l_name, (unsigned long) l_size,
 ↪  (unsigned long) l_blocks);
```

$IFLOG – Write a Log Record if a Flag is Set (Macro)

Format

```
#define $IFLOG(f, severity, fmt, ...)   f ? __util$logd(__FAC__, severity, fmt,
↪  __MODULE__, __FUNCTION__ , __LINE__ , ## __VA_ARGS__) : severity
```

Returns

The value of the expression: the result of `__util$logd()` if *f* is true, *severity* if not.

Arguments

- *f* – integer expression; read only; by value. The flag.
- *severity*, *fmt*, *...* – as `$LOG()`.

Description

Logs only when *f* is true. The macro is not parenthesised: put the whole use in parentheses when it is a part of a larger expression (a cast such as `(void) $IFLOG(...)` applies to *f* only), and put *f* in parentheses when it is not a simple name. It always calls `__util$logd()`, in debug and release builds alike, so ***MODULE*** is used in both.

Example

```
$IFLOG(g_verbose, STS$K_INFO, "queue length %u", l_count);
```

__util$log2buf – Format a Log Record into a Buffer

Format

```
unsigned    __util$log2buf  (void *out, int outsz, int * outlen, const char *fac,
↪  unsigned severity, const char *fmt, ...);
```

Returns

`unsigned`: the severity logged, as `__util$log()` returns it.

Arguments

- *out* – `void *`; write only; by reference. The buffer.
- *outsz* – `int`; read only; by value. Its size in octets.
- *outlen* – `int *`; write only; by reference. Receives the length of the record, without the terminating zero.
- *fac* – `const char *`; read only; by reference. The facility name.
- *severity* – `unsigned`; read only; by value. `STS$K_*`.
- *fmt* – `const char *`; read only; by reference. A printf format.
- *...* – the arguments of *fmt*.

Description

Builds `prefix %fac-S:text` in the caller's buffer, as `__util$log()` would write it but with no space after the colon, no line feed and a zero octet at the end. Nothing is written to the log. The record is cut to fit: at most *outsz* - 1 octets, then the zero. `STS$M_SYSLOG` is ignored (nothing is sent to the system log), and a severity out of the range 0 to 4 is shown as `?`. If *outsz* is less than 1 the buffer is not touched and *outlen* is set to 0. *outlen* must not be null.

Condition Values Returned

The routine returns the severity logged (`STS$M_SYSLOG` taken off, `STS$K_ERROR` for an undefined one, 5 to 7); it signals no condition of its own.

Example

```
char l_msg [256];
int  l_len;

__util$log2buf(l_msg, sizeof(l_msg), &l_len, "MYAPP", STS$K_WARN, "disk %d is full", 3);
```

__util$out – Write a Line to the Log Descriptor

Format

```
unsigned    __util$out  (char *fmt, ...);
```

Returns

`unsigned`: `STS$K_SUCCESS`.

Arguments

- *fmt* – `char *`; read only; by reference. A printf format.
- *...* – the arguments of *fmt*.

Description

Writes the formatted text and a line feed, with no prefix and no envelope: for a banner, a table, a help text. The line is cut at 1023 octets. The result of the `write()` is not tested.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

Example

```
__util$out("MYAPP version %s", "1.0");
```

__util$deflog – Direct the Log to a File and a Syslog Host

Format

```
int __util$deflog   (const char *, const char *);
```

Returns

`int`: a condition value.

Arguments

- *logfile* – `const char *`; read only; by reference. The path of the log file, or null or an empty string to leave the log where it is.
- *sloghost* – `const char *`; read only; by reference. The IPv4 address of a syslog server in dotted form, or null.

Description

If *sloghost* is not null, the address is stored for `__util$syslog()`, with the port 514. The address must be an IPv4 address in dotted form. Anything else (a host name, an IPv6 address) is logged as an error and ignored: the syslog host stays as it was, and with no host the syslog routine does nothing. In 1.6.11 a bad address was stored as zeros. **The routine only remembers the host.** No record is sent to it by the logging routines, `STS$M_SYSLOG` included (that flag calls the local `syslog()` of the C library); a record reaches the host only through `__util$syslog()`.

If *logfile* is not null and not empty, the file is opened for append (created with the mode 0644 if it does not exist; not inherited by child processes). The old file of an earlier call is closed. The new descriptor becomes the log descriptor, and **the standard output and the standard error are redirected to the same file** with `dup2()`. Then a record "Log file '…' has been opened." is written. The name of the file is kept for `__util$rotatelogfile()`; a name of 1024 octets or more is not kept and disables rotation.

If *logfile* is null or empty, the call changes nothing but the host and returns `STS$K_SUCCESS`. Call the routine once at start-up, before any thread logs. A program whose standard output carries data uses `__util$setlogfd()` instead.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS` (1)	No file was asked for, or the file was opened and the record written
`STS$K_ERROR` (2)	The file could not be opened; the log stays where it was and a record with `errno` is written there

Example

```
if ( !(1 & __util$deflog("/var/log/myapp.log", NULL)) )
    return  STS$K_ERROR;
```

__util$setlogfd – Direct the Log to an Open Descriptor

Format

```
int __util$setlogfd (int);
```

Returns

`int`: a condition value.

Arguments

- *a_fd* – `int`; read only; by value. An open descriptor.

Description

Makes *a_fd* the log descriptor. No file is opened, the standard streams are not touched, and the descriptor is not closed later. The log is not a file of ours after this call, so `__util$rotatelogfile()` does nothing. Typical use: `__util$setlogfd(STDERR_FILENO)` in a filter whose standard output carries data.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Done
`STS$K_ERROR`	*a_fd* is negative; the log is not changed

Example

```
__util$setlogfd(STDERR_FILENO);
```

__util$rewindlogfile – Truncate the Log File Past a Limit

Format

```
int __util$rewindlogfile    (size_t);
```

Returns

`int`: a condition value.

Arguments

- *limit* – `size_t`; read only; by value. The size in octets past which the file is truncated. Zero disables.

Description

If the log descriptor is not the standard output and *limit* is not zero, the current position of the descriptor is compared with *limit*. At or past it the file is truncated to zero and rewound. The old records are lost. The position is read as an `int`: the routine is meant for limits below 2 GB. Nothing is done on Windows (the position is rewound but the file is not truncated). `__util$rotatelogfile()` is the better routine for a file that must keep its history.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Done, or nothing to do
`STS$K_ERROR`	The position of the descriptor could not be read (for example it is a pipe)

__util$rotatelogfile – Rotate the Log File Past a Limit

Format

```
int __util$rotatelogfile    (size_t, unsigned);
```

Returns

`int`: a condition value.

Arguments

- *a_limit* – `size_t`; read only; by value. The size in octets past which the file is rotated. Zero: never.
- *a_keep* – `unsigned`; read only; by value. How many old files to keep, 0 to 99.

Description

Call it from time to time (every few seconds is enough) in a long-running program. If the log was opened by `__util$deflog()` and has grown to *a_limit* octets, the file is renamed to `<file>.1`, the older ones to `<file>.2` … `<file>.<a_keep>` (the oldest is dropped), and a new `<file>` is opened on the very same log descriptor, the standard output and the standard error with it. No data is copied. A record written by another thread at that moment lands in the old file or in the new one, whole. Only one thread rotates; a thread that finds a rotation under way returns `STS$K_SUCCESS` at once.

With *a_keep* = 0 the file is truncated in place, and the size is checked at every call.

The routine does nothing, and returns `STS$K_SUCCESS`, if *a_limit* is zero, the log is not a file opened by `__util$deflog()` (it is the standard output, or was set by `__util$setlogfd()`), or the file is still short of the limit. A record “Log file … has been rotated” or “… truncated” is written after a successful rotation.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Nothing to do, or another thread is rotating
`STS$K_INFO` (3)	The file was rotated or truncated; success (the low bit is set)
`STS$K_ERROR`	*a_keep* is above 99, or `fstat()`, `rename()`, `open()` or `dup2()` failed; the log goes on into the file it had

Example

```
for ( ;; )
    {
    __util$rotatelogfile(10 * 1024 * 1024, 5);  /* 10 MB, keep 5 */
    sleep(5);
    }
```

__util$syslog – Send a Message to a Syslog Host

Format

```
unsigned    __util$syslog   (int fac, int sev, const char *tag, const char *msg, int
↪  msglen);
```

Returns

`unsigned`: a condition value.

Arguments

- *fac* – `int`; read only; by value. The syslog facility code (`LOG_USER` and so on).
- *sev* – `int`; read only; by value. The syslog severity code (`LOG_DEBUG` and so on).
- *tag* – `const char *`; read only; by reference. The tag, as a program name.
- *msg* – `const char *`; read only; by reference. The text.
- *msglen* – `int`; read only; by value. Its length.

Description

Sends one UDP datagram, in the form of RFC 3164, to the host given to `__util$deflog()`, port 514: `<fac+sev> tag: text`. The tag is padded to 16 columns. The socket is created at the first call and kept (on Windows Winsock is started at the same time). The socket is created once, under a guard, so the first call may come from several threads at once. The routine is not called by the logging routines; call it yourself.

If `__util$deflog()` was not given a syslog host there is nowhere to send to, and the routine does nothing and returns `STS$K_SUCCESS`. It writes nothing to the standard error, calls neither `perror()` nor `abort()`, and never writes a message of its own, since the logging may be what called it. A failed creation of the socket is retried at the next call.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The datagram was sent, or no host is set and there is nothing to do
`STS$K_ERROR`	The socket could not be created, or `sendmsg()` failed

Example

```
__util$deflog(NULL, "192.168.1.10");
__util$syslog(LOG_USER, LOG_INFO, "myapp", "started", 7);
```

__util$trace – Write a Trace Record

Format

```
void    __util$trace    (int cond, const char *fmt, const char *mod, const char *func,
↪  unsigned line, ...);
```

Returns

None (`void`).

Arguments

- *cond* – `int`; read only; by value. The record is written only if this is not zero.
- *fmt* – `const char *`; read only; by reference. A printf format.
- *mod* – `const char *`; read only; by reference. The module name; null is allowed.
- *func* – `const char *`; read only; by reference. The routine name.
- *line* – `unsigned`; read only; by value. The line number.
- *...* – the arguments of *fmt*.

Description

Writes `prefix [mod\func:line]`, then pads with spaces to column 72 so that the texts of the records line up, then the text. With a null *mod* the form is `[func:line]`. When *mod* is not null the module and the routine are separated by one backslash, `[mod\func:line]`, as in the other routines (1.6.11 wrote two). The record is cut at 1023 octets. Normally called through `$TRACE()` and `$IFTRACE()`.

Condition Values Returned

None; the routine does not return a value.

$TRACE, $IFTRACE – Trace (Macros)

Format

```
#define $TRACE(fmt, ...)        __util$trace(1, fmt, __MODULE__, __FUNCTION__, __LINE__ ,
 ↪  ## __VA_ARGS__)   /* debug  */
#define $IFTRACE(cond, fmt, ...)    __util$trace(cond, fmt, __MODULE__, __FUNCTION__,
 ↪  __LINE__ , ## __VA_ARGS__)    /* debug  */
#define $TRACE(fmt, ...)        __util$nope()                               /* NDEBUG */
#define $IFTRACE(cond, fmt, ...)    ((void) sizeof(cond), __util$nope())
 ↪  /* NDEBUG */
```

Returns

`void`.

Arguments

- *cond* – integer expression. The trace is written if it is not zero.
- *fmt, ...* – a printf format and its arguments.

Description

A trace is for debugging. In a build with `NDEBUG` both macros compile to nothing, and *cond* is named in a `sizeof` so that a variable used only for tracing is not reported as unused; it is not evaluated. The macros are not redefined if the program has defined them already.

Example

```
$TRACE("entering, count = %u", l_count);
$IFTRACE(g_trace, "state %d", l_state);
```

__util$dumphex – Write a Hexadecimal Dump

Format

```
void    __util$dumphex  (const char *__fi, unsigned __li, const void *src, unsigned short
↪  srclen);
```

Returns

None.

Arguments

- *__fi* – `const char *`; read only; by reference. The routine name.
- *__li* – `unsigned`; read only; by value. The line number.
- *src* – `const void *`; read only; by reference. The data.
- *srclen* – `unsigned short`; read only; by value. The number of octets, up to 65535.

Description

Writes the header record `prefix [fi:li] Dump of N octets follows:`, then the data, 16 octets a line, each line 80 octets with its line feed:

```
\t+0010:  58 59 5a ...                                          XYZ
```

The line has the offset in hexadecimal, 16 columns of two hexadecimal digits, two spaces, and the same octets as characters, a dot for those that do not print. The last line is padded with spaces.

Condition Values Returned

None; the routine does not return a value.

$DUMPHEX – Write a Hexadecimal Dump (Macro)

Format

```
#define $DUMPHEX(s,l)   __util$dumphex(__FUNCTION__, __LINE__ , s, l)
```

Returns

`void`.

Arguments

- *s* – address; read only; by reference. The data.
- *l* – integer; read only; by value. The number of octets.

Description

Calls `__util$dumphex()` with the name of the current routine and the line.

Example

```
unsigned char l_hdr [24];

$DUMPHEX(l_hdr, sizeof(l_hdr));
```

2.3 Counted Strings and String Handling

ASC, ASC$K_SZ – The Counted String Type

Format

```
#define ASC$K_SZ    255

typedef struct _asc
{
    unsigned char   len;
    char    sts[ASC$K_SZ];
} ASC;
```

Description

A counted string: *len* is the number of octets of text, 0 to 255, and *sts* holds the text. The text need not end with a zero octet and may contain one. The routines that create an `ASC` of a C string also store a zero octet after the text, so that *sts* can be passed to a C function; a string of the full 255 octets has no room for it (see `__util$str2asc()`). `__ASC_TYPE__` is defined by the header to show that the type is defined here.

$ASCDECL, $ASCINI, $ASCLEN, $ASCPTR, $ASC, $ASCNIL – Counted String Macros

Format

```
#define $ASCDECL(a,sts) ASC a = {(unsigned char)sizeof(sts)-1,(sts)}
#define $ASCINI(a)  ((unsigned char) sizeof(a))-1,(a)
#define $ASCLEN(a)  (((ASC *) a)->len)
#define $ASCPTR(a)  (((ASC *) a)->sts)
#define $ASC(a)     (((ASC *) a)->len),(((ASC *) a)->sts)
#define $ASCNIL     0,0
```

Arguments

- *a* – for `$ASCDECL()` the name of the variable to declare; for `$ASCINI()` a string literal; for `$ASCLEN()`, `$ASCPTR()` and `$ASC()` a pointer to an `ASC`.
- *sts* – `$ASCDECL()` only: a string literal.

Description

`$ASCDECL(name, "text")` declares and initializes an `ASC` of a string literal. `$ASCINI("text")` expands to the first two initializers of an `ASC`, the length and the text, to be used inside braces (for example in an `OPTS` or a `KWDENT` table). `$ASCLEN(p)` and `$ASCPTR(p)` give the length and the text of the `ASC` that *p* points to; they can be assigned to. `$ASC(p)` expands to two values, the length and the address, for a `"%.*s"` conversion. `$ASCNIL` is two zeros, the initializers of an empty `ASC`.

The macros take a *pointer*. For a variable, write `$ASCLEN(&l_name)`.

Example

```
$ASCDECL(l_cmd, "SHOW");
ASC l_copy;

__util$cpyasc(&l_cmd, &l_copy);
$LOG(STS$K_INFO, "command '%.*s' (%u octets)", $ASC(&l_copy), $ASCLEN(&l_copy));
```

__util$str2asc – Copy a C String to a Counted String

Format

```
inline static int   __util$str2asc  (const char * a_src, void * a_dst);
```

Returns

`int`: the length of the text stored, 254 at the most.

Arguments

- *a_src* – `const char *`; read only; by reference. A zero-terminated string.
- *a_dst* – `void *`; write only; by reference. The address of an `ASC`.

Description

Copies *a_src* to the text of the `ASC`, sets its length, and stores a zero octet after the text. The `ASC` holds `ASC$K_SZ` (255) octets, and the zero octet needs one of them, so a longer text is cut at 254 octets and the returned length is 254. Nothing is written beyond the `ASC` (in 1.6.11 a text of 255 octets or more put the zero octet one octet past it).

Condition Values Returned

None; the routine returns a length.

Example

```
ASC l_name;

__util$str2asc("config.sys", &l_name);
```

__util$asc2str – Copy a Counted String to a C String

Format

```
inline static int   __util$asc2str  (ASC * src, char * dst);
```

Returns

`int`: the length of the text.

Arguments

- *src* – `ASC *`; read only; by reference. The counted string.
- *dst* – `char *`; write only; by reference. The buffer; it must hold the length plus one.

Description

Copies the text of *src* to *dst* and stores a zero octet after it.

Condition Values Returned

None; the routine returns a length.

__util$cpyasc – Copy a Counted String

Format

```
inline static int   __util$cpyasc   (const ASC * a_src, const ASC * a_dst);
```

Returns

`int`: the length copied.

Arguments

- *a_src* – `const ASC *`; read only; by reference. The source.
- *a_dst* – `const ASC *`; write only; by reference. The destination. It is declared `const` and is written all the same: pass the address of a non-constant `ASC`.

Description

Copies the length and the text, with `memmove()`, so the two may overlap. No zero octet is stored after the text.

Condition Values Returned

None; the routine returns a length.

__util$cmpasc – Compare Counted Strings

Format

```
inline static int   __util$cmpasc   (ASC * s1, ASC * s2);
```

Returns

`int`: zero if the strings are equal; non-zero otherwise.

Arguments

- *s1*, *s2* – `ASC *`; read only; by reference. The strings.

Description

If the lengths differ, the result is the length of *s1* minus the length of *s2*. If they are equal, it is the result of `memcmp()` of the text. The order is therefore by length first and is **not** the alphabetical order. Use the routine to test for equality.

Condition Values Returned

None; the routine returns a comparison result.

__util$cmpasc_blind – Compare Counted Strings, Ignoring Case

Format

```
inline static int   __util$cmpasc_blind (const ASC * s1, const ASC * s2);
```

Returns

`int`: zero if the strings are equal when the case is ignored; non-zero otherwise.

Arguments

- *s1*, *s2* – `const ASC *`; read only; by reference. The strings.

Description

As `__util$cmpasc()`, but the text is compared with `strncasecmp()`. On Windows it is compared with `_stricmp()`, which reads up to a zero octet, so on Windows the text must be zero-terminated.

Condition Values Returned

None; the routine returns a comparison result.

__util$strelem – Take an Element of a Delimited String

Format

```
inline static int   __util$strelem  (const ASC *a_src, int a_separator, int a_idx, ASC
↪  *a_dst);
```

Returns

`int`: the length of the element found; 0 if there is none.

Arguments

- *a_src* – `const ASC *`; read only; by reference. The string.
- *a_separator* – `int`; read only; by value. The delimiter octet.
- *a_idx* – `int`; read only; by value. The number of the element, from 0.
- *a_dst* – `ASC *`; write only; by reference. Receives the element. It is always set: an empty `ASC` when there is no such element.

Description

Cuts element number *a_idx* out of a string of elements separated by *a_separator*, and stores it in *a_dst*. The routine is the counterpart of the `F$ELEMENT` of DCL. It is right for every index (1.6.11 was dependable for the elements 0 and 1 only, and could read past the end of the string). The search never leaves the string. The element is followed by a zero octet when there is room for it. A negative index, or an index past the last element, gives an empty *a_dst* and the length 0. An empty element has the length 0 as well, the same as a missing one: `"a,b,,d"` has an empty element 2 and the result cannot tell it from the element 7.

Condition Values Returned

None; the routine returns a length.

Example

```
$ASCDECL(l_list, "alpha,beta");
ASC l_item;

if ( __util$strelem(&l_list, ',', 1, &l_item) )
    $LOG(STS$K_INFO, "second: %.*s", $ASC(&l_item));
```

__util$uint32_2_dec – Convert an Unsigned Integer to Decimal Text

Format

```
inline  static int  __util$uint32_2_dec (unsigned int a_src, char *a_dst, size_t a_dstsz,
↪  size_t *a_retlen);
```

Returns

`int`: the number of digits stored.

Arguments

- *a_src* – `unsigned int`; read only; by value. The value.
- *a_dst* – `char *`; write only; by reference. The buffer.
- *a_dstsz* – `size_t`; read only; by value. The size of the buffer; must not be 0.
- *a_retlen* – `size_t *`; write only; by reference. Receives the length; may be null.

Description

Writes the decimal digits of *a_src* and a terminating zero. The value 0 gives “0”. If the digits do not fit, the most significant ones are kept: 4000000000 into a buffer of 8 octets gives “4000000”. An *a_dstsz* of 0 stops a debug build at an `assert()`.

Condition Values Returned

None; the routine returns a length.

__util$trim – Remove Leading and Trailing White Space

Format

```
inline static int   __util$trim (char *src, int srclen);
```

Returns

`int`: the length of the text that remains; 0 if *src* is null, *srclen* is 0, or only white space was there.

Arguments

- *src* – `char *`; modify; by reference. The text.
- *srclen* – `int`; read only; by value. Its length.

Description

Removes the white space (`isspace()`: space, tab, line feed, vertical tab, form feed, carriage return) from both ends of the text, in place. The text is moved to the start of the buffer and a zero octet is stored after it. The octets are passed to `isspace()` as `char`: do not use the routine for text with octets above 0x7F on a platform where `char` is signed.

Condition Values Returned

None; the routine returns a length.

Example

```
char l_buf [] = "  value \n";
int  l_len = __util$trim(l_buf, (int) strlen(l_buf));   /* "value", 5 */
```

__util$collapse – Remove All White Space

Format

```
inline static int   __util$collapse (char *src, int srclen);
```

Returns

`int`: the length of the text that remains; 0 if *src* is null or *srclen* is 0.

Arguments

- *src* – `char *`; modify; by reference. The text.
- *srclen* – `int`; read only; by value. Its length.

Description

Removes every octet for which `isspace()` is true, closing the gaps. When something was removed, a zero octet is stored where the last removed one stood, so the text is a C string. When nothing was removed there is no room for a zero octet and none is stored: the routine never writes beyond the *srclen* octets it was given (in 1.6.11 it stored the zero at *src*[*srclen*] and the buffer had to be one octet longer).

Condition Values Returned

None; the routine returns a length.

__util$uncomment – Cut a Comment Off a Line

Format

```
inline static int   __util$uncomment    (char *src, int srclen, char marker);
```

Returns

`int`: the length of the text before the comment; *srclen* if there was no comment; 0 if *src* is null or *srclen* is 0.

Arguments

- *src* – `char *`; modify; by reference. The line.
- *srclen* – `int`; read only; by value. Its length.
- *marker* – `char`; read only; by value. The octet that starts a comment, for example `'!'`.

Description

Finds the first *marker* that is not inside a quoted part, stores a zero octet there and returns its offset. A part in double quotes (`"..."`) or in single quotes (`'...'`) is skipped as it stands: the other kind of quote inside it is an ordinary octet, there is no escape, and a quote left open runs to the end of the line, so that nothing is cut. (In 1.6.11 a marker inside quotes started a comment too.) A line that begins with the marker gives 0. The text before the comment is not trimmed; follow with `__util$trim()`.

Condition Values Returned

None; the routine returns a length.

Example

```
char l_line [] = "-port=80   ! the port";

__util$uncomment(l_line, (int) strlen(l_line), '!');    /* "-port=80   " */
```

__util$strcat_rx – Concatenate a List of Strings

Format

```
inline static   int __util$strcat_rx    (void * dst, int dstsz, int * dstlen, ...);
```

Returns

`int`: a condition value.

Arguments

- *dst* – `void *`; write only; by reference. The area to receive the result.
- *dstsz* – `int`; read only; by value. The size of the area.
- *dstlen* – `int *`; write only; by reference. Receives the length of the result.
- *...* – pairs of arguments: a `char *` address and an **`int`** length (not `size_t`), ending with a null address. A pair with the length 0 is skipped.

Description

Copies the pieces one after another into *dst* and stores a zero octet after them. One octet of the area is kept for the zero octet, so the pieces may fill it up to *dstsz* - 1 octets exactly, and that is a success. If a piece does not fit, the copy stops there: the pieces before it are kept, *dstlen* holds their length and the text is zero-terminated. Nothing is ever written at *dst*[*dstsz*]. A negative length is a piece that does not fit. A null *dst*, a null *dstlen* or a *dstsz* below 1 is an error (1.6.11 returned an error and wrote beyond the area when the pieces filled it exactly).

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	All the pieces were copied
`STS$K_ERROR`	A piece did not fit, or an argument was bad

Example

```
char l_path [128];
int  l_len;

__util$strcat_rx(l_path, sizeof(l_path), &l_len, "/etc/", 5, "myapp", 5, ".conf", 5,
 ↪  (char *) NULL);
```

__util$strstr – Find a Counted String in a Counted String

Format

```
char *  __util$strstr   (char *s1, size_t s1len, char *s2, size_t s2len);
```

Returns

`char *`: the address of the first occurrence; null if there is none.

Arguments

- *s1* – `char *`; read only; by reference. The area to search.
- *s1len* – `size_t`; read only; by value. Its length.
- *s2* – `char *`; read only; by reference. The pattern.
- *s2len* – `size_t`; read only; by value. Its length.

Description

Like `strstr()` but with lengths, so neither operand needs a zero octet and either may contain one. A null address, a length of 0 or *s2len* greater than *s1len* gives null. The comparison is exact, case matters.

Condition Values Returned

None; the routine returns an address or null.

Example

```
char *l_p = __util$strstr(l_buf, l_buflen, CRLFCRLF, 4);    /* the end of an HTTP header
 ↪  */
```

CRLF, CRLFCRLF, CRLFCRLF_LW – Line Terminator Constants

Format

```
#define CRLFCRLF_LW 0x0a0d0a0d
#define CRLFCRLF    "\r\n\r\n"
#define CRLF        "\r\n"
```

Description

A carriage return and a line feed, as a string; twice, as a string; twice, as a longword in the byte order of an x86 or an ARM in little-endian mode, for a one-compare test of four octets.

2.4 Memory

__util$movc5 – Move Characters with Fill

Format

```
static inline int   __util$movc5    (unsigned short *srclen, char *source, unsigned char
 ↪  *fill, unsigned short *dstlen, char *dest);
```

Returns

`int`: a condition value.

Arguments

- *srclen* – `unsigned short *`; read only; by reference. The length of the source.
- *source* – `char *`; read only; by reference. The source.
- *fill* – `unsigned char *`; read only; by reference. The fill octet; a null address means a zero octet.
- *dstlen* – `unsigned short *`; read only; by reference. The length of the destination.
- *dest* – `char *`; write only; by reference. The destination.

Description

The MOVC5 instruction of VAX as a routine, in the manner of `LIB$MOVC5`. `min(*srclen, *dstlen)` octets are copied from the source to the destination with `memmove()`, so the areas may overlap. If the destination is longer than the source, the rest of it is filled with the fill octet. If it is shorter, the extra part of the source is not moved. The lengths are 0 to 65535.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

Example

```
unsigned short l_srclen = 5, l_dstlen = 16;
unsigned char  l_fill = ' ';
char l_name [16];

__util$movc5(&l_srclen, "alpha", &l_fill, &l_dstlen, l_name);   /* "alpha" and 11 spaces
 ↪  */
```

__util$fill – Fill a Buffer with a Pattern

Format

```
static inline   int __util$fill (void * dst, int dstsz, const void * src, int srcsz);
```

Returns

`int`: a condition value.

Arguments

- *dst* – `void *`; write only; by reference. The buffer.
- *dstsz* – `int`; read only; by value. Its size.
- *src* – `const void *`; read only; by reference. The pattern.
- *srcsz* – `int`; read only; by value. Its size; must be greater than 0.

Description

Fills *dst* with copies of the pattern, one after another; the last copy is cut to fit. A pattern of zero (or negative) size would never fill anything and is refused (in 1.6.11 the routine looped forever on it).

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Filled
`STS$K_ERROR`	*dst* or *src* is null, *dstsz* is negative, or *srcsz* is not above 0

__util$bzero – Clear Memory

Format

```
inline static int __util$bzero  (void * bufp, size_t bufsz);
```

Returns

`int`: a condition value.

Arguments

- *bufp* – `void *`; write only; by reference. The area.
- *bufsz* – `size_t`; read only; by value. Its size.

Description

Stores a zero in each of the *bufsz* octets (it is `memset()` to zero). In 1.6.11 the last *bufsz* modulo 4 octets were left as they were; every octet is cleared now, whatever the length and the alignment. A length of 0 is a success and the address is not looked at.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Done
`STS$K_ERROR`	*bufp* is null and *bufsz* is not 0

__util$iszero – Test Memory for Zeros

Format

```
inline static int __util$iszero (void * bufp, int bufsz);
```

Returns

`int`: a condition value.

Arguments

- *bufp* – `void *`; read only; by reference. The area.
- *bufsz* – `int`; read only; by value. Its size.

Description

Tests the area eight octets at a time (through `memcpy()`, so an unaligned address is fine) and then the remaining 0 to 7 octets one by one. Every octet is looked at on every platform, including those where `long` has 4 octets (in 1.6.11 up to 4 octets could be left untested there). There is nothing to see in a null area or in a length of 0, and both count as all zero.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Every octet is zero (or the length is 0, or *bufp* is null)
`STS$K_WARN`	A non-zero octet was found

Example

```
if ( __util$iszero(l_key, sizeof(l_key)) == STS$K_SUCCESS )
    $LOG(STS$K_WARN, "the key is not set");
```

__util$strxor – Exclusive-OR Data with a Key

Format

```
static inline   int __util$strxor   (void * key, int keysz, void * src, int srcsz, void *
↪  dst, int dstsz, int * ctx);
```

Returns

`int`: a condition value.

Arguments

- *key* – `void *`; read only; by reference. The key.
- *keysz* – `int`; read only; by value. Its length; must be greater than 0.
- *src* – `void *`; read only; by reference. The data.
- *srcsz* – `int`; read only; by value. Its length.
- *dst* – `void *`; write only; by reference. The result; it may be the same as *src*.
- *dstsz* – `int`; read only; by value. The size of the result area.
- *ctx* – `int *`; modify; by reference. The position in the key, kept between calls. May be null.

Description

XORs `min(srcsz, dstsz)` octets of *src* with the key, repeating the key as needed. If *ctx* is not null, the position in the key is taken from it at the start and stored back at the end, so that a stream can be processed in pieces: the result is the same as for the whole at once. Start with `*ctx` = 0. A value of `*ctx` above *keysz* is refused.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Done
`STS$K_ERROR`	`*ctx` is greater than *keysz*

Example

```
int l_pos = 0;

__util$strxor(l_key, l_keylen, l_buf, l_n, l_buf, l_n, &l_pos); /* in place, a piece of a
↪  stream */
```

$ARRSZ – Number of Elements of an Array

Format

```
#define $ARRSZ(a)   (sizeof(a)/sizeof(a[0]))
```

Returns

`size_t`: the number of elements.

Arguments

- *a* – an array (not a pointer); read only. The macro evaluates to a constant; the array is not evaluated.

Example

```
KWDENT l_tbl [] = { ... };

for ( size_t i = 0; i < $ARRSZ(l_tbl); i++ )
    ;
```

2.5 Pattern Matching

__util$pattern_match – Match a String Against a Wildcard Pattern

Format

```
int __util$pattern_match    (char * a_str, char * a_pattern);
```

Returns

`int`: **not** a condition value. 1 if the string matches, 0 if it does not (also for a null argument).

Arguments

- *a_str* – `char *`; read only; by reference. The zero-terminated subject.
- *a_pattern* – `char *`; read only; by reference. The zero-terminated pattern.

Description

The whole subject must match the whole pattern. In the pattern:

Octet	Matches
*	Any run of octets, including none
% or ?	Exactly one octet
any other	Itself; the comparison is case-sensitive

There is no escape: a backslash is an ordinary octet, and `?` is always a wildcard. The matcher is iterative; its work is limited by the product of the lengths and it does not recurse. The wildcards are in the **pattern** only: a `*`, `%` or `?` in the subject is an ordinary octet. The subject `*ba` matches the pattern `*a`, and the pattern `*b*` matches it too. (In 1.6.11 a `*` of the subject facing a `*` of the pattern was taken literally and `*ba` did not match `*a`.)

Condition Values Returned

None; the routine returns 1 for a match and 0 otherwise, and 0 for a null argument.

Example

```
if ( __util$pattern_match(l_name, "SYS$*.COM") )
    $LOG(STS$K_INFO, "'%s' is a system procedure", l_name);
```

2.6 Hexadecimal, UUID, Base64 and CRC

__util$hex2bin – Convert Hexadecimal Text to Binary

Format

```
inline  static int __util$hex2bin   (const void * a_srchex, void * a_dstbin, unsigned
↪  short a_srchexlen);
```

Returns

`int`: the number of octets stored, *a_srchexlen* / 2; -1 for an invalid input.

Arguments

- *a_srchex* – `const void *`; read only; by reference. The text, in either case.
- *a_dstbin* – `void *`; write only; by reference. The result.
- *a_srchexlen* – `unsigned short`; read only; by value. The number of digits; it must be even.

Description

Converts two digits to one octet, in either case. The input is checked first: an **odd** number of digits, or a character that is not a hexadecimal digit (a blank, `0x`, a sign), makes the routine return -1 **with nothing written**, so the destination is never half filled. (Up to 1.7.0 the digits were not checked, and "abc" gave 0x0A 0xBC.) The text need not end with a zero; only *a_srchexlen* octets are read. For *a_srchexlen* 0 nothing is stored and the result is 0.

Condition Values Returned

None; the routine returns a length, or -1.

Example

```
unsigned char l_id [4];

__util$hex2bin("c0a80102", l_id, 8);
```

__util$bin2hex – Convert Binary to Hexadecimal Text

Format

```
inline  static int __util$bin2hex   (const void * a_srcbin, void * a_dsthex, unsigned
↪  short a_srcbinlen);
```

Returns

`int`: the number of characters stored, 2 * *a_srcbinlen*, without the zero.

Arguments

- *a_srcbin* – `const void *`; read only; by reference. The octets.
- *a_dsthex* – `void *`; write only; by reference. The text. It must hold 2 * *a_srcbinlen* + 1 octets.
- *a_srcbinlen* – `unsigned short`; read only; by value. The number of octets.

Description

Writes two lower-case hexadecimal digits for each octet and a terminating zero.

Condition Values Returned

None; the routine returns a length.

$BIN2HEX – Convert Binary to Hexadecimal (Macro)

Format

```
#define $BIN2HEX(s,d,l) __util$bin2hex((char*) s, (char*) d, (unsigned short) l)
```

Description

Calls `__util$bin2hex()` with the casts. Arguments as the routine's; *s* is the source, *d* the destination, *l* the length of the source.

Example

```
char l_hex [2 * 16 + 1];

$BIN2HEX(l_uuid, l_hex, 16);
```

__util$faohex – Format Octets as One Hexadecimal Dump Line

Format

```
int __util$faohex   (const void *src, unsigned short srclen, char *out, unsigned short
↪  outsz, unsigned short *outlen);
```

Returns

`int`: the number of characters written; 0 if the buffer is too small or an address is null.

Arguments

- *src* – `const void *`; read only; by reference. The octets.
- *srclen* – `unsigned short`; read only; by value. How many; more than 16 are taken as 16.
- *out* – `char *`; write only; by reference. The buffer.
- *outsz* – `unsigned short`; read only; by value. Its size; at least 67 octets.
- *outlen* – `unsigned short *`; write only; by reference. Receives the length. Optional: it may be null (in 1.6.11 a null address was dereferenced).

Description

Formats up to 16 octets as 16 columns of two hexadecimal digits and a space (48 characters), two spaces, and 16 characters of the same octets with a dot for those that do not print: 66 characters in all, and a zero octet. The unused columns are spaces. The routine never reads beyond *srclen* octets (or beyond 16 of them).

Condition Values Returned

None; the routine returns a length. 0 means that *outsz* was less than 67, or that *out* or *src* was null.

Example

```
char l_line [80];
unsigned short l_len;

__util$faohex(l_buf, 16, l_line, sizeof(l_line), &l_len);
```

__util$uuid2str – Convert a UUID to Text

Format

```
inline static int   __util$uuid2str (void * uuid, char * buf, int bufsz);
```

Returns

`int`: the length of the text stored.

Arguments

- *uuid* – `void *`; read only; by reference. 16 octets.
- *buf* – `char *`; write only; by reference. The buffer.
- *bufsz* – `int`; read only; by value. The number of characters it may receive, without the zero.

Description

Writes the UUID in the form `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` in lower case, in the order of the octets, with no change of byte order, and a terminating zero. At most *bufsz* characters are written; the zero follows them, so the buffer must hold *bufsz* + 1 octets (37 for the whole text).

Condition Values Returned

None; the routine returns a length (36 for a whole UUID).

Example

```
char l_text [37];

__util$uuid2str(l_uuid, l_text, 36);
```

__util$base64 – Encode Octets as Base64

Format

```
int __util$base64   (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

Returns

`int`: a condition value.

Arguments

- *src* – `void *`; read only; by reference. The data.
- *srclen* – `int`; read only; by value. Its length in octets.
- *dst* – `void *`; write only; by reference. The output buffer.
- *dstsz* – `int`; read only; by value. Its size.
- *dstlen* – `int *`; write only; by reference. Receives the length of the text; must not be null.

Description

Encodes with the alphabet of RFC 4648 (A-Z, a-z, 0-9, +, /), padded with = to a multiple of four characters: 4 characters for each 3 octets, rounded up. *dstlen* is set to the length that is needed even if *dstsz* is too small, and then nothing is stored. A zero octet is stored after the text if there is room for it beyond the text; it is not counted in *dstlen*.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Encoded
`STS$K_ERROR`	*dstlen* is null; a length is negative; *src* or *dst* is null while its length is not 0; *dstsz* is too small (then *dstlen* holds the size needed)

Example

```
char l_text [64];
int  l_len;

if ( 1 & __util$base64("Aladdin:open sesame", 19, l_text, sizeof(l_text), &l_len) )
    $LOG(STS$K_INFO, "%.*s", l_len, l_text);
```

__util$unbase64 – Decode Base64

Format

```
int __util$unbase64 (void *src, int srclen, void *dst, int dstsz, int *dstlen);
```

Returns

`int`: a condition value.

Arguments

- *src* – `void *`; read only; by reference. The text.
- *srclen* – `int`; read only; by value. Its length in characters.
- *dst* – `void *`; write only; by reference. The output buffer.
- *dstsz* – `int`; read only; by value. Its size.
- *dstlen* – `int *`; write only; by reference. Receives the number of octets decoded; must not be null.

Description

Decodes the text of RFC 4648. The = padding is accepted at the end and may be absent; a final group of two or three characters is decoded as it stands. The decoder is strict: a character outside the alphabet (white space too), a = anywhere but at the end, more than two of them, padding that does not complete the group, or a final group of one character is an error. The result is not terminated by a zero. *dstlen* is set to the size needed before the text is checked, and the output is written while it is checked: after an error the buffer may hold part of a result.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Decoded
`STS$K_ERROR`	*dstlen* is null; bad arguments as in `__util$base64()`; malformed text; *dstsz* too small

__util$crc32c – Compute a CRC-32

Format

```
unsigned    __util$crc32c   (unsigned int crc, const void *buf, size_t buflen);
```

Returns

`unsigned`: the checksum. **Not** a condition value.

Arguments

- *crc* – `unsigned int`; read only; by value. 0 to start, or the result of a previous call to continue.
- *buf* – `const void *`; read only; by reference. The data.
- *buflen* – `size_t`; read only; by value. Its length.

Description

Computes the CRC-32 of IEEE 802.3 and zlib (the reflected polynomial 0xEDB88320), **not** the Castagnoli polynomial that the name suggests. The name is kept, since programs depend on it; the header comment now says what the function is. The check value for "123456789" is 0xCBF43926. The result is the same on every platform. It is computed with the fastest engine the processor has: carry-less multiplication on x86-64 (PCLMULQDQ and SSE 4.1), the CRC32 instructions on ARMv8 under Linux, else slice-by-8 tables. The engine is chosen at the first call; the environment variable `STARLET_CRC32=SOFTWARE` forces the tables. A buffer of any alignment and length is accepted. A long buffer may be passed in pieces: the result of a piece is the *crc* of the next.

Condition Values Returned

None; the routine returns the checksum.

Example

```
unsigned l_crc = 0;

l_crc = __util$crc32c(l_crc, l_part1, l_len1);
l_crc = __util$crc32c(l_crc, l_part2, l_len2);
```

2.7 Keywords and Symbols

KWDENT, UTIL$K_LOOKUP_* – Keyword Table Types

Format

```
#define UTIL$K_LOOKUP_NCASE (1 << 0)
#define UTIL$K_LOOKUP_ABBREV    (1 << 1)

typedef struct _kwdent
{
    ASC kwd;
    union   {
        int val;
        void *  ptr;
        };
} KWDENT;
```

Description

An entry of a keyword table: the keyword as a counted string, and the value it stands for, an integer or an address (the two share the storage). A table is an array of entries; it need not be sorted. Two entries with the same value are *synonyms*, for example `SHOW` and `SH`. Fill every entry in the same way: synonyms are told by comparing the whole union.

`UTIL$K_LOOKUP_NCASE` makes the comparison ignore the case. `UTIL$K_LOOKUP_ABBREV` lets the string be the beginning of a keyword.

Example

```
enum { K_SHOW = 1, K_SET, K_SETUP };

static KWDENT s_cmds [] = {
    { {$ASCINI("SHOW")},    {.val = K_SHOW} },
    { {$ASCINI("SH")},  {.val = K_SHOW} },
    { {$ASCINI("SET")}, {.val = K_SET} },
    { {$ASCINI("SETUP")},   {.val = K_SETUP} },
};
```

__util$lookup_key – Look a Keyword Up in a Table

Format

```
inline  static  int __util$lookup_key   (const char *a_src, int a_srclen, KWDENT *a_ktbl,
↪  int a_ktblsz, KWDENT **a_kwd, int a_flags);
```

Returns

`int`: a condition value.

Arguments

- *a_src* – `const char *`; read only; by reference. The string to look up; it need not end with a zero.
- *a_srclen* – `int`; read only; by value. Its length.
- *a_ktbl* – `KWDENT *`; read only; by reference. The table.
- *a_ktblsz* – `int`; read only; by value. The number of entries in the table.
- *a_kwd* – `KWDENT **`; write only; by reference. Receives the address of the entry found, an address inside the table. It is not changed when the routine returns `STS$K_ERROR`.
- *a_flags* – `int`; read only; by value. 0, or `UTIL$K_LOOKUP_NCASE`, `UTIL$K_LOOKUP_ABBREV`, or both.

Description

Without `UTIL$K_LOOKUP_ABBREV` the string must equal a keyword in its whole length: `SETU` is not `SETUP`, and `SET` is found whatever its place in the table. With `UTIL$K_LOOKUP_NCASE` the case is ignored; without it the comparison is exact.

With `UTIL$K_LOOKUP_ABBREV` the string may be the beginning of a keyword, as DCL takes a command:

- a keyword spelled out in full wins, whatever else the string is the beginning of (`SET` selects `SET`, though it begins `SETUP`);
- otherwise, if the string is the beginning of one keyword only, that keyword is selected;
- keywords with the same value (synonyms) count as one: `S` for `SHOW` and `SH` selects the first of them;
- if the string is the beginning of keywords of different values the abbreviation is ambiguous. The routine returns `STS$K_WARN` and *a_kwd* is the first entry that matches.

An empty string (*a_srclen* not above 0) is an error. A string longer than a keyword never matches it. The result does not depend on the order of the entries, except that *a_kwd* is the first match for an ambiguous abbreviation and for synonyms. Note that `STS$K_WARN` is 0: the low bit is clear, so `1 & status` is false, although *a_kwd* is set.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Found; *a_kwd* is the entry
`STS$K_WARN`	ABBREV only: the abbreviation is ambiguous; *a_kwd* is the first entry it matches
`STS$K_ERROR`	The string is empty, or matches no keyword; *a_kwd* is not changed

Example

```
KWDENT *l_kwd;
int     l_flags = UTIL$K_LOOKUP_NCASE | UTIL$K_LOOKUP_ABBREV;

switch ( __util$lookup_key("sh", 2, s_cmds, $ARRSZ(s_cmds), &l_kwd, l_flags) )
    {
```

```
    case STS$K_SUCCESS:
        $LOG(STS$K_INFO, "command %d", l_kwd->val);
        break;

    case STS$K_WARN:
        $LOG(STS$K_WARN, "ambiguous, could be %.*s", $ASC(&l_kwd->kwd));
        break;

    default:
        $LOG(STS$K_ERROR, "unknown command");
    }
```

SYM_REC, $SYM_REC_INI, $SYM_REC_EOL – Symbol Table Types

Format

```
typedef struct __sym_rec__ {
    unsigned long long  val;
    unsigned char       len;
             char    *sym;
} SYM_REC;

#define $SYM_REC_INI(s) {s, sizeof(#s)-1,  #s}
#define $SYM_REC_EOL    {0, 0, NULL}
```

Description

A symbol table maps a value or a bit of a mask to its name. `$SYM_REC_INI(name)` makes an entry of a constant and its own name as a string (the name is stringised, so a macro is not expanded in it). `$SYM_REC_EOL` ends the table. A table for `__util$mask2sym()` ends at the first entry whose value is 0 or whose name is null; a table for `__util$val2sym()` ends at the first entry whose length is 0 or whose name is null. End both kinds with `$SYM_REC_EOL`; keep a symbol whose value is 0 out of a mask table.

Example

```
enum { F_READ = 1, F_WRITE = 2, F_EXEC = 4 };

static const SYM_REC s_flags [] = {
    $SYM_REC_INI(F_READ), $SYM_REC_INI(F_WRITE), $SYM_REC_INI(F_EXEC), $SYM_REC_EOL
};
```

__util$val2sym – Translate a Value to Its Name

Format

```
static inline int   __util$val2sym  (unsigned long long val, const SYM_REC *sym, char
↪  *out, int outsz);
```

Returns

`int`: the length of the name stored; 0 if the value is not in the table.

Arguments

- *val* – `unsigned long long`; read only; by value. The value.
- *sym* – `const SYM_REC *`; read only; by reference. The table.
- *out* – `char *`; write only; by reference. The buffer.
- *outsz* – `int`; read only; by value. Its size.

Description

Stores the name of the first entry whose value is equal to *val*, and a terminating zero. If there is no such entry, or the name does not fit (it needs *outsz* greater than its length), an empty string is stored and the result is 0. The zero octet is stored in every case, so *out* must hold at least one octet.

Condition Values Returned

None; the routine returns a length.

__util$mask2sym – Translate a Mask to Names

Format

```
static inline int   __util$mask2sym (unsigned long long mask, const SYM_REC *sym, char
↪  *out, int outsz);
```

Returns

`int`: the length of the text stored.

Arguments

- *mask* – `unsigned long long`; read only; by value. The mask.
- *sym* – `const SYM_REC *`; read only; by reference. The table of bits.
- *out* – `char *`; write only; by reference. The buffer.
- *outsz* – `int`; read only; by value. Its size.

Description

Stores the names of all the entries whose value has a bit in common with *mask*, separated with `|`, in the order of the table, and a terminating zero. The test for room counts the separator and the zero octet: a name is stored only if it, the `|` before it and the zero fit. When a name does not fit the routine stops; what was stored stays, and the text is always zero-terminated. A buffer smaller than 1 octet is not touched and the length is 0. (In 1.6.11 the test did not count the separator and the zero octet, and the buffer could be overrun.)

Condition Values Returned

None; the routine returns a length.

Example

```
char l_text [64];

__util$mask2sym(F_READ | F_EXEC, s_flags, l_text, sizeof(l_text));  /* "F_READ|F_EXEC" */
```

2.8 Parameters and Configuration

OPTS, OPTS$K_*, OPTS_NULL – Option Table Types

Format

```
#define OPTS$K_INT  0
#define OPTS$K_STR  1
#define OPTS$K_OPT  2
#define OPTS$K_PORT 3
#define OPTS$K_IP4  4
#define OPTS$K_CONF 5
#define OPTS$K_PWD  6

typedef struct _opts    {
        ASC name;
        void *  ptr;
        size_t  sz;
        int type;
    } OPTS;

#define OPTS_NULL { {0, 0}, NULL, 0, 0}
```

Description

An entry of a table of the options of a program: *name* is the name, *ptr* the variable that receives the value, *sz* its size, and *type* how the value is converted. The table ends with `OPTS_NULL` (an entry whose name has the length 0).

Type	*ptr* designates	The value
`OPTS$K_INT`	an integer of exactly *sz* octets: 1, 2, 4 or 8; *sz* 0 means an `int`	A decimal, `0x` hexadecimal or `0` octal number. Sizes 1, 2 and 8 are **unsigned**: a leading `-` is refused and the range is 0 to 2^n-1. Sizes 4 and 0 are **signed**: a minus sign is taken, the range is -2147483648 to 2147483647. Nothing may follow the number; otherwise it is a bad value and the variable is not changed
`OPTS$K_STR`	an `ASC`; *sz* the number of characters it may take	The text, cut at the smaller of *sz* and `ASC$K_SZ` - 1 (254), with a zero octet after it
`OPTS$K_OPT`	an `int`	No value: the option present sets the `int` to 1. Set it to 0 beforehand
`OPTS$K_PORT`	an `unsigned short`	A number 0 to 65535; **host byte order** (use `htons()` to put it on the wire). A larger number is a bad value
`OPTS$K_IP4`	a `struct in_addr`	A dotted address; network byte order, as the structure has it. A bad address is logged as an error and the variable is left

Type	*ptr* designates	The value
`OPTS$K_CONF`	an `ASC`, or null	`__util$getparams()` only: the file name; the file is read at once with `__util$readconfig()`. Inside a configuration file the option is refused with a message (no nesting)
`OPTS$K_PWD`	an `ASC`	As `OPTS$K_STR`; `__util$showparams()` does not show it

Give *sz* of an `OPTS$K_INT` as the size of the variable: 1 for a `char`, 2 for a `short`, 4 for an `int` or 8 for a `long long`; 0 is an `int`. Any other size is an error. A 32-bit mask above 2147483647 does not fit an `int` and is a bad value (up to 1.7.0 it was stored as a negative number, and a minus sign was taken for the sizes 1, 2 and 8 as well): give the option the size 8. The routines store exactly that many octets, so a `short` is no longer overrun (in 1.6.11 every size but 8 was written as an `int`). An `OPTS$K_PORT` is always an `unsigned short`, its *sz* is not used. The header comment of 1.6.11 said “network order” for the port; it was never so, and the comment now says host order.

Example

```
static ASC  g_logfile, g_cfgfile;
static int  g_verbose, g_count;
static unsigned short g_port = 8080;

static OPTS s_opts [] = {
    { {$ASCINI("config")},  &g_cfgfile, ASC$K_SZ - 1,   OPTS$K_CONF },
    { {$ASCINI("logfile")}, &g_logfile, ASC$K_SZ - 1,   OPTS$K_STR },
    { {$ASCINI("verbose")}, &g_verbose, 0,      OPTS$K_OPT },
    { {$ASCINI("count")},   &g_count,   0,      OPTS$K_INT },
    { {$ASCINI("port")},    &g_port,    0,      OPTS$K_PORT },
    OPTS_NULL
};
```

__util$getparams – Read the Options from the Command Line

Format

```
int __util$getparams    (int, char *[], const OPTS *);
```

Returns

`int`: a condition value.

Arguments

- *argc* – `int`; read only; by value. The number of arguments.
- *argv* – `char *[]`; read only; by reference. The arguments. They are never modified (in 1.6.11 the values were trimmed in place).
- *opts* – `const OPTS *`; read only; by reference. The table of options. The variables it names are written.

Description

Goes over `argv[1]` ... `argv[argc-1]`. An argument is `-name=value`, `/name=value` or `--name=value`; the value is optional for `OPTS$K_OPT`. The name is looked up in the table without regard to the case. A name spelled out in full wins; otherwise the text may be an abbreviation, the leading part of exactly one option name (the rule of DCL, the same as `__util$lookup_key()` with `UTIL$K_LOOKUP_ABBREV`). An abbreviation which fits several options is ambiguous and is an error. The value is trimmed of white space (in a copy: *argv* is not changed) and converted by the type of the entry. When the same option is given twice, the last one wins.

An empty argument is skipped. A bad value, an unknown or ambiguous option, and an argument that does not begin with `-` or `/` are each reported with `$LOG()` at `STS$K_ERROR` (the records carry the facility `UTILS`) and skipped, and the rest of the arguments is still processed; the status of the call is an error at the end. An `OPTS$K_CONF` option stores its value in the `ASC` (if *ptr* is not null) and reads that file at once with `__util$readconfig()`; options that follow on the command line override those of the file.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Every argument was taken
`STS$K_ERROR`	A bad value, an unknown or ambiguous option, an argument that is not an option, or an error reading the configuration file. The arguments that were good have been taken all the same
`STS$K_INFO`	Only the configuration file named by `OPTS$K_CONF` had unknown options (skipped and logged). The low bit is set: `1 & status` is true

In 1.6.11 the routine returned `STS$K_SUCCESS` after a bad value or an unknown option, and `STS$K_ERROR` only when the configuration file could not be read.

Example

```
int main (int argc, char *argv[])
{
    if ( !(1 & __util$getparams(argc, argv, s_opts)) )
        return  STS$K_ERROR;

    __util$showparams(s_opts);
```

```
    /* the work of the program */
    return  STS$K_SUCCESS;
}
```

__util$readconfig – Read the Options from a File

Format

```
int __util$readconfig   (const char *, OPTS *);
```

Returns

`int`: a condition value.

Arguments

- *fconf* – `const char *`; read only; by reference. The path of the file.
- *opts* – `OPTS *`; read only; by reference. The table of options.

Description

Reads the file line by line. Everything from a `!` to the end of a line is a comment, unless the `!` is inside a part in double or single quotes; the line is trimmed. A line that is not empty must be `-name=value` or `/name=value`; any other is reported with `$LOG()` at `STS$K_ERROR` and skipped. A line longer than 254 octets is an error too, unless the rest of it is a comment; the tail of such a line is dropped and is not read as a line of its own. A name that is not in the table is reported at `STS$K_WARN` and skipped: the file may serve several programs. Every message is logged with the facility `UTILS`, and the rest of the file is still processed.

A value that begins and ends with the same quote, `"` or `'`, after the white space is trimmed, loses the two quotes (for every type); a lone or unbalanced quote stays in the value.

The name may be abbreviated, with the rule of `__util$getparams()`: a name spelled out in full wins, otherwise it must be the beginning of exactly one option name, ignoring case; an abbreviation that fits several is an error. The conversion of the values is the same as in `__util$getparams()`. An `OPTS$K_CONF` entry is refused inside a file, with a message: a configuration file does not name another.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Every line was taken
`STS$K_INFO`	An option was unknown: it was skipped and logged, nothing else was wrong. The low bit is set, so it is not a failure
`STS$K_ERROR`	No file name, a file that cannot be read, a bad value, a line that is not an option, an ambiguous abbreviation, `OPTS$K_CONF`, or a line that is too long

Example

```
if ( !(1 & __util$readconfig("/etc/myapp.conf", s_opts)) )
    $LOG(STS$K_WARN, "no configuration, defaults are used");
```

__util$showparams – Log the Options

Format

```
int __util$showparams   (const OPTS *opts);
```

Returns

`int`: `STS$K_SUCCESS`.

Arguments

- *opts* – `const OPTS *`; read only; by reference. The table.

Description

Writes one `$LOG()` record at `STS$K_INFO` for each option of the table up to `OPTS_NULL`: `name = value`. A port is shown in decimal and hexadecimal, an address in dotted form, a flag as `ON` or `OFF`, a string with its length, a password as `<password>`. An integer is shown by its size: 1, 2 and 8 octets as an unsigned byte, word and quadword (`!UB`, `!UW`, `!UQ`), 4 and 0 as a signed longword (`!SL`), each with the hexadecimal form. An `OPTS$K_CONF` entry is not shown. The records are written by the library, so their facility is `UTILS` whatever `__FAC__` the program has.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

2.9 Queues, Locks and Atomic Operations

A **queue** is a doubly-linked list that is interlocked by a spin lock in its header. An element of a queue is a structure that *begins* with an `ENTRY`; the queue routines treat the address of the structure as the address of its `ENTRY`.

ENTRY, __QUEUE, QUEUE_INITIALIZER – Queue Types

Format

```
typedef struct  __entry {
    struct  __entry *left, *right;
    void    *queue;
} ENTRY;

typedef struct  __queue {
    ENTRY       *head, *tail;
    int     lock;           /* SRWLOCK on Windows */
    unsigned    count;
} __QUEUE;

#define QUEUE_INITIALIZER { (ENTRY *) 0, (ENTRY *) 0, 0, 0 }
```

Description

left and *right* link an element to its neighbours; they are null at the ends. *queue* is the address of the queue that holds the element, or null when it is in none. The queue header holds the addresses of the first and the last element, the lock and the number of elements. A new queue is initialized with `QUEUE_INITIALIZER`. The structures are packed with 8-octet alignment.

Example

```
typedef struct { ENTRY links; unsigned short len; char buf[128]; } MSG;

static __QUEUE s_free = QUEUE_INITIALIZER;
static MSG     s_pool [32];

for ( int i = 0; i < 32; i++ )
    {
    unsigned l_count;

    $INSQTAIL(&s_free, &s_pool[i], &l_count);
    }
```

__util$lockspin – Take a Spin Lock

Format

```
inline  static int __util$lockspin (void volatile * lock);
```

Returns

`int`: a condition value.

Arguments

- *lock* – `void volatile *`; modify; by reference. The address of the lock, an `int` set to 0 when free. On Windows, the address of an `SRWLOCK`.

Description

Spins until the lock is free, then takes it (it sets the `int` to 1 with an atomic exchange). The routine gives up after about 4 billion tries. The lock is not recursive: a thread that takes it twice waits for itself until it gives up.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The lock is taken
`STS$K_ERROR`	The lock did not become free

__util$unlockspin – Release a Spin Lock

Format

```
inline  static  int __util$unlockspin (void * lock);
```

Returns

`int`: `STS$K_SUCCESS`.

Arguments

- *lock* – `void *`; modify; by reference. The lock taken by `__util$lockspin()`.

Description

Releases the lock. Do not release a lock you do not hold.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

$LOCK_LONG, $UNLOCK_LONG – Lock Macros

Format

```
#define $LOCK_LONG(lock)    __util$lockspin(lock)
#define $UNLOCK_LONG(lock)  __util$unlockspin(lock)
```

Description

Aliases of the two routines.

Example

```
static int s_lock;

if ( 1 & $LOCK_LONG(&s_lock) )
    {
    /* ... a short critical section ... */
    $UNLOCK_LONG(&s_lock);
    }
```

__util$insqtail – Insert an Entry at the Tail

Format

```
inline static int __util$insqtail (void * que, void * ent, unsigned * count);
```

Returns

`int`: a condition value.

Arguments

- *que* – `void *`; modify; by reference. The queue (`__QUEUE *`).
- *ent* – `void *`; modify; by reference. The element (an `ENTRY` at its start).
- *count* – `unsigned *`; write only; by reference. Receives the number of elements in the queue before the insertion.

Description

Links the element after the last. If the element is in this queue already, the call does nothing and returns `STS$K_SUCCESS`, and *count* is not set. If it is in another queue, it is refused. The `ENTRY` of a new element must be zeroed (its *queue* must be null) before the first insertion.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Inserted, or already in this queue
`UTIL$S_INVARG` (0x102)	A null argument
`UTIL$S_INQUE` (0x202)	The element is in another queue
`UTIL$S_NOLOCK` (0x302)	The lock was not taken

__util$insqhead – Insert an Entry at the Head

Format

```
inline static int __util$insqhead (void * que, void * ent, unsigned * count);
```

Returns

`int`: a condition value.

Arguments

As `__util$insqtail()`.

Description

Links the element before the first. The same rules as `__util$insqtail()`, and the same condition values (in 1.6.11 every failure of this routine was a plain `STS$K_ERROR`).

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Inserted, or already in this queue
`UTIL$S_INVARG` (0x102)	A null argument
`UTIL$S_INQUE` (0x202)	The element is in another queue
`UTIL$S_NOLOCK` (0x302)	The lock was not taken

__util$remqhead – Remove the Entry at the Head

Format

```
inline static int __util$remqhead (void * que, void **ent, unsigned * count);
```

Returns

`int`: a condition value.

Arguments

- *que* – `void *`; modify; by reference. The queue.
- *ent* – `void **`; write only; by reference. Receives the address of the element removed, or null if the queue was empty.
- *count* – `unsigned *`; write only; by reference. Receives the number of elements before the removal.

Description

Unlinks the first element and clears its links and its *queue*. **On an empty queue the routine returns `STS$K_SUCCESS`, sets *ent* to null and *count* to 0.** An empty queue is a normal outcome, not an error; the caller tells "got an element" from "nothing there" by *ent* (or *count*), not by the status. In 1.6.11 *ent* was left unset in this case, so a caller that read it got a stale pointer.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Removed, or the queue was empty (*ent* is null and *count* is 0)
`STS$K_ERROR`	A null argument, or the lock was not taken (*ent* is not touched)

Example

```
MSG     *l_msg = NULL;
unsigned l_count;

if ( (1 & $REMQHEAD(&s_free, &l_msg, &l_count)) && l_msg )
    l_msg->len = 0;
```

__util$remqtail – Remove the Entry at the Tail

Format

```
inline static int __util$remqtail (void * que, void **ent, unsigned * count);
```

Returns

`int`: a condition value.

Arguments

As `__util$remqhead()`.

Description

Unlinks the last element. The same rules as `__util$remqhead()`, including the empty queue.

Condition Values Returned

As `__util$remqhead()`.

__util$remqent – Remove a Given Entry

Format

```
inline static int __util$remqent (void * que, void *ent, unsigned * count);
```

Returns

`int`: a condition value.

Arguments

- *que* – `void *`; modify; by reference. The queue.
- *ent* – `void *`; modify; by reference. The element to remove; the element itself, not the address of a pointer.
- *count* – `unsigned *`; write only; by reference. Receives the number of elements before the removal.

Description

Unlinks the element from wherever it stands, and clears its links and *queue*. The element must be in this queue. An element that is not in it, an empty queue included, is an error, since there is nothing to remove; this differs from the removal at the head or the tail, where an empty queue is a normal outcome. (In 1.6.11 an empty queue gave `STS$K_SUCCESS` and did nothing.)

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Removed
`STS$K_ERROR`	A null argument, the element is not in this queue, or the lock was not taken

__util$movqhead – Move an Entry to the Head

Format

```
inline static int __util$movqhead (void * que, void * ent);
```

Returns

`int`: a condition value.

Arguments

- *que* – `void *`; modify; by reference. The queue.
- *ent* – `void *`; modify; by reference. The element itself (an `ENTRY` at its start), as `$MOVQHEAD()` passes it. In 1.6.11 the parameter was declared `void **` while the macro and the code treated it as the element; the prototype now says what it means. A program that called the routine directly with a cast needs no change in the call, and one that passed a `void **` to satisfy the old prototype must pass the element.

Description

Moves the element to the head of the queue. The element must be in this queue (its *queue* field is checked). If it is the head already, the call is a success and nothing changes. The routine unlinks the element and puts it in front of the old head; when the element was the tail, the tail of the queue is re-pointed to its left neighbour and the left link of the old head is set (both were left stale in 1.6.11). The cost is constant: there is no search.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Moved, or the element was the head already
`STS$K_ERROR`	A null argument, the element is not in this queue, or the lock was not taken

__util$movqtail – Move an Entry to the Tail

Format

```
inline static int __util$movqtail (void * que, void * ent);
```

Returns

`int`: a condition value.

Arguments

As `__util$movqhead()`.

Description

Moves the element to the tail of the queue; the mirror of `__util$movqhead()`. The element must be in this queue. If it is the tail already, the call is a success and nothing changes. When the element was the head, the head of the queue is re-pointed to its right neighbour. (In 1.6.11 this routine damaged the queue: it lost the elements after the one moved, did not update the head, and in a debug build stopped at an `assert()`; the advice then was not to use it. It is safe now.)

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Moved, or the element was the tail already
`STS$K_ERROR`	A null argument, the element is not in this queue, or the lock was not taken

__util$q_unlink – Take an Entry Out of the Chain of a Queue

Format

```
inline static void __util$q_unlink (__QUEUE * que, ENTRY * ent);
```

Returns

None.

Arguments

- *que* – `__QUEUE *`; modify; by reference. The queue.
- *ent* – `ENTRY *`; modify; by reference. The entry to unlink; it must stand in *que*.

Description

The helper of the queue routines, new in 1.7.0. It joins the neighbours of the entry to each other, re-points the head or the tail of the queue when the entry was the first or the last one, and clears the *left* and *right* links of the entry. The caller must hold the spin lock of the queue, and must keep the counter and the *queue* field of the entry up to date: the routine touches neither. A program uses the queue routines, not this one.

Condition Values Returned

None; the routine does not return a value.

__util$clrqueue – Empty a Queue

Format

```
inline static int __util$clrqueue (void * que,  unsigned * count);
```

Returns

`int`: a condition value.

Arguments

- *que* – `void *`; modify; by reference. The queue.
- *count* – `unsigned *`; write only; by reference. Receives the number of elements the queue held.

Description

Clears the *left*, *right* and *queue* fields of every element and empties the queue header. The elements are not freed. *count* receives the number of elements the queue held, and **0 if it was empty** (in 1.6.11 it was left unset then, and the links of the elements were not cleared).

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Done (also for an empty queue)
`STS$K_ERROR`	A null argument, or the lock was not taken

$INSQHEAD, $INSQTAIL, $REMQHEAD, $REMQTAIL, $REMQENT, $MOVQHEAD, $MOVQTAIL, $CLRQUE – Queue Macros

Format

```
#define $INSQHEAD(que, ent, count)  __util$insqhead ((__QUEUE *) que, (void *) ent,
 ↪  (unsigned *) count)
#define $INSQTAIL(que, ent, count)  __util$insqtail ((__QUEUE *) que, (void *) ent,
 ↪  (unsigned *) count)
#define $REMQHEAD(que, ent, count)  __util$remqhead ((__QUEUE *) que, (void **) ent,
 ↪  (unsigned *) count)
#define $REMQTAIL(que, ent, count)  __util$remqtail ((__QUEUE *) que, (void **)ent,
 ↪  (unsigned *) count)
#define $MOVQHEAD(que, ent)     __util$movqhead ((__QUEUE *) que, (void *) ent)
#define $MOVQTAIL(que, ent)     __util$movqtail ((__QUEUE *) que, (void *) ent)
#define $REMQENT(que, ent, count)   __util$remqent ((__QUEUE *) que, (void **)ent,
 ↪  (unsigned *) count)
#define $CLRQUE(que, count)     __util$clrqueue ((__QUEUE *) que, (unsigned *) count)
```

Returns

`int`: what the routine returns.

Arguments

- *que* – the address of a `__QUEUE`.
- *ent* – for `$INSQHEAD()`, `$INSQTAIL()`, `$MOVQHEAD()`, `$MOVQTAIL()` and `$REMQENT()`, the address of the element. For `$REMQHEAD()` and `$REMQTAIL()`, the address of a pointer variable that receives the element.
- *count* – the address of an `unsigned`.

Description

The macros cast their arguments, so any pointer to a structure that begins with an `ENTRY` is accepted. They do not evaluate an argument more than once. The conditions and rules are those of the routines. `$MOVQHEAD()` and `$MOVQTAIL()` pass the element as `void *`, which is exactly what the prototypes of 1.7.0 take.

Example

```
MSG     *l_msg;
unsigned l_count;

$INSQTAIL(&s_free, &s_pool[0], &l_count);
$REMQHEAD(&s_free, &l_msg, &l_count);
$REMQENT(&s_free, &s_pool[1], &l_count);
$CLRQUE(&s_free, &l_count);
```

$CMP_STORE_LONG – Compare and Swap

Format

```
#define $CMP_STORE_LONG(src, old_val, new_val)  __sync_bool_compare_and_swap((int *) src,
↪  (int) old_val, (int) new_val )
```

Returns

Non-zero if the value was equal to *old_val* and *new_val* was stored; 0 if not.

Arguments

- *src* – address of an `int`; modify; by reference. The variable.
- *old_val* – `int`; read only; by value. The value it must have.
- *new_val* – `int`; read only; by value. The value to store.

Description

An atomic compare and swap of one longword (`__sync_bool_compare_and_swap`).

Example

```
static int s_state;

if ( $CMP_STORE_LONG(&s_state, 0, 1) )
    ;   /* this thread moved the state from 0 to 1 */
```

$ATOMIC_INC, $ATOMIC_DEC – Atomic Increment and Decrement

Format

```
#define $ATOMIC_INC(val)    __sync_fetch_and_add (&val, 1)
#define $ATOMIC_DEC(val)    __sync_fetch_and_sub (&val, 1)
```

Returns

The value the variable had **before** the change.

Arguments

- *val* – an integer variable (an lvalue, not a pointer); modify; the macro takes its address.

Description

Adds or subtracts 1 atomically with full memory barrier (GCC `__sync` built-ins).

Example

```
static unsigned s_users;

$ATOMIC_INC(s_users);
```

2.10 Ranges, Minimum and Maximum

__util$min, __util$max, $MIN, $MAX – Smaller and Larger of Two Integers

Format

```
inline static int __util$min (int x, int y);
inline static int __util$max (int x, int y);

#define $MIN(x, y)  __util$min((int) (x), (int) (y))
#define $MAX(x, y)  __util$max((int) (x), (int) (y))
```

Returns

`int`: the smaller or the larger.

Arguments

- *x*, *y* – `int`; read only; by value.

Description

The macros cast both arguments to `int` and evaluate each once. A value that does not fit an `int` (a `size_t` above 2^31 or a 64-bit number) is truncated; use `$MIN()` and `$MAX()` for sizes that are known to be small.

Example

```
int l_n = $MIN(l_avail, (int) sizeof(l_buf));
```

__util$isinrange, $ISINRANGE – Test a Value Against a Closed Range

Format

```
inline static int __util$isinrange (int x, int __min__, int __max__ );

#define $ISINRANGE(x, __min__, __max__) __util$isinrange(x, (__min__), (__max__))
```

Returns

`int`: 1 if ***min*** <= *x* <= ***max***, else 0.

Arguments

- *x* – `int`; read only; by value. The value.
- ***min***, ***max*** – `int`; read only; by value. The borders; given in either order.

Description

The borders are in the range.

Example

```
if ( !$ISINRANGE(l_port, 1, 65535) )
    return  STS$K_ERROR;
```

__util$isinrange2, $ISINRANGE2 – Test a Value Against a Closed Range (Second Form)

Format

```
inline static int __util$isinrange2 (int x, int __min__, int __max__ );

#define $ISINRANGE2(x, __min__, __max__)    __util$isinrange2(x, (__min__), (__max__))
```

Returns

`int`: 1 if ***min*** <= *x* <= ***max***, else 0.

Arguments

As `__util$isinrange()`.

Description

The borders are in the range, as in `__util$isinrange()`, and may be given in either order. In 1.6.11 this routine excluded the borders, against the comment of the header, which said that they were included; the code now does what the comment said. A program that relied on the strict test must write it by hand: `(x > lo) && (x < hi)`.

__util$range, $RANGE – Bound a Value to a Range

Format

```
inline static int __util$range (int x, int  __min__, int __max__ );

#define $RANGE(x, __min__, __max__) __util$range(x, __min__, __max__)
```

Returns

`int`: *x* if it is in the range; else the nearest border.

Arguments

As `__util$isinrange()`.

Example

```
l_timeout = $RANGE(l_timeout, 1, 3600);
```

2.11 Item Lists

ILE2, ILE3 – Item List Element Types

Format

```
typedef struct  __ile2  {
    unsigned short  code, len;
    char *  ptr;
} ILE2;

typedef struct  __ile3  {
    unsigned short  code, len;
    char *  ptr;
    unsigned short * retlen;
} ILE3;
```

Description

An item list is an array of elements that ends with an element of zeros. *code* says what the element is, *len* the length of the buffer at *ptr*. In an `ILE3`, *retlen* is the address of a word that receives the length actually returned.

The layout is not that of OpenVMS. The `ILE3` of OpenVMS begins with the length word and has the code second; this `ILE3` has the **code first** and the length second. Both words are 16 bits, so the size is the same, but a list built here must not be handed to an OpenVMS system service as it stands (or to code written for that layout). The same holds for `ILE2`. The types serve the programs that define item lists; the routines of this library that take them are described with those routines.

$ILE2INI, $ILE3INI, $ILENULL – Item List Macros

Format

```
#define $ILE2INI(code, len, ptr)        {code, len, (char *)ptr}
#define $ILE3INI(a_ile, a_code, a_len, a_ptr, a_retlen) do {            \
        (a_ile).code = (unsigned short) (a_code);           \
        (a_ile).len = (unsigned short) (a_len);         \
        (a_ile).ptr = (char *) (a_ptr);             \
        (a_ile).retlen = (a_retlen);                    \
    } while (0)
#define $ILENULL    {0}
```

Arguments

- *code* – `unsigned short`; the item code.
- *len* – `unsigned short`; the length of the buffer.
- *ptr* – address; the buffer.
- *a_ile* – `$ILE3INI()` only: an `ILE3` object, a variable or an element of an array. Not a pointer to it.
- *a_code*, *a_len*, *a_ptr*, *a_retlen* – `$ILE3INI()` only: the code, the length, the buffer and the `unsigned short *` for the returned length.

Description

`$ILE2INI()` is an initializer of an `ILE2`. `$ILENULL` is the initializer of the terminating element. `$ILE3INI()` is a statement that fills an `ILE3` object: it is written `$ILE3INI(l_item, 1, sizeof(l_name), l_name, &l_ret);` with a semicolon, and can stand where a statement stands, such as the unbraced branch of an `if`. In 1.6.11 it did not compile with ordinary arguments, because its parameters were named as the members it sets; the parameters are now `a_ile`, `a_code`, `a_len`, `a_ptr` and `a_retlen`, and each is converted to the type of the member.

Example

```
char l_name [32];

ILE2 l_list [] = {
    $ILE2INI(1, sizeof(l_name), l_name),
    $ILENULL
};

ILE3 l_item;
unsigned short l_ret;

$ILE3INI(l_item, 1, sizeof(l_name), l_name, &l_ret);
```

2.12 Time Helpers

The time routines of Chapter 3 work on `fao_time_t` (seconds). These work on `struct timespec` and `struct tm`. The routine `s___time()` (the clock used by the logging routines, `CLOCK_REALTIME_COARSE` on Linux) is `static inline` in the header and is not part of the interface.

__util$timbuf – Break Down the Local Time

Format

```
inline  static void __util$timbuf   (time_t *__src, struct tm * __dst);
```

Returns

None.

Arguments

- *__src* – `time_t *`; read only; by reference. The time, or null for the present moment.
- *__dst* – `struct tm *`; write only; by reference. The result.

Description

Converts to the local time with `localtime_r()` (`localtime_s()` on Windows) and then changes two fields to the calendar values: *tm_mon* is 1 to 12 and *tm_year* the full year, 2026. This differs from the usual `struct tm`; do not pass the result to `mktime()` or `strftime()`.

Example

```
struct tm l_tm;

__util$timbuf(NULL, &l_tm);
$LOG(STS$K_INFO, "%04d-%02d-%02d", l_tm.tm_year, l_tm.tm_mon, l_tm.tm_mday);
```

__util$add_time – Add Two Times

Format

```
inline static void __util$add_time(const struct timespec* time1, const struct timespec*
↪  time2, struct timespec* result_time);
```

Returns

None.

Arguments

- *time1*, *time2* – `const struct timespec *`; read only; by reference. The times, with *tv_nsec* from 0 to 999999999.
- *result_time* – `struct timespec *`; write only; by reference. The sum; it may be one of the operands.

Description

Adds the seconds and the nanoseconds and carries the nanoseconds into the seconds. There is no check for an overflow of the seconds.

__util$sub_time – Subtract Two Times

Format

```
inline static void __util$sub_time(const struct timespec * time1, const struct timespec*
↪  time2, struct timespec* result_time);
```

Returns

None.

Arguments

- *time1*, *time2* – `const struct timespec *`; read only; by reference. The minuend and the subtrahend.
- *result_time* – `struct timespec *`; write only; by reference. *time1* minus *time2*.

Description

Subtracts the seconds and the nanoseconds, field by field, and normalises the result: *tv_nsec* is from 0 to 999999999, and when the nanoseconds of *time1* are fewer than those of *time2* a second is borrowed. A negative difference is *tv_sec* below zero with a non-negative *tv_nsec*; the value is *tv_sec* plus *tv_nsec*/10^9. In 1.6.11 *tv_nsec* could be negative.

__util$cmp_time – Compare Two Times

Format

```
inline static int __util$cmp_time(const struct timespec * time1, const struct timespec*
 ↪  time2);
```

Returns

`int`: 0 if the times are equal; -1 if *time1* is earlier than *time2*; 1 if it is later. Not a condition value.

Arguments

- *time1*, *time2* – `const struct timespec *`; read only; by reference. The times.

Description

Compares the seconds, and if they are equal the nanoseconds. Nothing is subtracted, so the result cannot overflow. Only the sign is defined by the interface, but the value is always -1, 0 or 1. In 1.6.11 the result was the difference of the seconds converted to `int`, which could have the wrong sign for times more than about 68 years apart.

Example

```
if ( __util$cmp_time(&l_now, &l_deadline) >= 0 )
    $LOG(STS$K_WARN, "the deadline has passed");
```

2.13 The No-Operation Routine

__util$nope – Do Nothing

Format

```
static inline void __util$nope (void) {}
```

Returns

None.

Description

An empty routine. In a build with `NDEBUG` the macros `$TRACE()` and `$IFTRACE()` expand to a call of it, so that the macro is still a statement. It is defined only when `NDEBUG` is defined, and only once (guarded by `__UTIL$_NOPE__`).

Miscellaneous Definitions

Format

```
#define likely(x)   __builtin_expect((x),1)
#define unlikely(x) __builtin_expect((x),0)
#define __PATH_SEPARATOR    '/'     /* '\\' on Windows */
#define __PATH_SEPARATOR_S  "/"     /* "\\" on Windows */
```

Description

`likely()` and `unlikely()` are hints for the compiler about a branch; on Windows they expand to their argument. They are defined only if the program has not defined them. `__PATH_SEPARATOR` and `__PATH_SEPARATOR_S` are the separator of the parts of a file path, as a character and as a string; they too may be defined by the program before the header is included.

Chapter 3. FAO and Time Services

This chapter describes the formatter and the time routines of the STARLET module (`starlet.h`, `starlet.c`), together with the time helpers that `utility_routines.h` defines. The module needs no C run-time routine except `time()`. It does not use `<stdio.h>`, `<string.h>`, `<ctype.h>`, `localtime()` or `gmtime()`.

Routine	Purpose
`__util$fao`	Format a control string with a variable list of parameters
`__util$faol`	Format a control string with an array of parameters
`$FAO_DESC`	Initialize an output descriptor over a character array
`__util$gettim`	Return the current binary time
`__util$bintim`	Convert an ASCII time string to binary time
`__util$asctim`	Convert a binary time to an ASCII string
`__util$add_times`	Add two binary times
`__util$sub_times`	Subtract two binary times
`__util$add_time`	Add two `struct timespec` values
`__util$sub_time`	Subtract two `struct timespec` values
`__util$cmp_time`	Compare two `struct timespec` values
`__util$timbuf`	Break a `time_t` into a `struct tm` with natural year and month

3.1 Conventions of the FAO Routines

The output descriptor. The formatter writes into a `fao_desc_t`. Only two fields are used:

```
typedef struct __fao_desc
{
    unsigned short dsc$w_length;   /* capacity of the buffer, bytes     */
    unsigned char  dsc$b_dtype;    /* not interpreted                   */
    unsigned char  dsc$b_class;    /* not interpreted                   */
    char          *dsc$a_pointer;  /* address of the buffer             */
} fao_desc_t;
```

`dsc$w_length` is the capacity of the buffer. The routines never change it. The length of the result is returned through the separate `outlen` argument. The result is not zero-terminated. The capacity cannot exceed 65535 bytes.

The parameters. Every parameter is an `fao_prm_t`, an `uint64_t` passed BY VALUE. This holds for all sizes and kinds of directive: a byte, a word, a longword, a quadword, a time value and a pointer. A pointer is passed as `(fao_prm_t)(uintptr_t) ptr`. In a call of `__util$fao` the caller must cast every parameter to `fao_prm_t`. An un-cast `int` is promoted to `int`, not to a 64-bit value, and the formatter reads the wrong width. The macros `$LOG_FAO` and `$PUTMSG_FAO` of `utility_routines.h` apply the cast and count the parameters for you (see Chapter 2).

The completion status. The status is “sticky”: it is set when an error happens and is returned at the end. The first error wins; a later error does not replace it. The values are condition values in the OpenVMS layout (severity in bits 0 to 2, message number above it, no facility), all even, and they are stable from Version 1.7.0 on. They are in Appendix A.

Name	Value	Meaning
`STS$K_SUCCESS`	1	Conversion complete
`FAO$K_BUFOVF`	0xA	The result was truncated to the capacity of the buffer
`FAO$K_BADDIR`	0x12	A directive is unknown, malformed or truncated by the end of the control string
`FAO$K_IVTIME`	0xC	Invalid time string, or an invalid pair of operands (time routines)
`FAO$K_NEGTIM`	8	The result of a time subtraction would be negative
`FAO$K_NOPRM`	0x1A	The control string wants more parameters than there are
`FAO$K_FLDOVF`	0x22	The text of a `!n<...!>` field was longer than `FAO$K_FLDBUF` and was cut
`STS$K_ERROR`	2	A required argument is NULL

All these values except `STS$K_SUCCESS` are even, so the test `1 & status` is false for them. Only `FAO$K_BADDIR` changed its number in Version 1.7.0 (it was 6); `FAO$K_NOPRM` and `FAO$K_FLDOVF` are new. A program should compare with the names, not with the numbers.

Limits.

Name	Value	Meaning
`FAO$K_MAXPRM`	64	Parameters cached by `__util$fao`; the parameter with index 64 or higher reads as 0
`FAO$K_MAXDEPTH`	4	Nesting depth of `!n(...!)` and `!n<...!>` groups
`FAO$K_FLDBUF`	512	The longest text of a `!n<...!>` field (`starlet.h`)

Compile-time options of `starlet.c`.

Macro	Default	Effect
`FAO$T_TIME`	`time_t`	Type behind `fao_time_t`; define it before `starlet.h` in a freestanding build
`FAO$GETTIME()`	`time(NULL)`	Source of the current time
`FAO$K_TZOFFSET`	0	Seconds added to UTC by `!%T`, `!%D`, `__util$asctim` and subtracted by `__util$bintim`

The time zone is fixed when `starlet.c` is compiled. It cannot be changed at run time.

3.2 The FAO Directives

A directive starts with `!`. Its general form is

```
!  [ n | # ]  directive
```

`n` is a decimal number. `#` takes the number from the parameter list: it consumes one parameter (read as a 32-bit unsigned value) before the parameters of the directive itself. The number is a field width, a repeat count or a skip count, as the directive defines. A directive that does not use the number ignores it.

In the table, “Params” is the number of parameters that the directive consumes. “n” is the optional number.

Directive	Params	Output	Effect of n
`!AS`	1	String described by an `fao_desc_t *`; length from `dsc$w_length`	Field width
`!AZ`	1	Zero-terminated string (`char *`)	Field width
`!AC`	1	Counted string (`char *`); the first byte is the length	Field width
`!AD`	2	String; parameters are the length, then the address	Field width
`!AF`	2	As `!AD`, but a byte outside 0x20..0x7E is shown as .	Field width
`!UB !UW !UL !UQ`	1	Unsigned decimal of 8, 16, 32, 64 bits	Field width
`!SB !SW !SL !SQ`	1	Signed decimal of 8, 16, 32, 64 bits	Field width
`!XB !XW !XL !XQ`	1	Hexadecimal, upper case, zero-filled to 2, 4, 8, 16 digits	Field width
`!OB !OW !OL !OQ`	1	Octal, zero-filled to 3, 6, 11, 22 digits	Field width
`!ZB !ZW !ZL !ZQ`	1	Unsigned decimal, zero-filled to n digits	Field width
`!n*c`	0	The character `c` (the next byte of the control string) n times	Repeat count, default 1
`!n( … !)`	0	The enclosed directives, repeated	Repeat count, default 1
`!n< … !>`	0	The enclosed text as a field	Field width
`!n-`	0	Nothing; moves the parameter pointer back n places	Count, default 1
`!n+`	0	Nothing; skips the next n parameters	Count, default 1
`!/`	0	Carriage return and line feed (two bytes, `\r\n`)	Ignored
`!_`	0	Horizontal tab	Ignored
`!^`	0	Form feed	Ignored

Directive	Params	Output	Effect of n
`!!`	0	The character `!`	Ignored
`!%S`	0	`s` if the last converted number is not 1, else nothing	Ignored
`!%T`	1	Time of day `hh:mm:ss.00` of a binary time	Ignored
`!%D`	1	Date and time `dd-MMM-yyyy hh:mm:ss.00` of a binary time	Ignored
`!%M`	1	MAC address `aa:bb:cc:dd:ee:ff` from a pointer to 6 octets	Ignored
`!%I`	1	IPv4 address `d.d.d.d` from a pointer to 4 octets	Ignored
`!%6`	1	IPv6 address in the form of RFC 5952, from a pointer to 16 octets	Ignored
`!%U`	1	UUID `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` from a pointer to 16 octets	Ignored

A directive that is not in the table sets `FAO$K_BADDIR`. The formatter consumes the directive characters, writes nothing for them and goes on. It does not consume a parameter. The same holds for a directive that the control string cuts short: `"abc!"`, `"abc!5"`, `"a!A"`, `"a!%"` and `"a!3*"` all give the text `abc` or `a` and `FAO$K_BADDIR`; the scanner stops at the terminating zero and never reads past it.

3.2.1 String Directives

`!AS`, `!AZ`, `!AC`, `!AD` and `!AF` copy bytes. A NULL descriptor or a NULL pointer writes nothing and still consumes its parameter (`!AD` and `!AF` consume the length first, then the address). With a field width n the string is left-justified: it is cut on the right if it is longer than n and is padded on the right with blanks if it is shorter. `!0AS` writes nothing.

`!AS` takes the length from `dsc$w_length` of the descriptor. `!AC` reads the length byte as an unsigned value, so the string can have up to 255 bytes. `!AD` and `!AF` read the length as an unsigned 32-bit value.

`!AF` replaces every byte outside the range 0x20..0x7E with a period. Width is applied after the replacement.

```
fao_desc_t d = { 4, 0, 0, "abcdef" };
/* !AS|!8AS|!2AS  with &d three times   ->  abcd|abcd    |ab       */
/* !AZ|!6AZ       with "hello" twice    ->  hello|hello              */
/* !AC            with "\003xyz"        ->  xyz                      */
/* !AD|!AF        with 3,"abcd",4,"a\001b\n"  ->  abc|a.b.          */
```

3.2.2 Numeric Directives

The second letter of the directive sets the radix and the third the size. The size letter is `B` (8 bits), `W` (16), `L` (32) or `Q` (64). The parameter is a 64-bit value. The formatter first cuts it to the size: the high bits are dropped. For `S` the result is sign-extended; for the other letters it is zero-extended. Thus `!SB` with the parameter 200 prints `-56`, and `!XB` with 0x1FF prints `FF`. Any other third letter sets `FAO$K_BADDIR` and the directive is skipped; the two letters after `!` are consumed, but no parameter is (`"x!ZZ y"` gives `x y`).

Without a width, the layout is as follows.

Directive	Layout
`!U`	Natural number of digits
`!S`	Natural number of digits, with a leading `-` if the value is negative
`!Z`	Natural number of digits, no zero fill (the same as `!U`)
`!X`	Upper-case digits, zero-filled to 2, 4, 8 or 16 digits
`!O`	Digits zero-filled to 3, 6, 11 or 22 digits

With a width n, the result occupies exactly n columns:

Directive	n larger than the digits	n smaller than the digits
`!U !S`	Blanks on the left	n asterisks
`!Z`	Zeros on the left, up to n digits	n asterisks
`!X !O`	Blanks on the left of the zero-filled number	The left digits are cut off; the low n digits remain

Note the difference between `!Z` and `!X`. `!5ZL` of 42 is `00042`. `!10XL` of 0xAB is `000000AB`: the number is first zero-filled to 8 digits and the extra two columns are blanks. The sign of `!S` counts as one column.

`!X` and `!O` treat the value as a bit pattern, so they never print a sign.

Every numeric directive stores the value it converted (after the size cut and the sign extension) for the directive `!%S`.

```
/* !UL|!8UL|!2UL         with 123,123,123       ->  123|     123|**          */
/* !SL|!SB|!6SL          with -5,200,-5         ->  -5|-56|    -5            */
/* !XL|!XB|!4XL|!10XL    with 0xBEEF,0xA,0x12345678,0xAB
                                                 ->  0000BEEF|0A|5678|  000000AB */
/* !OB|!OL               with 8,8               ->  010|00000000010           */
/* !ZL|!5ZL|!3ZB         with 42,42,7           ->  42|00042|007              */
```

3.2.3 Width from the Parameter List: `!#`

`!#UL` takes the width from the parameter list: the first parameter is the width, the second the value. `__util$faol("!#UL", ..., {5, 77}, 2)` gives `77`. The same works for the repeat count (`!#(`), the field width of `!#<`, the repeat count of `!#*`, and the counts of `!#-` and `!#+`.

3.2.4 Repeat and Fixed-Width Groups

`!n(` text `!)` repeats the text n times (default 1; with 0 the group is skipped). Each pass consumes the next parameters. To use the same parameters again in a pass, place `!n-` in the group. The end of the group is the sequence `!)`. A group may hold other groups, up

to FAO$K_MAXDEPTH (4) levels. A group deeper than that, or a group with no terminator, sets FAO$K_BADDIR; the formatter then goes on with the byte after the opener, so the rest of the text is read as ordinary text and directives.

```
/* !3(!UL,!)   with 1,2,3   ->  1,2,3,                                  */
/* !2(!UL!)    nested inside !3<...!>:  !3<!2(!UL!)!>  with 1,2  ->  "12 " */
```

!n< text !> formats the text in a scratch buffer of 512 bytes and then writes it as a field of n columns: left-justified, padded on the right with blanks, cut on the right. Without n the field has its natural width. The parameters are shared with the outer string. If the text of the field is longer than FAO$K_FLDBUF (512 bytes), it is cut there and FAO$K_FLDOVF is set; the field is not a cut of the output buffer, so this is not FAO$K_BUFOVF. Any other status of the inside (FAO$K_BADDIR, for one) is passed on to the caller.

```
/* !5<ab!>|!5<abcdefg!>|!<x!>   ->  ab   |abcde|x                       */
```

3.2.5 Repeat a Character: !n*c

!n*c writes the byte c n times. c is the byte that follows the * in the control string; it can be any byte. With no n it writes one. !3*- writes ---. If the control string ends right after the *, nothing is written and the status is FAO$K_BADDIR.

3.2.6 Parameter Pointer: !n- and !n+

The formatter keeps an index of the next parameter. !n- lowers the index by n (default 1), but not below 0, so the same parameters are used again. !n+ raises the index by n (default 1), so that n parameters are skipped. In __util$faol the index counts into the array; a parameter past the end of the array reads as 0 and sets FAO$K_NOPRM. In __util$fao the first 64 parameters are cached so that !n- can go back to them; a parameter with the index 64 or higher reads as 0 and also sets FAO$K_NOPRM. __util$fao reads the parameters it needs from the argument list and has no count to check against, so the caller must pass at least as many as the control string consumes (a missing one is read from the stack and its value is undefined). Use __util$faol, or the counting macros $LOG_FAO and $PUTMSG_FAO, when the control string is not a literal of yours.

```
/* !UL !-!UL     with 7     ->  7 7        */
```

3.2.7 Plural: !%S

!%S writes s if the last number converted by !U, !S, !Z, !X or !O is not 1; it writes nothing if that number is 1. If no number was converted before, the value is 0 and the directive writes s. It consumes no parameter. The comparison is made on the 64-bit value after the size cut: !UB with 257 leaves the value 1.

```
/* "!UL file!%S"  with 1  ->  1 file          with 2  ->  2 files       */
```

3.2.8 Time: !%T and !%D

The parameter is a binary time (see Section 3.3), passed by value. Its sign decides what is shown:

- A positive value is an absolute time, the number of seconds since 01-JAN-1970 00:00:00 UTC. FAO$K_TZOFFSET seconds are added.
- The value 0 means “no time given”: the current time is shown. The epoch itself cannot be shown; one second after it can.
- A negative value is a delta time of that many seconds. !%D shows it as days, a blank and hh:mm:ss.00 (1 01:01:01.00 for -90061); !%T shows hh:mm:ss.00 and drops the days.

!%T writes hh:mm:ss.00, 11 bytes. !%D writes dd-MMM-yyyy hh:mm:ss.00, 23 bytes. The day is padded on the left with a blank when it is less than 10 (6-JAN-1970). The month is the English abbreviation in upper case. The year has four digits up to 9999; a later year is

written in full (`31-DEC-18029`), and the text is then longer than 23 bytes. The hundredths are always `00`, because the binary time has a resolution of one second. A width is ignored. An absolute time before the epoch cannot be expressed, since a negative value is a delta.

```
/* !%T with 86399        ->  23:59:59.00                              */
/* !%D with 1700000000   ->  14-NOV-2023 22:13:20.00                  */
/* !%D with 86400*5      ->   6-JAN-1970 00:00:00.00                  */
/* !%D with -90061       ->  1 01:01:01.00      (a delta of 1 d 1 h 1 m 1 s)  */
```

3.2.9 Address Directives: `!%M, !%I, !%6, !%U`

These four directives take a pointer to the raw octets, in network order. The routine writes text with no regard to a width. A NULL pointer writes `(null)` and the parameter is consumed. The text is built in a local buffer and then written as a unit, so a result that does not fit in the output buffer is cut on the right and `FAO$K_BUFOVF` is set.

Directive	Reads	Form	Example
`!%M`	6 octets	Two lower-case hex digits per octet, separated by `:`	`00:1b:21:3c:4d:5e`
`!%I`	4 octets	Decimal, no leading zeros, separated by `.`	`192.168.1.10`
`!%6`	16 octets	RFC 5952 canonical form (below)	`2001:db8::1`
`!%U`	16 octets	8-4-4-4-12 groups, lower-case hex	`550e8400-e29b-41d4-a716-446655440000`

The rules of `!%6` follow RFC 5952:

- Each of the eight 16-bit groups is written in lower case, with no leading zeros. A zero group is `0`.
- The longest run of two or more zero groups is replaced by `::`. If two runs have the same length, the left one is replaced. A single zero group is not replaced.
- An IPv4-mapped address (ten zero octets, then `ff ff`, then four octets) is written as `::ffff:` followed by the dotted IPv4 address, for example `::ffff:192.168.1.10`. The output is the same as that of `inet_ntop(AF_INET6, ...)`. An address in the IPv4-compatible form (`::a.b.c.d`) is written in hexadecimal groups, for example `::c0a8:10a`.
- The loopback address is `::1`; the unspecified address is `::`.

3.2.10 Literal Characters

`!!` writes `!`. `!/` writes the two bytes CR and LF. `!_` writes a tab. `!^` writes a form feed. Any text outside directives is copied as it is.

3.2.11 Overflow of the Output Buffer

When the result does not fit in the buffer, the formatter writes as many bytes as fit, sets `FAO$K_BUFOVF`, and goes on to the end of the control string, so the other statuses are still found. The buffer is never written past its capacity. If an error other than the overflow has happened earlier in the string, that earlier status stays: the first error wins.

__util$fao – Formatted ASCII Output

The Formatted ASCII Output routine formats a variable list of parameters under the control of an FAO control string.

Format

```
int __util$fao (const char *a_ctlstr, unsigned short *a_outlen,
                fao_desc_t *a_outbuf, ...);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_ctlstr`	`const char *`	read only	by reference	Zero-terminated control string
`a_outlen`	`unsigned short *`	write only	by reference	Receives the number of bytes written. May be NULL
`a_outbuf`	`fao_desc_t *`	read only (descriptor), write only (buffer)	by reference	Descriptor of the output buffer; `dsc$w_length` is the capacity, `dsc$a_pointer` must not be NULL
`...`	`fao_prm_t`	read only	by value	The parameters, each cast to `fao_prm_t`

Description

`__util$fao` scans the control string, converts the directives of Section 3.2, and writes the result into the buffer. The result is not zero-terminated. The descriptor is not changed; the length is returned in `a_outlen`.

The parameters are read from the argument list when a directive needs them, and the first 64 are kept in a cache. Parameters not consumed by the control string are ignored. A control string that consumes more parameters than the caller passed reads values that do not exist: the result is undefined, because a variable argument list has no end marker that the routine could see. Only the cache limit is checked: a parameter beyond the 64th reads as 0 and sets `FAO$K_NOPRM`. Use `__util$faol` when the number of parameters is known only at run time.

If `a_ctlstr`, `a_outbuf` or `a_outbuf->dsc$a_pointer` is NULL, the routine returns `STS$K_ERROR` and writes nothing, not even `*a_outlen`.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The string was formatted
`FAO$K_BUFOVF`	The buffer is too small; the result is truncated
`FAO$K_BADDIR`	A directive is unknown, malformed or truncated

Value	Meaning
FAO$K_FLDOVF	The text of a !n<...!> field was cut at 512 bytes
FAO$K_NOPRM	More than 64 parameters were wanted
STS$K_ERROR	A required argument is NULL

When several of these happen in one call, the status returned is the first one found. The text is produced anyway.

Example

```
#include "starlet.h"

char           buf [128];
fao_desc_t     dsc = $FAO_DESC (buf);
unsigned short len;
unsigned char  ip [4] = { 192, 168, 1, 10 };
int            sts;

sts = __util$fao ("peer !%I got !UL file!%S, mask !XL", &len, &dsc,
                  (fao_prm_t)(uintptr_t) ip, (fao_prm_t) 2, (fao_prm_t) 0xFF00);
if ( !(1 & sts) )
    return sts;

buf [len] = '\0';   /* the result is not zero-terminated */
/* buf: "peer 192.168.1.10 got 2 files, mask 0000FF00" */
```

__util$faol – Formatted ASCII Output with List

The Formatted ASCII Output with List routine formats an array of parameters under the control of an FAO control string.

Format

```
int __util$faol (const char *a_ctlstr, unsigned short *a_outlen,
                 fao_desc_t *a_outbuf,
                 const fao_prm_t *a_prmlst, int a_prmcnt);
```

Returns

`int`, write only, by value. A condition value, as for `__util$fao`.

Arguments

Argument	Type	Access	Mechanism	Description
`a_ctlstr`	`const char *`	read only	by reference	Zero-terminated control string
`a_outlen`	`unsigned short *`	write only	by reference	Receives the number of bytes written. May be NULL
`a_outbuf`	`fao_desc_t *`	read only (descriptor), write only (buffer)	by reference	Descriptor of the output buffer
`a_prmlst`	`const fao_prm_t *`	read only	by reference	Array of parameters. May be NULL; then every parameter reads as 0
`a_prmcnt`	`int`	read only	by value	Number of elements in `a_prmlst`

Description

`__util$faol` does what `__util$fao` does, but the parameters come from an array. A directive that asks for a parameter past the end of the array receives 0, and the status is `FAO$K_NOPRM` (`"!UL !UL !UL|"` with two parameters gives `1 2 0|`). The count is explicit; in OpenVMS the list is not counted. Because the array is read without a cache, there is no limit of 64 parameters. Parameter reuse with `!n-` and skipping with `!n+` work on the array index.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The string was formatted
`FAO$K_BUFOVF`	The buffer is too small; the result is truncated
`FAO$K_BADDIR`	A directive is unknown, malformed or truncated
`FAO$K_FLDOVF`	The text of a `!n<...!>` field was cut at 512 bytes
`FAO$K_NOPRM`	The control string wanted more than `a_prmcnt` parameters

Value	Meaning
STS$K_ERROR	A required argument is NULL

Example

```
char           buf [64];
fao_desc_t     dsc = $FAO_DESC (buf);
unsigned short len;
fao_prm_t      prm [2] = { 5, 77 };

__util$faol ("[!#UL]", &len, &dsc, prm, 2);    /* buf: "[   77]", len 7 */
```

$FAO_DESC – Initialize an Output Descriptor

Format

```
#define $FAO_DESC(buf)   { (unsigned short) sizeof(buf), 0, 0, (buf) }
```

Returns

A brace initializer for an `fao_desc_t`. It is not an expression, so it can be used only in an initialization.

Arguments

Argument	Type	Access	Mechanism	Description
`buf`	`char[N]`	write only	by reference	A character array, not a pointer: `sizeof(buf)` is the capacity

Description

With a pointer instead of an array, `sizeof` gives the size of the pointer and the capacity is wrong. To assign to an existing descriptor, use a compound literal: `dsc = (fao_desc_t) $FAO_DESC (buf);`. There is no `$FAO` macro in `starlet.h`; the routine is called directly, or through `$LOG_FAO` and `$PUTMSG_FAO` of `utility_routines.h`.

Condition Values Returned

None.

Example

```
char       buf [256];
fao_desc_t dsc = $FAO_DESC (buf);       /* capacity 256 */
```

3.3 The Binary Time Format

The time routines of this module do NOT use the OpenVMS format (a quadword of 100-nanosecond ticks since 17-NOV-1858). A binary time here is an `fao_time_t`, which is `time_t` unless `FAO$T_TIME` is defined otherwise:

- A value greater than 0 is an ABSOLUTE time: the number of seconds since 01-JAN-1970 00:00:00 UTC. The time zone fixed by `FAO$K_TZOFFSET` is added when the time is shown and subtracted when it is read.
- The value 0 is the epoch for the arithmetic routines, but for the display routines it means “no time given”, that is, now.
- A value less than 0 is a DELTA time: its magnitude is the length of the interval in seconds.
- The resolution is one second. Hundredths of a second are accepted in a time string but dropped, and a time is always shown with `.00`.

Consequences:

- The format is the one that `!%T`, `!%D`, `__util$gettim`, `__util$bintim` and `__util$asctim` all use, so a time round-trips between them. It is not converted to the OpenVMS format by any routine.
- An absolute time before 01-JAN-1970 cannot be expressed, because a negative value is a delta. From Version 1.7.0 `__util$bintim` refuses such a date (`FAO$K_IVTIME`; “17-NOV-1858” is an error) instead of returning a value that would be read back as a delta.
- The value 0 is the epoch. It is a valid absolute time for `__util$add_times` and `__util$sub_times` (as the first operand, with a delta), but `!%T`, `!%D` and `__util$asctim` read a 0 as “the current time”, and `__util$bintim` returns 0 for “01-JAN-1970 00:00:00” in the zone UTC.
- On a target whose `time_t` has 32 bits, the range ends in January 2038.
- The year of a date is not limited to 4 digits by the parser, and a year above 9999 is shown in full. A date past the range of the type is not detected.

The routines `__util$add_time`, `__util$sub_time` and `__util$cmp_time` of `utility_routines.h` use another representation: the POSIX `struct timespec` (seconds and nanoseconds).

__util$gettim – Get Binary Time

Format

```
int __util$gettim (fao_time_t *a_timadr);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_timadr`	`fao_time_t *`	write only	by reference	Receives the current time

Description

Stores the current time as seconds since 01-JAN-1970 00:00:00 UTC in `*a_timadr`. The time zone offset is not applied. The source is `FAO$GETTIME()`, which is `time(NULL)` by default.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The time was stored
`STS$K_ERROR`	`a_timadr` is NULL

Example

```
fao_time_t now;

if ( 1 & __util$gettim (&now) )
    __util$fao ("started !%D", &len, &dsc, (fao_prm_t) now);
```

__util$bintim – Convert ASCII Time to Binary Time

Format

```
int __util$bintim (const char *a_timstr, fao_time_t *a_timadr);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_timstr`	`const char *`	read only	by reference	Zero-terminated time string
`a_timadr`	`fao_time_t *`	write only	by reference	Receives the binary time. Not changed on an error

Description

Blanks and tabs at the start and the end of the string are ignored. Three forms are accepted.

Absolute date: `dd-MMM-yyyy[ hh:mm[:ss[.cc]]]`. The month is three letters, in any case. The day is validated against the length of the month, with the Gregorian leap-year rule. The year is a decimal number as written; a two-digit year is not expanded. A missing day or year is replaced with the one of the current date, so `-JAN-2020` and `15-OCT-` are valid; the dashes stay, and the month stays mandatory (`15-OCT` is invalid). A missing time of day is 00:00:00. The string is read in the zone of `FAO$K_TZOFFSET`.

Absolute time of day: `hh:mm[:ss[.cc]]`. The date is the current date. A field of the time of day can be left out: `::10` is 00:00:10 and `12:30` is 12:30:00.

Delta: `dddd hh:mm[:ss[.cc]]` or `dddd-hh:mm[:ss[.cc]]`. A day count, then a blank or a dash, then a time of day. The result is negative: `-(dddd*86400 + hh*3600 + mm*60 + ss)`. A time of day without days, such as `12:30`, is an absolute time, not a delta. Use `0 12:30` for a delta of 12 hours 30 minutes.

Limits: hours 0 to 23, minutes 0 to 59, seconds 0 to 59, hundredths 0 to 99. A number has at most 9 digits. The hundredths are checked and dropped. A delta has no limit on days other than the range of the type.

An absolute date before 01-JAN-1970 is refused with `FAO$K_IVTIME`: a negative value would be read as a delta (Section 3.3). A date of 01-JAN-1970 00:00:00 in the zone UTC gives 0.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The string was converted
`FAO$K_IVTIME`	Syntax error, a field out of range, trailing text, or an absolute date before 1970
`STS$K_ERROR`	`a_timstr` or `a_timadr` is NULL

Example

```
fao_time_t t;

__util$bintim ("15-OCT-2018 10:20:30.55", &t);   /* t == 1539598830 (zone UTC) */
```

```
__util$bintim ("1 02:03:04", &t);               /* t == -93784 (a delta)      */
__util$bintim ("31-FEB-2020", &t);              /* FAO$K_IVTIME, t unchanged  */
__util$bintim ("17-NOV-1858", &t);              /* FAO$K_IVTIME, t unchanged  */
```

__util$asctim – Convert Binary Time to ASCII

Format

```
int __util$asctim (unsigned short *a_outlen, fao_desc_t *a_outbuf,
                   const fao_time_t *a_timadr, int a_cvtflg);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_outlen`	`unsigned short *`	write only	by reference	Receives the length of the result. May be NULL
`a_outbuf`	`fao_desc_t *`	read only (descriptor), write only (buffer)	by reference	Output buffer
`a_timadr`	`const fao_time_t *`	read only	by reference	The time. If NULL, the current time is used
`a_cvtflg`	`int`	read only	by value	0: full form. Any other value: time of day only

Description

Time	`a_cvtflg` = 0	`a_cvtflg` non-zero
Absolute (> 0)	`dd-MMM-yyyy hh:mm:ss.00` (23 bytes; day blank-padded)	`hh:mm:ss.00` (11 bytes)
Delta (< 0)	`d hh:mm:ss.00` (days without padding)	`hh:mm:ss.00`, the days dropped
Zero, or `a_timadr` NULL	The current time, as an absolute one	The current time of day

The renderer is the one of `!%D` and `!%T`, so the two always agree. The result is not zero-terminated. The output is accepted by `__util$bintim`, except that the value 0 prints the current time, not 01-JAN-1970, and a year above 9999 is written in full, so the text is longer.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The time was converted
`FAO$K_BUFOVF`	The buffer is too small; the result is truncated
`STS$K_ERROR`	`a_outbuf` or its buffer pointer is NULL

Example

```
char            buf [32];
fao_desc_t      dsc = $FAO_DESC (buf);
```

```
unsigned short len;
fao_time_t     t = -93784;

__util$asctim (&len, &dsc, &t, 0);    /* "1 02:03:04.00", len 13 */
```

__util$add_times – Add Two Binary Times

Format

```
int __util$add_times (const fao_time_t *a_tim1, const fao_time_t *a_tim2,
                      fao_time_t *a_result);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_tim1`	`const fao_time_t *`	read only	by reference	First operand
`a_tim2`	`const fao_time_t *`	read only	by reference	Second operand
`a_result`	`fao_time_t *`	write only	by reference	Receives the sum. Not changed on an error

Description

An operand that is >= 0 is absolute; an operand that is < 0 is a delta. The rules are:

Operands	Result
delta + delta	A delta: the sum of the (negative) values
absolute + delta	An absolute time: the absolute value plus the magnitude of the delta
absolute + absolute	Error `FAO$K_IVTIME`

There is no check for overflow of the type.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The sum was stored
`FAO$K_IVTIME`	Both operands are absolute
`STS$K_ERROR`	An argument is NULL

Example

```
fao_time_t t0 = 1000, d = -100, r;

__util$add_times (&t0, &d, &r);     /* r == 1100 */
```

__util$sub_times – Subtract Two Binary Times

Format

```
int __util$sub_times (const fao_time_t *a_tim1, const fao_time_t *a_tim2,
                      fao_time_t *a_result);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_tim1`	`const fao_time_t *`	read only	by reference	Minuend
`a_tim2`	`const fao_time_t *`	read only	by reference	Subtrahend
`a_result`	`fao_time_t *`	write only	by reference	Receives the difference. Not changed on an error

Description

Operands	Result
absolute - absolute	A delta, `-(tim1 - tim2)`; `FAO$K_NEGTIM` if tim1 is earlier than tim2
absolute - delta	An absolute time, tim1 minus the magnitude of the delta; `FAO$K_NEGTIM` if it would be below 0
delta - delta	A delta, `tim1 - tim2` (the larger delta minus the smaller); `FAO$K_NEGTIM` if the magnitude of tim1 is smaller than that of tim2
delta - absolute	Error `FAO$K_IVTIME`

If two absolute times are equal, the result is 0, which is an absolute time, not a delta of zero length.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The difference was stored
`FAO$K_NEGTIM`	The result would be negative
`FAO$K_IVTIME`	A delta minus an absolute time
`STS$K_ERROR`	An argument is NULL

Example

```
fao_time_t a = 1000, b = 400, r;

__util$sub_times (&a, &b, &r);      /* r == -600: a delta of 600 seconds */
```

__util$add_time – Add Two Timespecs

This routine, and the next three, are inline functions of `utility_routines.h`. They work on POSIX `struct timespec` values, not on `fao_time_t`.

Format

```
inline static void __util$add_time (const struct timespec *time1,
                                    const struct timespec *time2,
                                    struct timespec *result_time);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
`time1`	`const struct timespec *`	read only	by reference	First operand
`time2`	`const struct timespec *`	read only	by reference	Second operand
`result_time`	`struct timespec *`	write only	by reference	Receives the sum

Description

Adds the seconds and the nanoseconds, and carries whole seconds out of the nanoseconds. The operands must have `tv_nsec` in the range 0 to 999999999. The result may share its address with an operand. There is no check for overflow of `tv_sec`.

Condition Values Returned

None.

Example

```
struct timespec a = { 1, 700000000 }, b = { 2, 500000000 }, r;

__util$add_time (&a, &b, &r);       /* r = { 4, 200000000 } */
```

__util$sub_time – Subtract Two Timespecs

Format

```
inline static void __util$sub_time (const struct timespec *time1,
                                    const struct timespec *time2,
                                    struct timespec *result_time);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
time1	const struct timespec *	read only	by reference	Minuend
time2	const struct timespec *	read only	by reference	Subtrahend
result_time	struct timespec *	write only	by reference	Receives time1 - time2

Description

Subtracts the seconds and the nanoseconds, and normalizes the result: tv_nsec is always in the range 0 to 999999999, and when the nanoseconds of time1 are less than those of time2 a second is borrowed from tv_sec. Before Version 1.7.0 the result was left unnormalized, with a negative tv_nsec. A negative difference (time1 earlier than time2) has a negative tv_sec and a non-negative tv_nsec, as timespec values usually do.

Condition Values Returned

None.

Example

```
struct timespec a = { 5, 100 }, b = { 3, 400 }, r;

__util$sub_time (&a, &b, &r);       /* r = { 1, 999999700 }: normalized */
```

__util$cmp_time – Compare Two Timespecs

Format

```
inline static int __util$cmp_time (const struct timespec *time1,
                                   const struct timespec *time2);
```

Returns

`int`, write only, by value. Zero (0) if the times are equal, 1 if `time1` is later than `time2`, -1 if `time1` is earlier.

Arguments

Argument	Type	Access	Mechanism	Description
`time1`	`const struct timespec *`	read only	by reference	First time
`time2`	`const struct timespec *`	read only	by reference	Second time

Description

The seconds are compared first and the nanoseconds second; nothing is subtracted, so nothing can overflow. The result is exactly -1, 0 or 1. Before Version 1.7.0 the routine returned a difference cut to `int`, whose sign could flip for times more than about 68 years apart; a caller that tests the sign (`< 0`, `> 0`) is unaffected.

Condition Values Returned

None. The return value is not a condition value.

Example

```
struct timespec deadline = { 100, 0 }, now = { 100, 5 };

if ( __util$cmp_time (&deadline, &now) < 0 )
    handle_timeout ();       /* deadline is earlier than now: cmp_time gave -1 */
```

__util$timbuf – Break Time into a Calendar Structure

Format

```
inline static void __util$timbuf (time_t *__src, struct tm *__dst);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
__src	time_t *	read only	by reference	The time to convert. If NULL, the current time is used
__dst	struct tm *	write only	by reference	Receives the broken-down LOCAL time

Description

The routine calls localtime_r() (localtime_s() on Windows) and then adds 1 to tm_mon and 1900 to tm_year. So tm_mon is 1 to 12 and tm_year is the full year. This differs from the convention of struct tm, and the result must not be passed to mktime() or asctime(). The time zone is the local zone of the process, unlike the routines of starlet.h, which use UTC and the fixed FAO$K_TZOFFSET.

Condition Values Returned

None.

Example

```
struct tm tm;

__util$timbuf (NULL, &tm);
/* tm.tm_year == 2026, tm.tm_mon == 10 in October */
```

3.4 FAO in Logging and Message Calls

The macros `$LOG_FAO` and `$PUTMSG_FAO` of `utility_routines.h` pass an FAO control string and its parameters to `__util$faol`. They count the parameters (up to 16) and cast each to `fao_prm_t`, so a call reads `$LOG_FAO (STS$K_INFO, "if=!AZ rx=!UL peer=!%I", "eth0", 777, l_ip4);`. They are described in Chapter 2. The routine `__util$faohex`, which formats up to 16 octets as one line of hexadecimal and ASCII text, is not an FAO directive; it is also described in Chapter 2.

3.5 Differences from OpenVMS

Item	OpenVMS	STARLET
Binary time	100 ns ticks since 17-NOV-1858	Seconds since 01-JAN-1970 UTC; delta is negative
Parameters of `SYS$FAOL`	Longwords	`fao_prm_t`, 64 bits, quadwords and times by value
Count of parameters in `$FAOL`	Not given	Given by `a_prmcnt`
Output length	Word in a descriptor	`outlen` argument; the descriptor is not changed
`!%T`, `!%D` parameter	Address of a quadword; 0 = current time	The time by value; 0 = current time
Hundredths of a second	Real	Always `00`
`!%M !%I !%6 !%U`	Not in FAO	Added

3.6 Changes in Version 1.7.0

The 1.6.11 edition of this manual listed defects of this module. They are corrected. What a consumer can notice:

1. A control string that ends inside a directive (`"ab!"`, `"!A"`, `"!%"`, `"!5"`, `"!n*"`) no longer makes the scanner read past the terminating zero. The directive sets `FAO$K_BADDIR` and the scan stops at the end of the string.
2. The `FAO$K_*` values are condition values. `FAO$K_BADDIR` is now 0x12 (it was 6). `FAO$K_NOPRM` (0x1A) and `FAO$K_FLDOVF` (0x22) are new. The first error wins; it used to be the last.
3. `__util$faol` sets `FAO$K_NOPRM` when the control string wants more parameters than `a_prmcnt`; `__util$fao` sets it for a parameter beyond the 64th. The text is still produced.
4. A `!n<...!>` field longer than 512 bytes is cut with `FAO$K_FLDOVF` instead of silently.
5. `!%D`, `!%T` and `__util$asctim` share one renderer. A negative time is a delta, 0 is “now”, and a year above 9999 is written in full instead of reading outside a table.
6. `__util$bintim` refuses an absolute date before 01-JAN-1970 with `FAO$K_IVTIME`.
7. `__util$sub_time` returns a normalized `timespec`; `__util$cmp_time` returns -1, 0 or 1 and cannot overflow.
8. `starlet.h` includes `utility_routines.h` itself, so the order of the two includes no longer matters, and its fallback definitions of `STS$K_*` are gone.

In 1.7.1: `$LOG_FAO`, `$PUTMSG_FAO` and their debug forms add `[FAO BADDIR]`, `[FAO NOPRM]`, `[FAO FLDOVF]` or `[FAO BUFOVF]` to a record whose text did not format cleanly; the return value is unchanged (Chapter 2, `__util$log_fao`).

What is still true, by design: the time is a count of seconds of the UNIX epoch and not the OpenVMS quadword; the epoch itself cannot be displayed because 0 means “now”; `__util$fao` cannot detect that the caller passed too few parameters, because a C variable argument list carries no count.

Chapter 4. Command Language Routines

This chapter describes the CLI routines of `cli_routines.h` and `cli_routines.c`. They parse a command line into a verb, up to eight positional parameters and any number of qualifiers, check the values, and call the action routine of the verb. The design follows the Command Definition Utility of OpenVMS: the grammar is a set of tables that the program defines.

The grammar is

```
verb [subverb] [p1 p2 ... p8] [/qualifier[=value] ...]
```

A qualifier may also be written in the GNU form `--qualifier[=value]`. A single dash (`-name`) is not a qualifier. Names are matched without regard to case, and a name may be shortened to any prefix that is not ambiguous.

The module includes `utility_routines.h` and `starlet.h`. It uses `ASC`, the counted string of `utility_routines.h`, and `__util$bintim` for the type `CLI$K_TIME`.

Routine	Purpose
`__cli$parse`	Parse an argument vector against the tables and build a context
`__cli$dispatch`	Call the action routine of the matched verb
`__cli$get_value`	Get the value of a parameter or qualifier
`__cli$present`	Test whether a parameter or qualifier was given, and in what form
`__cli$cleanup`	Release a context
`__cli$show_verbs`	List a table of verbs to the log
`__cli$get_keyword`	Resolve a `CLI$K_KWD` value to its entry of the keyword table
`__cli$help`	Print a help screen from the tables
`__cli$show_ctx`	List a context to the log

4.1 The Command Tables

Four structures describe a grammar. A table is an array that ends with a terminator entry, as the following sections give for each kind of table.

4.1.1 ASC: the Counted String

```
typedef struct _asc {
    unsigned char len;        /* length of the text, 0 to 255         */
    char          sts[255];   /* the text; need not be zero-terminated */
} ASC;
```

The macros `$ASCINI("text")` (initializer of the fields `len` and `sts`), `$ASCLEN(p)`, `$ASCPTR(p)` and `$ASC(p)` (length and pointer, for a `%.*s` conversion) belong to `utility_routines.h` (Chapter 2). The values of the context are stored with `__util$str2asc`, which keeps room for the terminating zero: a text longer than 254 bytes is cut.

4.1.2 CLI_KEYWORD: a Keyword

```
typedef struct __cli_keyword__ {
    ASC                name;   /* the keyword                          */
    unsigned long long val;    /* the value associated with it         */
} CLI_KEYWORD;
```

A keyword list ends with an entry whose `name.len` is 0 (`{0}`).

4.1.3 CLI_PQDESC: a Parameter or a Qualifier

```
typedef struct __cli_pqdesc__ {
    ASC            name;     /* name of the parameter or qualifier      */
    unsigned short type;     /* CLI$K_FILE ... CLI$K_MAC                */
    unsigned char  pn;       /* CLI$K_P1 ... CLI$K_P8; CLI$K_QUAL       */
    unsigned char  flag;     /* CLI$M_NEGATABLE, _LIST, _REQUIRED       */
    ASC            defval;   /* default value, empty string = none      */
    CLI_KEYWORD   *kwd;      /* keyword list for CLI$K_KWD, or NULL     */
} CLI_PQDESC;
```

Field	Description
`name`	For a qualifier, the name that the user types after `/` or `--`. For a parameter, a label for the help screen and the messages
`type`	The type of the value. See Section 4.2
`pn`	For a parameter, its position, `CLI$K_P1` to `CLI$K_P8` (1 to 8). For a qualifier, `CLI$K_QUAL` (9) by convention; the parser does not read it
`flag`	A combination of the `CLI$M_*` flags in Section 4.3
`defval`	The value that `__cli$get_value` returns when the item is absent. Not checked against `type`
`kwd`	The list of the keywords of a `CLI$K_KWD` item

A table of PARAMETERS ends with an entry whose `pn` is 0. A table of QUALIFIERS ends with an entry whose `name.len` is 0. Writing `{ .name = {0} }` ends both kinds of table. Parameters must be in the order of their `pn` values, and the position in the table decides which command-line word is bound to the parameter, not the value of `pn`.

4.1.4 CLI_VERB: a Verb

```
typedef struct __cli__verb__ {
    ASC                 name;      /* the verb                              */
    struct __cli__verb__ *next;    /* table of sub-verbs, or NULL           */
    CLI_PQDESC          *params;   /* parameters, or NULL                   */
    CLI_PQDESC          *quals;    /* qualifiers, or NULL                   */
    int (*act_rtn)(struct __cli_ctx__ *clictx, void *act_arg);
    void                *act_arg;  /* handed to the action routine          */
} CLI_VERB;
```

Field	Description
`name`	The verb, matched without regard to case
`next`	A pointer to a TABLE of sub-verbs (an array ended by an entry with an empty name). The name `next` is misleading: it is not a link to the next verb of the same table. A verb with `next` is a container; its own `params`, `quals` and `act_rtn` are not used
`params`	The positional parameters of the verb, or NULL
`quals`	The qualifiers of the verb, or NULL
`act_rtn`	The action routine. It receives the context and `act_arg`, and returns a condition value
`act_arg`	A value for the action routine, such as a command code cast to `void *`

A table of verbs ends with an entry whose `name.len` is 0.

4.1.5 The Context

```
typedef struct __cli_item__ {
    struct __cli_item__ *next;
    unsigned            type;    /* 0 = verb; 1..8 = P1..P8; 9 = qualifier */
    unsigned char       state;   /* CLI$M_PRESENT, _NEGATED, _CONSUMED     */
    union { CLI_VERB *verb; CLI_PQDESC *pqdesc; };
    ASC                 val;     /* the value as typed (a keyword: in full) */
} CLI_ITEM;

typedef struct __cli_ctx__ {
    int       opts;              /* CLI$M_OPTRACE | CLI$M_OPSIGNAL         */
    CLI_ITEM *vlist;             /* the verb and sub-verbs, in order       */
    CLI_ITEM *avlist;            /* parameters and qualifiers, in order    */
} CLI_CTX;
```

The context is built by `__cli$parse`. A program does not write to it. `avlist` holds one item per parameter and one per qualifier occurrence (and one per element of a list qualifier), in the order of the command line. The state flags are in Section 4.3.

4.2 Value Types

The value of a parameter or a qualifier is checked against the field `type` before it is stored. The check is done only if the value is not empty (`/Q=` and `/Q` have no value, and the check is skipped). In the table, “error” is `STS$K_ERROR`.

Constant	Value	Check
`CLI$K_FILE`	1	None; any text
`CLI$K_DATE`	2	Strict. `dd-mm-yyyy` (one or two digits for the day and the month, up to four for the year, a year of at least 1), then optionally a separator (`-`, a blank or `T`) and `hh:mm` or `hh:mm:ss`. The month, the day (with the leap-year rule) and the time of day are checked; text after the value is an error
`CLI$K_NUM`	3	An unsigned number: `strtoull` with base 0 must consume all the text, with no range error: decimal, octal (leading 0) and hexadecimal (leading 0x). A minus sign or a leading blank is an error
`CLI$K_IPV4`	4	`inet_pton(AF_INET)` must succeed (strict dotted decimal)
`CLI$K_IPV6`	5	`inet_pton(AF_INET6)` must succeed
`CLI$K_OPT`	7	None; a flag. A value, if given, is stored and not checked
`CLI$K_QSTRING`	8	None. The quotes, if any, are part of the text; the shell removes them first
`CLI$K_UUID`	9	Exactly 36 characters in the form 8-4-4-4-12 of hexadecimal digits
`CLI$K_DEVICE`	0xA	`stat()` of `/dev/<value>` must succeed; a value that already begins with `/dev/` is used as it is
`CLI$K_KWD`	0xB	The value must select one keyword of the list `kwd` (Section 4.5); what is stored is the full keyword. With no list, anything is accepted
`CLI$K_TIME`	0xC	`__util$bintim` must accept the text: an absolute date, a time of day or a delta
`CLI$K_MAC`	0xD	Six hexadecimal octets of one or two digits separated by `:` or by `-`, and nothing else

The value 6 is not used. `CLI$K_QUAL` is also 9, but it is a kind of item, not a value type, and the two are never mixed. The text of a value is stored as it was typed, except a `CLI$K_KWD` value, which is stored as the full keyword of the table. The check does not convert the value: the program converts it (with `strtoull`, `inet_pton`, `__util$bintim`) when it uses it.

4.3 Flags and Options

Descriptor flags (`CLI_PQDESC.flag`).

Constant	Value	Meaning
`CLI$M_NEGATABLE`	0x01	The qualifier also has the form `/NO<name>`
`CLI$M_LIST`	0x02	The qualifier takes a list of values: `/Q=(a,b,c)` or `/Q=a,b,c`
`CLI$M_REQUIRED`	0x10	The parameter or qualifier must be given

Item state (`CLI_ITEM.state`, returned by `__cli$present`).

Constant	Value	Meaning
`CLI$M_PRESENT`	0x04	The item was given on the command line
`CLI$M_NEGATED`	0x08	It was given in the form `/NO<name>`
`CLI$M_CONSUMED`	0x20	The value was already returned by `__cli$get_value` (list qualifiers)

Processing options (`__cli$parse`, `opts`).

Constant	Value	Meaning
`CLI$M_OPTRACE`	1	Write a trace of the parse with `$IFTRACE`
`CLI$M_OPSIGNAL`	2	Report each error of the parse to the log with `$LOG`. Without it, the routines are silent and only return the status. The “absent” answer of `__cli$get_value` is not an error and is never reported

Sizes. `CLI$SSZ_MAXVERB` (32) is the number of leading characters of the command word that are compared with the verbs. `ASC$K_SZ` (255) is the size of the container of a value; the text itself is at most 254 bytes.

4.4 Parsing Rules

1. `argv[0]` is the verb. This is the first word after the program name, so a `main` passes `argc - 1` and `argv + 1`.
2. If `argv[0]` is a help request (`/HELP`, `-HELP`, `--help`, `/H`, `-h`, `--h`, in any case), the overview of all verbs is printed and the parse returns `STS$K_WARN`. A bare `help` is an ordinary word, so a program may define its own verb `HELP`.
3. The verb is matched against the table by the rules of DCL, the same as `__util$lookup_key` in the abbreviation mode. A verb typed in full wins: with the verbs `SET` and `SETUP`, the word `SET` selects `SET`. Otherwise the word must be a prefix of one verb (`SETU` selects `SETUP`). If it is a prefix of two different verbs, the parse fails with `STS$K_FATAL` ("Ambiguous"). Verb entries that are synonyms - the same action routine and argument, the same parameter, qualifier and sub-verb tables - count as one verb, so two spellings of a command never make an abbreviation ambiguous. An entry with none of these is not a synonym of anything. Only the first `CLI$SSZ_MAXVERB` (32) characters of the word are compared.
4. If any word after the verb is a help request (`/HELP`, `-h`, `--help`), the help screen of the verb is printed and the parse returns `STS$K_WARN` before any value is checked. This applies to the last verb of a nested command: `SHOW STATISTICS /HELP` shows the help of `STATISTICS` (see Section 4.6).
5. If the verb has a sub-verb table (`next`), the next word is matched against that table, by rule 3. Otherwise the following words are bound to the parameters in the order of the table.
6. A parameter takes the next word, with these exceptions: the scan of the parameters stops at a word that is a qualifier of the verb (`/name` or `--name`, whether or not it is shortened, even if ambiguous). So an optional parameter can be omitted in favour of a qualifier. A word that begins with `--` and is none of the qualifiers of the verb is an error (`Unrecognized qualifier`), always. So is a word that begins with `/` and matches no qualifier - unless the parameter waiting for a value is of the type `CLI$K_FILE` or `CLI$K_DEVICE`: then the word is a path (`/tmp`, `/etc/passwd`) and becomes the value of that parameter. A mistyped qualifier is therefore never taken as a value of a text parameter.
7. The rest of the words must be qualifiers. A word that is not a qualifier (it does not start with `/` or `--`) fails the parse with `STS$K_FATAL`. If it follows a qualifier that was written without `=`, the message says that the value must follow `=` (`/blocks 5` is an error, `/blocks=5` is right); otherwise it is "Too many parameters", a surplus word.
8. A qualifier is `/name`, `--name`, `/name=value` or `--name=value`. The name is matched with the table of the verb: an exact name wins; otherwise a prefix that fits one qualifier is accepted; a prefix that fits several fails with `STS$K_FATAL`. If nothing matches and the name starts with `NO` and is longer than two letters, the rest is matched against the qualifiers with `CLI$M_NEGATABLE` only; a match is a negated qualifier, which has no value. An unknown qualifier fails with `STS$K_FATAL`.
9. A value of a `CLI$M_LIST` qualifier is split at commas; one optional pair of parentheses is removed. Each element is checked and stored as an item of its own, up to 254 bytes each. A comma inside quotes is not protected.
10. When a qualifier is repeated, each occurrence is stored. `__cli$present` reports the last one. `__cli$get_value` returns the first one (or, for a list, the occurrences in turn).
11. After the words are consumed, every qualifier and parameter with `CLI$M_REQUIRED` that is absent fails the parse with `STS$K_FATAL`.
12. A verb that has neither parameters nor qualifiers takes no words after it: a surplus word fails the parse with `STS$K_FATAL` ("Too many parameters"), as DCL does.

The result is the condition value of the first error. A failed parse releases the context and sets `*clictx` to NULL.

4.5 Keywords

The keyword list of a `CLI$K_KWD` item is searched by `strncasecmp` for the typed text. A text shorter than a keyword matches it as an abbreviation. An exact match of a whole keyword wins over abbreviations (so `INET` can be typed in full when `INET6` exists). If no exact match is found and two or more keywords begin with the text, the parse fails with `STS$K_FATAL` (“Ambiguous”); if none do, with `STS$K_ERROR`. An empty text is an error.

The value that the parse stores is the FULL keyword of the table, in the spelling of the table, not the abbreviation or the case that the user typed: `/MODE=SL` is stored as `slow`. `__cli$get_value` therefore returns the full name, and a program can compare it with a plain string compare. To reach the field `val` of the keyword as well, call `__cli$get_keyword`, which returns the table entry. In Version 1.6.11 the typed text was stored and the program had to search the table itself.

4.6 Nested Verbs

A container verb has a sub-verb table in `next`. The sub-verb follows its parent at once, with no qualifier between them. Qualifiers after the sub-verb belong to the sub-verb. Nesting may go on to any depth. `__cli$dispatch` calls the action routine of the last (deepest) verb that was matched.

```
static CLI_PQDESC s_stat_q [] = {
    { .name = {$ASCINI("csv")}, .type = CLI$K_OPT, .pn = CLI$K_QUAL },
    { .name = {0} }
};
static CLI_VERB s_show_subs [] = {
    { .name = {$ASCINI("statistics")}, .act_rtn = s_act_stat, .quals = s_stat_q },
    { .name = {$ASCINI("version")},    .act_rtn = s_act_ver },
    { .name = {0} }
};
static CLI_VERB s_cmds [] = {
    { .name = {$ASCINI("show")}, .next = s_show_subs },
    { .name = {0} }
};
```

The words `show statistics --csv` call `s_act_stat`, and `__cli$present (ctx, &s_stat_q[0], &state)` then reports `--csv`. The word `show nonesuch` fails with `STS$K_FATAL`. A help request that stands directly after a container verb (`show /help`) shows the list of its sub-verbs. A help request after a sub-verb (`show statistics /help`) shows the help of that sub-verb: the sub-verb owns the rest of the command line. Before Version 1.7.0 the help of a sub-verb could not be asked for.

__cli$parse – Parse a Command Line

Format

```
int __cli$parse (CLI_VERB *verbs, int opts, int argc, char **argv, void **clictx);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`verbs`	`CLI_VERB *`	read only	by reference	The table of verbs
`opts`	`int`	read only	by value	`CLI$M_OPTRACE` and `CLI$M_OPSIGNAL`, or 0
`argc`	`int`	read only	by value	Number of words, the verb included. Must be at least 1
`argv`	`char **`	read only	by reference	The words; `argv[0]` is the verb
`clictx`	`void **`	write only	by reference	Receives the address of the new `CLI_CTX`

Description

Allocates a context, matches `argv` with the tables by the rules of Section 4.4, checks the values, and stores the items. `*clictx` is set to NULL first, so a caller that calls `__cli$cleanup` after a failure is safe. On success, `*clictx` points to a `CLI_CTX` that the caller releases with `__cli$cleanup`. On any status that is not odd, the context is released inside the routine and `*clictx` is NULL.

The status `STS$K_WARN` means “a help screen was printed”. It is not an error and the context is NULL, but the test `1 & status` is false (the low bit is clear). A caller that must tell help from a failure compares the status with `STS$K_WARN`, and exits with success. In either case the program should not dispatch.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The command line is valid
`STS$K_WARN`	A help request was found and a help screen was printed
`STS$K_FATAL`	`argc` is less than 1; no memory; unknown or ambiguous verb or qualifier; a surplus word (“Too many parameters”); a value after a space instead of `=`; a required item is missing; an ambiguous keyword
`STS$K_ERROR`	A value failed the check of its type (keywords: unknown keyword)

Example

```
void *ctx;
int   sts;

sts = __cli$parse (g_verbs, CLI$M_OPSIGNAL, argc - 1, argv + 1, &ctx);
if ( sts == STS$K_WARN )
    return 0;                       /* help was printed */
if ( !(1 & sts) )
    return 1;                       /* the error is in the log */

sts = __cli$dispatch (ctx);
__cli$cleanup (ctx);
```

__cli$dispatch – Call the Action Routine

Format

```
int __cli$dispatch (CLI_CTX *clictx);
```

Returns

`int`, write only, by value. The value returned by the action routine, or a status of the routine.

Arguments

Argument	Type	Access	Mechanism	Description
`clictx`	`CLI_CTX *`	read only	by reference	A context made by `__cli$parse`. Must not be NULL

Description

Takes the last verb in the verb list of the context (the deepest sub-verb) and calls `act_rtn (clictx, act_arg)`. The action routine reads its values with `__cli$get_value` and `__cli$present`. A verb with no action routine gives `STS$K_WARN`.

Condition Values Returned

Value	Meaning
(the return value of `act_rtn`)	The action routine was called
`STS$K_WARN`	The verb has no action routine
`STS$K_FATAL`	The context holds no verb

Example

```
static int s_act_copy (CLI_CTX *a_ctx, void *a_arg)
{
    ASC l_src;

    if ( !(1 & __cli$get_value (a_ctx, &g_copy_p[0], &l_src)) )
        return STS$K_ERROR;
    /* ... */
    return STS$K_SUCCESS;
}
```

__cli$get_value – Get a Value

Format

```
int __cli$get_value (CLI_CTX *clictx, CLI_PQDESC *pq, ASC *val);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`clictx`	`CLI_CTX *`	read only	by reference	The context
`pq`	`CLI_PQDESC *`	read only	by reference	The descriptor of the item, the SAME address as the entry in the table that was parsed
`val`	`ASC *`	write only	by reference	Receives a copy of the value. May be NULL

Description

The routine searches the item list for `pq` (by address). If it finds an item:

- For an ordinary item, it returns the first occurrence.
- For a `CLI$M_LIST` qualifier, it returns the next element that was not returned before, and marks it as consumed. Call it in a loop until it fails to read all the elements.
- If the stored value has length zero (a flag such as `/VERBOSE`, a negated qualifier, or `/Q=`), the routine returns `STS$K_WARN`, with `*val` set to the empty string. Use `__cli$present` to test a flag.

If there is no item, and `val` is not NULL, and the descriptor has a non-empty `defval`, the routine returns the default with `STS$K_SUCCESS`. A list qualifier that was given and has been read to the end does NOT return the default; it fails, so a loop ends.

If `clictx` or `pq` is NULL, the routine writes a message to the log with `$LOG`, whatever `opts` says, and returns `STS$K_FATAL`. An absent item with no default gives `STS$K_ERROR`. That answer is a normal result of a probe, and from Version 1.7.0 it is never written to the log, with `CLI$M_OPSIGNAL` or without it; before, every probe of an absent item logged an error. The empty value of a present item (`STS$K_WARN`) is still reported with `CLI$M_OPSIGNAL`.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	A value was returned (found, or the default)
`STS$K_WARN`	The item is present but its value is empty
`STS$K_ERROR`	The item is absent and has no default; or the elements of a list are used up
`STS$K_FATAL`	`clictx` or `pq` is NULL

Example

```
ASC l_tag;

while ( 1 & __cli$get_value (ctx, &g_run_q[TAGS], &l_tag) )
    printf ("tag %.*s\n", $ASC (&l_tag));
```

__cli$get_keyword – Resolve a Keyword Value

The Resolve a Keyword Value routine finds the entry of the keyword table that a CLI$K_KWD value belongs to, so that the program can read the val field of the keyword.

Format

```
int __cli$get_keyword (CLI_CTX *clictx, CLI_PQDESC *pq, const ASC *val,
                       CLI_KEYWORD **kwd);
```

Returns

int, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
clictx	CLI_CTX *	read only	by reference	The context
pq	CLI_PQDESC *	read only	by reference	The descriptor of the item, with a keyword list in kwd
val	const ASC *	read only	by reference	A value that __cli$get_value returned, or NULL for the first value of the item (its default if the item is absent)
kwd	CLI_KEYWORD **	write only	by reference	Receives the address of the entry inside the table of pq. Set to NULL first

Description

The parse stores the full keyword of the table (Section 4.5), so the value is looked up by its whole name, without regard to case, and no abbreviation is accepted. *kwd then points into the keyword table of the descriptor: the name is in (*kwd)->name and the value associated with it in (*kwd)->val. With val NULL the routine takes the first value of the item, or the default value of the descriptor if the item is absent. The routine does not advance the cursor of a CLI$M_LIST qualifier, so it can be called between the calls of __cli$get_value.

Nothing is written to the log, whatever opts says, except for a NULL clictx, pq or kwd, which is a programming error.

Condition Values Returned

Value	Meaning
STS$K_SUCCESS	The entry was found
STS$K_WARN	The item is present with an empty value (/mode, /mode=); the same as __cli$get_value says. *kwd is NULL
STS$K_ERROR	The descriptor has no keyword list; the item is absent and has no default; or the value is not a keyword of the table
STS$K_FATAL	clictx, pq or kwd is NULL

Example

A program with a qualifier `/MODE` of the type `CLI$K_KWD`, whose table holds `fast` (1) and `slow` (2). The command `run /mode=SL` gives:

```
static CLI_KEYWORD s_modes [] = {
    { .name = {$ASCINI("fast")}, .val = 1 },
    { .name = {$ASCINI("slow")}, .val = 2 },
    { .name = {0} }
};
/* ... s_quals[0] is { "mode", CLI$K_KWD, .kwd = s_modes } ... */

ASC          l_val;
CLI_KEYWORD *l_kwd;

if ( 1 & __cli$get_value (ctx, &s_quals[0], &l_val) )
    printf ("stored: %.*s\n", $ASC (&l_val));          /* stored: slow          */

if ( 1 & __cli$get_keyword (ctx, &s_quals[0], NULL, &l_kwd) )
    printf ("keyword %.*s, val %llu\n", $ASC (&l_kwd->name), l_kwd->val);
                                                       /* keyword slow, val 2   */
```

__cli$present – Test Whether an Item Is Present

Format

```
int __cli$present (CLI_CTX *clictx, CLI_PQDESC *pq, unsigned *state);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`clictx`	`CLI_CTX *`	read only	by reference	The context
`pq`	`CLI_PQDESC *`	read only	by reference	The descriptor of the item (same address as in the table)
`state`	`unsigned *`	write only	by reference	Receives the state flags. May be NULL

Description

Scans the whole item list and uses the LAST occurrence of `pq`. So `/HIGHLIGHT /NOHIGHLIGHT` is reported as negated. `*state` is set to 0 for an absent item, and otherwise to `CLI$M_PRESENT`, with `CLI$M_NEGATED` if the last occurrence was `/NO<name>`, and `CLI$M_CONSUMED` if it was read. The routine does not use the default value.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The item is present, in the affirmative or in the negated form
`STS$K_WARN`	The item is absent
`STS$K_FATAL`	`clictx` or `pq` is NULL

Example

```
unsigned st;

if ( (1 & __cli$present (ctx, &g_run_q[VERBOSE], &st)) && !(st & CLI$M_NEGATED) )
    l_verbose = 1;
```

__cli$cleanup – Release a Context

Format

```
int __cli$cleanup (CLI_CTX *clictx);
```

Returns

`int`, write only, by value. Always `STS$K_SUCCESS`.

Arguments

Argument	Type	Access	Mechanism	Description
`clictx`	`CLI_CTX *`	modify	by reference	The context. May be NULL. It must not be used after the call

Description

Frees the items of both lists and the context. The tables are not touched. The `ASC` values that `__cli$get_value` copied out remain valid, because they are copies.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

Example

```
__cli$cleanup (ctx);
ctx = NULL;
```

__cli$show_verbs – List a Table of Verbs

Format

```
void __cli$show_verbs (CLI_VERB *verbs, int level);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
`verbs`	`CLI_VERB *`	read only	by reference	The table of verbs
`level`	`int`	read only	by value	Indentation level of the first verb; 0 at top level

Description

Writes the verbs, their sub-verbs, parameters (with type), keywords (with `val` in hexadecimal) and qualifiers to the log with `$LOG (STS$K_INFO, ...)`. It is meant for debugging a grammar. At level 0 the line of a verb reads `Show verb 'NAME' :;` at a deeper level it is the name, indented by two spaces per level. The sub-verb table is listed once, one level deeper than its parent; the level of the caller is not changed, so the verbs that follow a container verb in the same table are written in the same form as the ones before it.

Condition Values Returned

None.

Example

```
__cli$show_verbs (g_verbs, 0);
```

__cli$help – Print a Help Screen

Format

```
int __cli$help (CLI_VERB *verbs, CLI_VERB *verb);
```

Returns

`int`, write only, by value. Always `STS$K_SUCCESS`.

Arguments

Argument	Type	Access	Mechanism	Description
`verbs`	`CLI_VERB *`	read only	by reference	The top table, used when `verb` is NULL
`verb`	`CLI_VERB *`	read only	by reference	The verb to describe, or NULL for the overview

Description

The text goes through `__util$out`, without a prefix. `__cli$parse` calls this routine when it finds a help request; a program may call it too.

- `verb` is NULL: "Available commands:" and the name of each verb.
- `verb` has sub-verbs: "Usage: NAME ..." and a list of the sub-verbs.
- Otherwise, a "Usage:" line with the parameters (required ones bare, others in brackets) and "[/qualifier ...]" if the verb has qualifiers, then a list of the parameters (position, name, type) and of the qualifiers. A qualifier line says "required", "/NO accepted", "list" and "default=..." as the flags and `defval` say. The usage line is cut to 512 bytes.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	Always

Example

```
__cli$help (g_verbs, NULL);           /* the list of commands    */
__cli$help (g_verbs, &g_verbs[1]);    /* the help of one command */
```

__cli$show_ctx – List a Context

Format

```
void __cli$show_ctx (CLI_CTX *clictx);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
clictx	CLI_CTX *	read only	by reference	The context to show. Must not be NULL

Description

Writes the verb list and each parameter (Pn) and qualifier value of the context to the log with $LOG (STS$K_INFO, ...). It is for debugging.

Condition Values Returned

None.

Example

```
__cli$show_ctx (ctx);
```

4.7 Changes in Version 1.7.0

The 1.6.11 edition of this manual listed defects of this module. They are corrected. What a consumer can notice:

1. A verb is matched by the DCL rules: typed in full it wins (`SET` against `SETUP`), a unique prefix selects, synonyms never make a prefix ambiguous.
2. A verb with neither parameters nor qualifiers rejects a surplus word ("Too many parameters").
3. The help of a sub-verb can be asked for (`SHOW STATISTICS /HELP`).
4. A `CLI$K_KWD` value is stored as the full keyword; the new `__cli$get_keyword` returns the table entry, so `val` is reachable.
5. An unmatched `/name` is taken as a path only for a pending `CLI$K_FILE` or `CLI$K_DEVICE` parameter; for any other parameter, and always for `--name`, it is an error. Before, it became the value of the next optional parameter.
6. A surplus word is "Too many parameters" instead of "Unrecognized token"; a value after a space (`/blocks 5`) has its own message, and is still an error.
7. `CLI$K_DATE` and `CLI$K_NUM` are strict (Section 4.2).
8. `__cli$get_value` does not log the "absent" answer, so a probe is quiet under `CLI$M_OPSIGNAL`.
9. `__cli$show_verbs` restores its level, `__cli$show_ctx` is declared in the header, and the comment of `CLI_PQDESC.flag` is correct.

In 1.7.1 `__cli$get_keyword` returns `STS$K_WARN` for an item present with an empty value (`/mode`, `/mode=`), as `__cli$get_value` does; absent stays `STS$K_ERROR`.

What is still true, by design: `/blocks 5` needs `=`; `__cli$parse` returns `STS$K_WARN` after it printed a help screen, and the low bit of that status is clear, so the program must test for `STS$K_WARN` before it tests for failure; `__cli$dispatch` and the routines not named in this chapter do not check their context for NULL.

Chapter 5. TLV Protocol Routines

This chapter describes AVProto, the Attribute/Value (Tag-Length-Value) protocol of `avproto.h` and `avproto.c`. A protocol data unit (PDU) is a fixed header followed by a payload of TLV records. All the numbers in a PDU are in network byte order (big-endian), so a PDU can cross machines of either byte order. The routines are named `__tlv$...`.

Routine	Purpose
`__tlv$hset`	Initialize the header of a PDU
`__tlv$hget`	Check the signature of a PDU and read its header
`__tlv$encode_tag`	Pack a tag type and a tag identifier into the tag word
`__tlv$decode_tag`	Unpack a tag word
`__tlv$put`	Append a TLV record to a PDU
`__tlv$get`	Find a record by tag and copy its value out
`__tlv$lookup`	Find a record by tag and return the address of its value
`__tlv$dump`	List the header and the records of a PDU to the log

`avproto.c` includes `utility_routines.h`; the programs that use the routines compile and link both modules. On Windows, `avproto.h` supplies `be16toh`, `htobe16` and the other conversions through the `_byteswap_*` functions, because `<endian.h>` is not there.

5.1 The PDU Layout

A PDU is the header (20 octets) and the payload. The payload is a series of records with nothing between them: no padding and no alignment. The field `len` of the header is the length of the payload in octets, the header not included.

```
 octet
 0       8       12      16      20
 +-------+-------+-------+-------+---------------------------------+
 | magic | len   | csr   | seq   | payload: TLV 0, TLV 1 ... TLV N |
 | 8     | 4     | 4     | 4     | len octets                      |
 +-------+-------+-------+-------+---------------------------------+
   signature, zero-padded to 8 octets
           |       |       +-- sequence number of the PDU
           |       +-- command / status word
           +-- length of the payload, octets (big-endian)
```

Field	Size	Description
`magic`	8 octets	The signature of the protocol, a string of up to 8 octets, padded with zeros
`len`	4 octets	Length of the payload. Big-endian. `__tlv$put` keeps it up to date
`csr`	4 octets	Command / Status Register: a command in a request, a status in a reply. Big-endian
`seq`	4 octets	Sequence number of the PDU, to help match the replies of parallel or asynchronous exchanges. Big-endian

The C types are `AVPROTO_HDR` (the header), `AVPROTO_TLV` (a record header with a zero-length array for the value) and `AVPROTO_PDU` (a header followed by a zero-length array of records). They are packed (`#pragma pack(1)`). The fields `u_len`, `u_csr` and `u_seq` of the header and the fields `w_tag` and `w_len` of a record hold network-order values; convert them with `be32toh` and `be16toh`, or read them through the routines.

5.1.1 The Record

```
 bit   0          3 4                   15 16                          31
       +----------+----------------------+------------------------------+
       |   type   |          id          |            length            |
       |  4 bits  |       12 bits        |           16 bits            |
       +----------+----------------------+------------------------------+
       |  value: `length` octets (none if length is 0)                  |
       +-----------------------------------------------------------------+
        |<------- w_tag (16 bits) ------->|<-------- w_len (16 bits) ----->|
```

Both 16-bit words are in network byte order. The length counts the octets of the value only. The next record starts right after the value. A record is therefore 4 + `length` octets long, and a value can hold up to 65535 octets.

The tag. `w_tag` holds the type in the 4 high bits and the identifier in the 12 low bits. The masks are `TAG$M_TYPE` (0xF000) and `TAG$M_ID` (the complement of 0xF000). The identifier (0 to 4095) is chosen by the application. The type tells how to convert the value.

Constant	Value	Value of the record
TAG$K_BBLOCK	0	A block of octets; no conversion. The default type
TAG$K_WORD	3	16-bit number, big-endian, length 2
TAG$K_LONGWORD	4	32-bit number, big-endian, length 4
TAG$K_QWORD	5	64-bit number, big-endian, length 8
TAG$K_UUID	8	16 octets, no conversion, length 16
TAG$K_MAX	9	End-of-list marker of the enumeration; not a type

The type numbers 1, 2, 6, 7 and 9 to 15 have no name. __tlv$put and __tlv$get treat them like TAG$K_BBLOCK.

5.1.2 An Example

These calls

```
__tlv$hset (pdu, (const unsigned char *) "TSTSIG", 0x1234, 42);
__tlv$put  (pdu, sizeof pdu, 1,   TAG$K_WORD,   &w,   sizeof w);    /* w == 0xABCD */
__tlv$put  (pdu, sizeof pdu, 100, TAG$K_BBLOCK, blob, 3);           /* 01 02 03   */
```

build this PDU of 33 octets:

```
 offset  octets                      meaning
 ------  --------------------------  ----------------------------------
      0  54 53 54 53 49 47 00 00      magic "TSTSIG" + two zero octets
      8  00 00 00 0D                  len = 13
     12  00 00 12 34                  csr = 0x1234
     16  00 00 00 2A                  seq = 42
     20  30 01                        tag: type 3 (WORD), id 1
     22  00 02                        length 2
     24  AB CD                        value 0xABCD
     26  00 64                        tag: type 0 (BBLOCK), id 100
     28  00 03                        length 3
     30  01 02 03                     value
```

__tlv$hset – Set the PDU Header

Format

```
inline static int __tlv$hset (void *a_pdup, const unsigned char *a_sig,
                              unsigned a_u_csr, unsigned a_u_seq);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_pdup`	`void *`	write only	by reference	The PDU buffer; at least 20 octets
`a_sig`	`const unsigned char *`	read only	by reference	The signature, a zero-terminated string of 1 to 8 octets
`a_u_csr`	`unsigned`	read only	by value	The command/status word, in host order
`a_u_seq`	`unsigned`	read only	by value	The sequence number, in host order

Description

Clears the 20 octets of the header, copies the signature into `magic` (the rest of the field stays zero), and stores `csr` and `seq` in network order. The payload length is 0. Call it once before the first `__tlv$put`. A signature that is NULL or longer than 8 octets is refused, exactly as `__tlv$hget` refuses it, and the PDU is not touched; before Version 1.7.0 a long signature was cut to 8 octets without notice. The routine does not check the size of the buffer.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The header was set
`STS$K_ERROR`	`a_sig` is NULL or longer than 8 octets

Example

```
unsigned char pdu [512];

__tlv$hset (pdu, (const unsigned char *) "VCLOUD", CMD_LOGIN, ++seq);
```

__tlv$hget – Get the PDU Header

Format

```
inline static int __tlv$hget (void *a_pdup, const unsigned char *a_sig,
                              unsigned *a_u_len, unsigned *a_u_csr,
                              unsigned *a_u_seq);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_pdup`	`void *`	read only	by reference	The received PDU; at least 20 octets
`a_sig`	`const unsigned char *`	read only	by reference	The expected signature, a zero-terminated string of at most 8 octets
`a_u_len`	`unsigned *`	write only	by reference	Receives the payload length, host order
`a_u_csr`	`unsigned *`	write only	by reference	Receives the command/status word, host order
`a_u_seq`	`unsigned *`	write only	by reference	Receives the sequence number, host order

Description

Compares the signature, zero-padded to 8 octets, with `magic`. If they are equal, it converts the three fields to host order and stores them. If not, it returns `STS$K_ERROR` and none of the three output variables is written. (Version 1.6.11 stored `STS$K_ERROR` in `*a_u_csr` on a mismatch.) All three pointers must be valid: none may be NULL.

The routine checks neither that the received buffer holds 20 octets nor that `len` fits in what was received. Use the real size of the buffer with `__tlv$get` and `__tlv$lookup`, which clamp `len` to it.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The signature matched; the three fields are stored
`STS$K_ERROR`	The signature is NULL or longer than 8 octets, or it does not match

Example

```
unsigned char rxbuf [512], sig [] = "VCLOUD";   /* rxbuf: a received PDU */
unsigned      len, csr, seq;

if ( !(1 & __tlv$hget (rxbuf, sig, &len, &csr, &seq)) )
    return STS$K_ERROR;             /* not our protocol */
```

__tlv$encode_tag – Pack a Tag

Format

```
inline static unsigned short __tlv$encode_tag (unsigned short a_v_type,
                                               unsigned short a_v_tag);
```

Returns

`unsigned short`, write only, by value. The tag word in NETWORK byte order, ready to store in `w_tag`.

Arguments

Argument	Type	Access	Mechanism	Description
`a_v_type`	`unsigned short`	read only	by value	The type, a `TAG$K_*` constant. Only the low 4 bits fit
`a_v_tag`	`unsigned short`	read only	by value	The identifier, 0 to 4095

Description

Returns `htobe16 ((a_v_type << 12) | a_v_tag)`. The routine does not mask the arguments. An identifier above 4095 changes the type bits, and a type above 15 is lost in the shift.

Condition Values Returned

None. The return value is a tag word, not a condition value.

Example

```
unsigned short w = __tlv$encode_tag (TAG$K_WORD, 1);   /* octets 30 01 in memory */
```

__tlv$decode_tag – Unpack a Tag

Format

```
inline static unsigned short __tlv$decode_tag (unsigned short a_w_tag,
                                               unsigned short *a_v_type,
                                               unsigned short *a_v_tag);
```

Returns

`unsigned short`, write only, by value. The tag word in HOST byte order.

Arguments

Argument	Type	Access	Mechanism	Description
`a_w_tag`	`unsigned short`	read only	by value	The tag word as it is stored in the PDU (network order)
`a_v_type`	`unsigned short *`	write only	by reference	Receives the type, 0 to 15
`a_v_tag`	`unsigned short *`	write only	by reference	Receives the identifier, 0 to 4095

Description

Converts the word to host order, then splits it: the high 4 bits are the type and the low 12 bits are the identifier. Both pointers must be valid.

Condition Values Returned

None.

Example

```
unsigned short type, id;

__tlv$decode_tag (rec->w_tag, &type, &id);
```

__tlv$put – Append a Record

Format

```
int __tlv$put (void *a_pdu, unsigned a_pdusz, unsigned a_v_tag,
               unsigned a_v_type, void *a_val, unsigned a_valsz);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_pdu`	`void *`	modify	by reference	The PDU, with the header set by `__tlv$hset`
`a_pdusz`	`unsigned`	read only	by value	The size of the whole buffer, header included (an `unsigned` from Version 1.7.0; it was `unsigned short`)
`a_v_tag`	`unsigned`	read only	by value	The identifier, 0 to 4095
`a_v_type`	`unsigned`	read only	by value	The type, a `TAG$K_*` constant, 0 to 15
`a_val`	`void *`	read only	by reference	The value, in HOST order for the numeric types. May be NULL only if the record is empty
`a_valsz`	`unsigned`	read only	by value	The size of the value in octets. For a fixed-size type at least the size of the type

Note the order of the two arguments: the identifier first, the type second.

Description

Adds one record at the end of the payload (at offset `len` of the header), and then adds the size of the record to `len`. The record needs `4 + n` octets, where `n` is the real length of the value (below), and the space left is `a_pdusz - 20 - len`. The check is made against the real length, so the routine never writes outside the buffer, and nothing is written on an error.

By type:

- `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD`: the value is a 16-, 32- or 64-bit number in host order. The routine converts it to network order. The length field is 2, 4 or 8. `a_valsz` must be at least that size; only the size of the type is stored. A `a_valsz` of 0, which was allowed before Version 1.7.0, is now an error.
- `TAG$K_UUID`: 16 octets are copied as they are. The length field is 16 and `len` of the header grows by 20. `a_valsz` must be at least 16.

- Any other type: `a_valsz` octets are copied as they are; the length field is `a_valsz`, 0 to 65535.

The identifier must be 0 to 4095 and the type 0 to 15; otherwise nothing fits its bits and the call fails (the identifier used to be silently masked). The routine checks that the header `len` fits in the buffer. It does not write the signature. It writes a message to the log with `$LOG` for each error, whatever the options are.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The record was appended
`STS$K_ERROR`	`a_pdusz` is below 20; the identifier is above 4095 or the type above 15; `a_valsz` is too small for a fixed-size type, or above 65535 for a block; `a_val` is NULL for a non-empty value; the header `len` is larger than the payload area; the record does not fit

Example

```
uint32_t id  = 4711;
char     name[] = "Ruslan";

__tlv$hset (pdu, sig, CMD_LOGIN, ++seq);
__tlv$put  (pdu, sizeof pdu, TAG_ID,   TAG$K_LONGWORD, &id,  sizeof id);
__tlv$put  (pdu, sizeof pdu, TAG_NAME, TAG$K_BBLOCK,   name, strlen (name));
```

__tlv$get – Get a Value

Format

```
int __tlv$get (void *a_pdu, unsigned a_pdusz, int *a_context, unsigned a_v_tag,
               unsigned *a_v_type, void *a_val, unsigned *a_valsz);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_pdu`	`void *`	read only	by reference	The PDU
`a_pdusz`	`unsigned`	read only	by value	The real size of the buffer, in octets: the number of octets received
`a_context`	`int *`	modify	by reference	The search position; -1 to start. May be NULL
`a_v_tag`	`unsigned`	read only	by value	The identifier to find
`a_v_type`	`unsigned *`	write only	by reference	Receives the type of the record. Must not be NULL
`a_val`	`void *`	write only	by reference	The buffer that receives the value
`a_valsz`	`unsigned *`	modify	by reference	On input, the size of `a_val`; on output, the number of octets stored. May be NULL for the numeric types only

Description

Searches the records from the position in `*a_context` (or from the start) for the first one whose identifier equals `a_v_tag`; the type is not compared. The search limit is the smaller of the payload length in the header and the number of octets really present (`a_pdusz - 20`), so a false header cannot make the routine read outside the buffer. The routine stops at the first record that does not fit within the limit and reports it as `STS$K_ERROR`: a truncated record is an error and not “absent” (before Version 1.7.0 it gave `STS$K_WARN`).

The context is an octet offset from the start of the payload. It is -1 (or the routine is called with NULL) to start at the first record. After each call, `*a_context` is set to the offset just behind the last record that was looked at, so the next call with the same context finds the next record of the same identifier. A NULL context always starts at the beginning.

When the record is found, the value is copied to `a_val`:

- `TAG$K_WORD`, `TAG$K_LONGWORD`, `TAG$K_QWORD`: converted to host order; the size stored is 2, 4 or 8.

- `TAG$K_UUID`: exactly 16 octets.
- Any other type: copied as it is; the size stored is the length of the record.

`*a_valsz` is set to the number of octets stored. For every type except the numeric ones, `a_valsz` is required and must hold the size of the destination; a value that is longer than that is not copied and the routine returns `STS$K_ERROR`. For the numeric types a NULL `a_valsz` means that the buffer is large enough for the type.

The length field of a numeric record must be equal to 2, 4 or 8, and that of a UUID record to 16. Any other length is a malformed record, and the routine returns `STS$K_ERROR` without copying anything. Before Version 1.7.0 such a record was read past its value.

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The record was found and its value copied
`STS$K_WARN`	No record with this identifier in the rest of the PDU
`STS$K_ERROR`	`a_pdusz` below 20; a record runs past the PDU; the length of a fixed-size record is wrong; `*a_valsz` is smaller than the record; `a_valsz` is NULL for a type that needs it; `a_val` or `a_v_type` is NULL

Example

```
int      ctx = -1;
unsigned type, sz;
char     item [64];

for ( ;; )
{
    sz = sizeof item;
    if ( !(1 & __tlv$get (rx, rxlen, &ctx, TAG_ITEM, &type, item, &sz)) )
        break;
    printf ("item: %.*s\n", (int) sz, item);
}
```

__tlv$lookup – Look Up a Record

Format

```
int __tlv$lookup (void *a_pdu, unsigned a_pdusz, int *a_context, unsigned a_v_tag,
                  unsigned *a_v_type, void **a_val, unsigned *a_valsz);
```

Returns

`int`, write only, by value. A condition value.

Arguments

Argument	Type	Access	Mechanism	Description
`a_pdu`	`void *`	read only	by reference	The PDU
`a_pdusz`	`unsigned`	read only	by value	The real size of the buffer, in octets
`a_context`	`int *`	modify	by reference	The search position; -1 to start. May be NULL
`a_v_tag`	`unsigned`	read only	by value	The identifier to find
`a_v_type`	`unsigned *`	write only	by reference	Receives the type. Must not be NULL
`a_val`	`void **`	write only	by reference	Receives the address of the value inside the PDU. Must not be NULL
`a_valsz`	`unsigned *`	write only	by reference	Receives the length of the value. May be NULL

Description

The search is the same as in `__tlv$get`, but nothing is copied and nothing is converted: the address returned points into the PDU, and the value is in network order. The address is valid as long as the buffer of the PDU is. It may be unaligned, so read a number from it with `memcpy`, followed by `be16toh`, `be32toh` or `be64toh`. This routine is the right choice for a large block. The length of a fixed-size record is checked as `__tlv$get` does (word 2, longword 4, quadword 8, UUID 16): a record of another length is `STS$K_ERROR` and no address is given. (Up to 1.7.0 the address of a short record was returned and the caller had to check `*a_valsz`.)

Condition Values Returned

Value	Meaning
`STS$K_SUCCESS`	The record was found
`STS$K_WARN`	No such record in the rest of the PDU
`STS$K_ERROR`	`a_pdusz` is below 20; a record runs past the PDU; a fixed-size record has a wrong length

Example

```
void    *p;
unsigned type, sz;
int      ctx = -1;

if ( 1 & __tlv$lookup (rx, rxlen, &ctx, TAG_BLOB, &type, &p, &sz) )
    process_blob (p, sz);
```

__tlv$dump – List a PDU

Format

```
void __tlv$dump (void *a_pdu, unsigned a_pdusz);
```

Returns

None.

Arguments

Argument	Type	Access	Mechanism	Description
a_pdu	void *	read only	by reference	The PDU
a_pdusz	unsigned	read only	by value	The real size of the buffer, in octets

Description

Writes one line for the header (`PDU [len=..., csr=..., seq=...]`) and one for each record to the log with `$LOG (STS$K_INFO, ...)`. The record line shows the index, the tag word, the identifier and the type, the length, and the record header and the value in hexadecimal. A value longer than 64 octets is listed in chunks of 64 octets, the continuation lines starting with `+offset`, so a long value cannot overflow the buffer (before Version 1.7.0 a value of 256 octets or more wrote one byte past it). A record that does not fit within the limit is reported as truncated and ends the listing. It is for debugging.

Condition Values Returned

None.

Example

```
__tlv$dump (pdu, sizeof (AVPROTO_HDR) + len);
```

5.2 Changes in Version 1.7.0

The 1.6.11 edition of this manual listed defects of this module. They are corrected. What a consumer can notice:

1. `__tlv$put` checks the free space by the real length of the record and never writes past the buffer. A fixed-size type needs `a_valsz` of at least its size; 0 is refused. A UUID record adds 20 octets to `len`, so the header and the record agree.
2. `__tlv$put` refuses an identifier above 4095 and a type above 15 instead of masking them. Its buffer size is an `unsigned`.
3. `__tlv$get` checks the wire length of a numeric or UUID record and reports a wrong one, and a record that runs past the PDU, as `STS$K_ERROR`. `__tlv$lookup` reports a record that runs past the PDU in the same way.
4. `__tlv$dump` lists a long value in chunks of 64 octets.
5. `__tlv$hset` refuses a signature over 8 octets (or NULL); `__tlv$hget` leaves its output variables alone on an error.
6. `avproto.h` no longer switches off compiler warnings (`-Wparentheses`, `-Wdate-time`, `-Wunused-variable`) for every file that includes it.

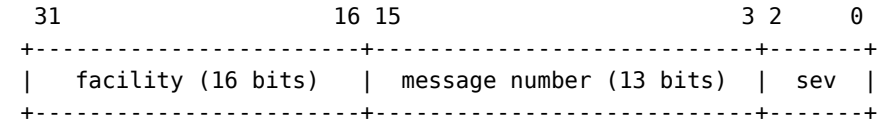
In 1.7.1 `__tlv$lookup` checks the wire length of a fixed-size record as `__tlv$get` does.

What is still true: `__tlv$lookup` returns an address without converting it, and `__tlv$hset` and `__tlv$hget` do not check the size of the buffer they are given.

Appendix A. Condition Values and Severity

The routines of STARLET return a 32-bit condition value in an `int` or an `unsigned`. The convention is that of OpenVMS: bit 0 set means success, bit 0 clear means failure. The test is `1 & status`. The severity constants and the macros are defined in `utility_routines.h`. From Version 1.7.0 the header `starlet.h` includes `utility_routines.h` itself, and no longer repeats the severity constants. The header `starlet.h` defines the `FAO$K_*` values, and `utility_routines.h` the `UTIL$S_*` values of the queue routines and of `__util$inimsg`; the header `cli_routines.h` and the header `avproto.h` define no condition values and use the ones of this appendix.

A.1 Layout of a Condition Value



Field	Bits	Width	Description
Severity	0 to 2	3	One of the `STS$K_*` values below
Message number	3 to 15	13	The number of the message within its facility
Facility	16 to 31	16	The number of the facility. The default facility of the utility module, `__FAC$_UTILS__`, is 0x01

There are no control bits: the facility field takes the 16 high bits. A message table registered with `__util$inimsg` is found by the facility and message fields; see Chapter 2.

Because the severity values 1, 3, 5 and 7 are odd, they all read as success under the test `1 & status`. In practice only `STS$K_SUCCESS` and `STS$K_INFO` are used as success.

A.2 Severity Values

Constant	Value	Letter (STS$C__*)	Success?	Meaning
STS$K_WARN	0	W	no	Warning; also "not found" and "absent" in several routines
STS$K_SUCCESS	1	S	yes	Success
STS$K_ERROR	2	E	no	Error
STS$K_INFO	3	I	yes	Information
STS$K_FATAL	4	F	no	Severe error
STSK_UNDEF, STSK_UNDEF5	5	? (STS$C__UNDEF)	yes	Not defined; __util$log and similar routines show an unknown severity as ? and return STS$K_ERROR for it (up to 1.7.0, this value)
STS$K_UNDEF6	6	none	no	Not defined
STS$K_UNDEF7	7	none	yes	Not defined
STS$K_MAX	8	none	no	End-of-list marker

STS$M_SYSLOG (16) is not a part of a condition value. It is a flag for the severity argument of the logging routines: it asks them to send the message also to the syslog service.

A.3 Macros

All these macros are in `utility_routines.h`. The argument is a condition value, or a number, as stated.

Macro	Result
`$SEV(code)`	`(code) & 0x7`: the severity
`$MSG(code)`	`((code) >> 3) & 0x1FFF`: the message number
`$FAC(code)`	`(code) >> 16`: the facility. It is not masked, so the argument must be 32-bit unsigned
`$DEFSEV(n)`	`(n) & 0x7`: a severity, to build a value
`$DEFMSG(n)`	`((n) << 3) & 0xFFF8`: a message number placed in bits 3 to 15
`$DEFFAC(n)`	`(n) << 16`: a facility placed in bits 16 to 31
`$STS_ERROR(code)`	True if the severity is `STS$K_ERROR` or `STS$K_FATAL`

A value is built by adding the three parts:

```
#define MY$_NOFILE   ( $DEFFAC (0x20) | $DEFMSG (3) | $DEFSEV (STS$K_ERROR) )

$SEV (MY$_NOFILE) == STS$K_ERROR;   /* 2    */
$MSG (MY$_NOFILE) == 3;
$FAC (MY$_NOFILE) == 0x20;
```

A.4 Values of the Library Modules

The completion codes of the FAO and time routines (`starlet.h`) and the codes `UTIL$S_*` of the queue routines (`utility_routines.h`) follow the layout of Section A.1 without a facility: the severity is in the three low bits and the message number above it. Every one of them is even, so `1 & status` is false. They were chosen for that (the history of `starlet.c` records the change from odd values). Their numbers are stable from Version 1.7.0 on, with one exception: `FAO$K_BADDIR` was 6 in Version 1.6.11 and `UTIL$S_NOLOCK` was 0x202.

Constant	Value	`$SEV`	`$MSG`	Used by
`FAO$K_NEGTIM`	0x08	0 (warn)	1	`__util$sub_times`: negative result
`FAO$K_BUFOVF`	0x0A	2 (error)	1	`__util$fao`, `__util$faol`, `__util$asctim`: truncated result
`FAO$K_IVTIME`	0x0C	4 (fatal)	1	`__util$bintim`, `__util$add_times`, `__util$sub_times`: invalid time
`FAO$K_BADDIR`	0x12	2 (error)	2	`__util$fao`, `__util$faol`: unknown, malformed or truncated directive
`FAO$K_NOPRM`	0x1A	2 (error)	3	`__util$fao`, `__util$faol`: too few parameters
`FAO$K_FLDOVF`	0x22	2 (error)	4	`__util$fao`, `__util$faol`: a `!n<...!>` field was cut
`UTIL$S_INVARG`	0x102	2 (error)	0x20	The queue routines and `__util$inimsg`: a NULL or unusable argument
`UTIL$S_INQUE`	0x202	2 (error)	0x40	`__util$insqhead`, `__util$insqtail`: the entry is on another queue
`UTIL$S_NOLOCK`	0x302	2 (error)	0x60	The queue routines: the spin lock was not acquired
`UTIL$S_DUPFAC`	0x400	0 (warn)	0x80	`__util$inimsg`: a catalogue of the facility is registered already

The status of the FAO routines is the first error found in the call (Section 3.1). The queue routines return `STS$K_SUCCESS` also for an empty queue in `__util$remqhead` and `__util$remqtail`; the caller tells the empty queue by `*ent` being NULL (Chapter 2).

A configuration file read by `__util$readconfig`, or named by an `OPTS$K_CONF` option of `__util$getparams`, may hold an option the table does not know (the file may serve several programs). The option is skipped and logged, and the routine returns `STS$K_INFO`; its low bit is set, so the test `1 & status` still says success. A bad value, a line that does not start with `-` or `/`, and an unknown option on the command line are `STS$K_ERROR` (Chapter 2).

The routines of the CLI and the TLV module return only `STS$K_SUCCESS`, `STS$K_WARN`, `STS$K_ERROR` and `STS$K_FATAL`.

Module	`STS$K_SUCCESS`	`STS$K_WARN`	`STS$K_ERROR`	`STS$K_FATAL`
`__cli$parse`	valid	help printed	a value failed its type check, or an unknown keyword	syntax, lookup, ambiguity, surplus word or missing item
`__cli$get_value`	value returned	empty value	absent or used up (never logged)	NULL argument
`__cli$get_keyword`	entry found	empty value	not a keyword, absent, no list	NULL argument
`__cli$present`	present	absent	not used	NULL argument
`__tlv$hset`	header set	not used	bad signature	not used
`__tlv$hget`	signature matched	not used	no match, or a bad signature	not used
`__tlv$put`	appended	not used	does not fit, or a bad argument	not used
`__tlv$get`	copied	not found	bad area, buffer or record	not used
`__tlv$lookup`	found	not found	bad area or record	not used

Appendix B. Index of Routines and Macros

The table lists the routines, macros, types and constants of all four modules in alphabetical order (the leading underscores and `$` are ignored in the ordering). "Ch." is the chapter that describes the item. The four time helpers `__util$add_time`, `__util$sub_time`, `__util$cmp_time` and `__util$timbuf` belong to the module `utility_routines` but are described in Chapter 3.

Name	Purpose	Module	Ch.
`$ARRSZ`	Number of elements of an array	utility_routines	2
`ASC`	Counted string (length byte and 255 characters)	utility_routines	2
`$ASC`	Length and address of an ASC, for a %.*s conversion	utility_routines	2
`ASC$K_SZ`	Longest counted string (255)	utility_routines	2
`$ASCDECL`	Declare and initialize an ASC variable	utility_routines	2
`$ASCINI`	Initializer fields of an ASC	utility_routines	2
`$ASCLEN`	Length of an ASC	utility_routines	2
`$ASCNIL`	Empty ASC initializer	utility_routines	2
`$ASCPTR`	Address of the text of an ASC	utility_routines	2
`$ATOMIC_DEC`	Atomic decrement (fetch and subtract 1)	utility_routines	2
`$ATOMIC_INC`	Atomic increment (fetch and add 1)	utility_routines	2
`AVPROTO_HDR`	Structure: PDU header (20 octets)	avproto	5
`AVPROTO_PDU`	Structure: header followed by records	avproto	5
`AVPROTO_TLV`	Structure: TLV record header	avproto	5
`$BIN2HEX`	Macro form of __util$bin2hex	utility_routines	2
`__cli$cleanup`	Release a parse context	cli_routines	4

Name	Purpose	Module	Ch.
`__cli$dispatch`	Call the action routine of the matched verb	cli_routines	4
`__cli$get_keyword`	Resolve a keyword value to its table entry	cli_routines	4
`__cli$get_value`	Get the value of a parameter or qualifier	cli_routines	4
`__cli$help`	Print a help screen from the tables	cli_routines	4
`CLI$K_DATE`	Value type: date	cli_routines	4
`CLI$K_DEVICE`	Value type: device in /dev	cli_routines	4
`CLI$K_FILE`	Value type: file name	cli_routines	4
`CLI$K_IPV4`	Value type: IPv4 address	cli_routines	4
`CLI$K_IPV6`	Value type: IPv6 address	cli_routines	4
`CLI$K_KWD`	Value type: keyword	cli_routines	4
`CLI$K_MAC`	Value type: MAC address	cli_routines	4
`CLI$K_NUM`	Value type: number (decimal, octal, hex)	cli_routines	4
`CLI$K_OPT`	Value type: flag with no value	cli_routines	4
`CLI$K_P1 ...` `CLI$K_P8`	Positions of the parameters (1 to 8)	cli_routines	4
`CLI$K_QSTRING`	Value type: quoted string	cli_routines	4
`CLI$K_QUAL`	Item kind of a qualifier (9)	cli_routines	4
`CLI$K_TIME`	Value type: absolute or delta time	cli_routines	4
`CLI$K_UUID`	Value type: UUID	cli_routines	4
`CLI$M_CONSUMED`	State: list value was already returned	cli_routines	4
`CLI$M_LIST`	Flag: qualifier takes a list	cli_routines	4
`CLI$M_NEGATABLE`	Flag: qualifier has a /NO form	cli_routines	4
`CLI$M_NEGATED`	State: item was given as /NO	cli_routines	4
`CLI$M_OPSIGNAL`	Option: report errors to the log	cli_routines	4
`CLI$M_OPTRACE`	Option: trace the parse	cli_routines	4
`CLI$M_PRESENT`	State: item was given	cli_routines	4
`CLI$M_REQUIRED`	Flag: item must be given	cli_routines	4
`__cli$parse`	Parse an argument vector against the command tables	cli_routines	4

Name	Purpose	Module	Ch.
`__cli$present`	Test whether an item was given, and how	cli_routines	4
`__cli$show_ctx`	List a context to the log	cli_routines	4
`__cli$show_verbs`	List a table of verbs to the log	cli_routines	4
`CLI$SSZ_MAXVERB`	Length of the verb compared (32)	cli_routines	4
`CLI_CTX`	Structure: the parse context	cli_routines	4
`CLI_ITEM`	Structure: a parsed item	cli_routines	4
`CLI_KEYWORD`	Structure: a keyword and its value	cli_routines	4
`CLI_PQDESC`	Structure: a parameter or qualifier	cli_routines	4
`CLI_VERB`	Structure: a verb	cli_routines	4
`$CLRQUE`	Macro form of __util$clrqueue	utility_routines	2
`$CMP_STORE_LONG`	Atomic compare-and-store of a longword	utility_routines	2
`$DEFSEV, $DEFMSG, $DEFFAC`	Build the fields of a condition value	utility_routines	2
`$DUMPHEX`	Macro form of __util$dumphex	utility_routines	2
`EMSG_RECORD, EMSG_RECORD_DESC`	Structures of a message table	utility_routines	2
`ENTRY, __QUEUE`	Structures of a queue and its entries	utility_routines	2
`FAO$GETTIME`	Compile-time source of the current time	starlet	3
`FAO$K_BADDIR`	Condition value: bad, unknown or truncated directive (0x12)	starlet	3
`FAO$K_BUFOVF`	Condition value: output truncated	starlet	3
`FAO$K_FLDBUF`	Limit: the longest text of a !n<...!> field (512)	starlet	3
`FAO$K_FLDOVF`	Condition value: a !n<...!> field was cut	starlet	3
`FAO$K_IVTIME`	Condition value: invalid time	starlet	3
`FAO$K_MAXDEPTH`	Limit: nesting depth of FAO groups (4)	starlet	3
`FAO$K_MAXPRM`	Limit: parameters cached by __util$fao (64)	starlet	3
`FAO$K_NEGTIM`	Condition value: negative time result	starlet	3

Name	Purpose	Module	Ch.
FAO$K_NOPRM	Condition value: too few parameters	starlet	3
FAO$K_TZOFFSET	Compile-time time zone offset in seconds	starlet	3
FAO$T_TIME	Compile-time type behind fao_time_t	starlet	3
$FAO_DESC	Initializer of an output descriptor over a character array	starlet	3
fao_desc_t	Output buffer descriptor	starlet	3
fao_prm_t	Type of an FAO parameter (64-bit, by value)	starlet	3
fao_time_t	Binary time type (seconds since 01-JAN-1970; negative = delta)	starlet	3
$IFLOG	Macro: write a log record if a condition is true	utility_routines	2
$IFTRACE	Macro: write a trace record if a condition is true	utility_routines	2
ILE2, ILE3, $ILE2INI, $ILE3INI, $ILENULL	Item list structures; $ILE3INI is a statement macro	utility_routines	2
$INSQHEAD	Macro form of __util$insqhead	utility_routines	2
$INSQTAIL	Macro form of __util$insqtail	utility_routines	2
$ISINRANGE	Macro form of __util$isinrange	utility_routines	2
$ISINRANGE2	Macro form of __util$isinrange2	utility_routines	2
KWDENT	Structure: a keyword table entry	utility_routines	2
$LOCK_LONG	Macro form of __util$lockspin	utility_routines	2
$LOG	Macro: write a log record at the call site; returns the severity	utility_routines	2
$LOG_FAO	Macro: write a log record with FAO control string	utility_routines	2
$MAX	Macro form of __util$max	utility_routines	2
$MIN	Macro form of __util$min	utility_routines	2
$MOVQHEAD	Macro form of __util$movqhead	utility_routines	2
$MOVQTAIL	Macro form of __util$movqtail	utility_routines	2

Name	Purpose	Module	Ch.
OPTS$K_INT ... OPTS$K_PWD	Types of an option value	utility_routines	2
OPTS, OPTS_NULL	Structure of an option table entry	utility_routines	2
$PUTMSG	Macro: write the message of a condition value	utility_routines	2
$PUTMSG_FAO	Macro: write the message of a condition value with FAO arguments	utility_routines	2
$RANGE	Macro form of __util$range	utility_routines	2
$REMQENT	Macro form of __util$remqent	utility_routines	2
$REMQHEAD	Macro form of __util$remqhead	utility_routines	2
$REMQTAIL	Macro form of __util$remqtail	utility_routines	2
s___time	Internal: current time as a timespec	utility_routines	2
$SEV, $MSG, $FAC	Extract severity, message number, facility (Appendix A)	utility_routines	2
STS$C__WARN ... STS$C__UNDEF	One-letter severity codes	utility_routines	2
STS$K_WARN ... STS$K_FATAL, STSK_UNDEF, STSK_MAX	Severity values (Appendix A)	utility_routines	2
STS$M_SYSLOG	Flag of a severity argument: also send to syslog	utility_routines	2
$STS_ERROR	True for an error or fatal condition value	utility_routines	2
SYM_REC, $SYM_REC_INI, $SYM_REC_EOL	Entry of a symbol table for mask2sym and val2sym	utility_routines	2
TAG$K_BBLOCK	Tag type: octet block (0)	avproto	5
TAG$K_LONGWORD	Tag type: 32-bit number (4)	avproto	5
TAG$K_MAX	End-of-list marker of the tag types	avproto	5
TAG$K_QWORD	Tag type: 64-bit number (5)	avproto	5
TAG$K_UUID	Tag type: UUID (8)	avproto	5
TAG$K_WORD	Tag type: 16-bit number (3)	avproto	5
TAG$M_ID	Mask of the identifier bits of a tag word	avproto	5
TAG$M_TYPE	Mask of the type bits of a tag word (0xF000)	avproto	5
__tlv$decode_tag	Unpack a tag word	avproto	5
__tlv$dump	List a PDU to the log	avproto	5

Name	Purpose	Module	Ch.
`__tlv$encode_tag`	Pack a tag type and identifier	avproto	5
`__tlv$get`	Find a record and copy its value out	avproto	5
`__tlv$hget`	Check the signature and read the header of a PDU	avproto	5
`__tlv$hset`	Initialize the header of a PDU	avproto	5
`__tlv$lookup`	Find a record and return the address of its value	avproto	5
`__tlv$put`	Append a TLV record to a PDU	avproto	5
`$TRACE`	Macro: write a trace record	utility_routines	2
`$UNLOCK_LONG`	Macro form of __util$unlockspin	utility_routines	2
`__util$add_time`	Add two timespec values	utility_routines	3
`__util$add_times`	Add two binary times	starlet	3
`__util$asc2str`	Copy a counted string to a zero-terminated string	utility_routines	2
`__util$asctim`	Convert a binary time to an ASCII string	starlet	3
`__util$base64`	Encode octets in Base64 (RFC 4648)	utility_routines	2
`__util$bin2hex`	Convert octets to hexadecimal text	utility_routines	2
`__util$bintim`	Convert an ASCII time string to binary time	starlet	3
`__util$bzero`	Clear a memory block	utility_routines	2
`__util$clrqueue`	Empty a queue	utility_routines	2
`__util$cmp_time`	Compare two timespec values (-1, 0, 1)	utility_routines	3
`__util$cmpasc`	Compare two ASC	utility_routines	2
`__util$cmpasc_blind`	Compare two ASC, case-blind	utility_routines	2
`__util$collapse`	Remove all blanks and tabs from a string	utility_routines	2
`__util$cpyasc`	Copy an ASC to an ASC	utility_routines	2
`__util$crc32c`	CRC-32 of a buffer (the IEEE polynomial, despite the name)	utility_routines	2
`__util$deflog`	Select the log file or the syslog host	utility_routines	2

Name	Purpose	Module	Ch.
__util$dumphex	Write a hex dump of a buffer to the log	utility_routines	2
__util$fao	Format a control string with a variable parameter list	starlet	3
__util$faohex	Format up to 16 octets as one hex and ASCII line	utility_routines	2
__util$faol	Format a control string with a parameter array	starlet	3
__util$fill	Fill a buffer by repeating a pattern	utility_routines	2
__util$getmsg	Find the message record of a condition value	utility_routines	2
__util$getparams	Fill an option table from the command line	utility_routines	2
__util$gettim	Return the current binary time	starlet	3
__util$hex2bin	Convert hexadecimal text to octets	utility_routines	2
__util$inimsg	Register a table of messages	utility_routines	2
__util$insqhead	Insert an entry at the head of a queue	utility_routines	2
__util$insqtail	Insert an entry at the tail of a queue	utility_routines	2
__util$isinrange	Test that a value lies in a range, borders included	utility_routines	2
__util$isinrange2	Test that a value lies in a range, borders included, in either order	utility_routines	2
__util$iszero	Test a buffer for all zero octets	utility_routines	2
UTIL$K_LOOKUP_ABBREV	Option of __util$lookup_key: allow abbreviations	utility_routines	2
UTIL$K_LOOKUP_NCASE	Option of __util$lookup_key: ignore case	utility_routines	2
__util$lockspin	Take a spin lock	utility_routines	2
__util$log	Write a log record (printf format)	utility_routines	2
__util$log2buf	Format a log record into a caller buffer	utility_routines	2
__util$log_fao	Write a log record (FAO control string)	utility_routines	2
__util$logd	As __util$log, with module, function and line	utility_routines	2

Name	Purpose	Module	Ch.
`__util$logd_fao`	As __util$log_fao, with module, function and line	utility_routines	2
`__util$lookup_key`	Translate a keyword to a value using a table	utility_routines	2
`__util$mask2sym`	Translate a bit mask to text	utility_routines	2
`__util$max`	Larger of two integers	utility_routines	2
`__util$min`	Smaller of two integers	utility_routines	2
`__util$movc5`	Move characters with fill (LIB$MOVC5)	utility_routines	2
`__util$movqhead`	Move an entry (`void *`) to the head of its queue	utility_routines	2
`__util$movqtail`	Move an entry (`void *`) to the tail of its queue	utility_routines	2
`__util$nope`	Empty function used when tracing is compiled out	utility_routines	2
`__util$out`	Write an unprefixed line to the log destination	utility_routines	2
`__util$pattern_match`	Match a string with a wildcard pattern	utility_routines	2
`__util$putmsg`	Write the message of a condition value (printf arguments)	utility_routines	2
`__util$putmsg_fao`	Write the message of a condition value (FAO arguments)	utility_routines	2
`__util$putmsgd`	As __util$putmsg, with module, function and line	utility_routines	2
`__util$putmsgd_fao`	As __util$putmsg_fao, with module, function and line	utility_routines	2
`__util$q_unlink`	Internal: take an entry out of its queue chain	utility_routines	2
`__util$range`	Bound a value to a range	utility_routines	2
`__util$readconfig`	Fill an option table from a configuration file	utility_routines	2
`__util$remqent`	Remove a given entry from a queue	utility_routines	2
`__util$remqhead`	Remove the head entry of a queue	utility_routines	2
`__util$remqtail`	Remove the tail entry of a queue	utility_routines	2

Name	Purpose	Module	Ch.
__util$rewindlogfile	Truncate the log file past a size	utility_routines	2
__util$rotatelogfile	Rotate the log file past a size	utility_routines	2
UTIL$S_DUPFAC	Condition value: catalogue of the facility registered already (0x400)	utility_routines	2
__util$setlogfd	Send the log to an open file descriptor	utility_routines	2
__util$showparams	Show the options table	utility_routines	2
UTIL$S_INQUE	Condition value: entry on another queue (0x202)	utility_routines	2
UTIL$S_INVARG	Condition value: unusable argument (0x102)	utility_routines	2
UTIL$S_NOLOCK	Condition value: spin lock not acquired (0x302)	utility_routines	2
__util$str2asc	Copy a zero-terminated string to an ASC	utility_routines	2
__util$strcat_rx	Concatenate a list of strings into one area	utility_routines	2
__util$strelem	Get a delimited element of a string	utility_routines	2
__util$strstr	Find a substring in a counted buffer	utility_routines	2
__util$strxor	Exclusive-or a string with a key	utility_routines	2
__util$sub_time	Subtract two timespec values (normalized)	utility_routines	3
__util$sub_times	Subtract two binary times	starlet	3
__util$syslog	Send a message to a syslog server (a quiet success with no host)	utility_routines	2
__util$timbuf	Break a time_t into a local struct tm with natural year and month	utility_routines	3
__util$trace	Write a trace record if a condition is true	utility_routines	2
__util$trim	Remove leading and trailing blanks	utility_routines	2
__util$uint32_2_dec	Convert an unsigned 32-bit number to decimal text	utility_routines	2
__util$unbase64	Decode Base64	utility_routines	2
__util$uncomment	Remove a comment from a line, quoted parts skipped	utility_routines	2

Name	Purpose	Module	Ch.
`__util$unlockspin`	Release a spin lock	utility_routines	2
`__util$uuid2str`	Convert a UUID to text	utility_routines	2
`__util$val2sym`	Translate a value to text	utility_routines	2